Universidade Federal do Piauí
Centro de Educação Aberta e a Distância

QUALIDADE DE *SOFTWARE*

Prof. Msc. Moacir Orsano Pereira
Prof. Msc. Gilcifran Vieira de Sousa



Ministério da Educação - MEC
Universidade Aberta do Brasil - UAB
Universidade Federal do Piauí - UFPI
Universidade Aberta do Piauí - UAPI
Centro de Educação Aberta e a Distância - CEAD

Qualidade de *Software*

Prof. Msc. Moacir Orsano Pereira
Prof. Msc. Gilcifran Vieira De Sousa

2010

PRESIDENTE DA REPÚBLICA	*Luiz Inácio Lula da Silva*
MINISTÉRIO DA EDUCAÇÃO	*Fernando Haddad*
GOVERNADOR DO ESTADO	*Wilson Nunes Martins*
REITOR DA UNIVERSAIDADE FEDERAL DO PIAUÍ	*Luiz de Sousa Santos Júnior*
SECRETÁRIO DE EDUCAÇÃO A DISTÂNCIA DO MEC	*Carlos Eduardo Bielshowsky*
PRESIDENTE DA CAPES	*Jorge Almeida Guimarães*
COORDENADORIA GERAL DA UNIVERSIDADE ABERTA DO BRASIL	*Celso Costa*
DIRETOR DO CENTRO DE EDUCAÇÃO ABERTA A DISTÂNCIA DA UFPI	*Gildásio Guedes Fernandes*

CONSELHO EDITORIAL	*Prof. Dr. Ricardo Alaggio Ribeiro (Presidente)*
	Des. Tomaz Gomes Campelo
	Prof. Dr. José Renato de Araújo Sousa
	Profª. Drª. Teresinha de Jesus Mesquita Queiroz
	Profª. Francisca Maria Soares Mendes
	Profª. Iracildes Maria de Moura Fé Lima
	Prof. Dr. João Renór Ferreira de Carvalho

COORDENAÇÃO DE MATERIAL DIDÁTICO	*Cleidinalva Maria Barbosa Oliveira*
COORDENAÇÃO DE REVISÃO	*Naziozênio Antonio Lacerda*
PROJETO GRÁFICO	*Samuel Falcão Silva*
DIAGRAMAÇÃO	*Mariane de Meneses Costa Alves*
REVISÃO	*Lígia Carvalho de Figueiredo*
REVISOR GRÁFICO	*Genuvina de Lima Melo Neta*

A responsabilidade pelo conteúdo e imagens desta obra é dos autores. O conteúdo desta obra foi licenciado temporária e gratuitamente para utilização no âmbito do Sistema Universidade Aberta do Brasil, através da UFPI. O leitor se compromete a utilizar o conteúdo desta obra para aprendizado pessoal, sendo que a reprodução e distribuição ficarão limitadas ao âmbito interno dos cursos. A citação desta obra em trabalhos acadêmicos e/ou profissionais poderá ser feita com indicação da fonte. A cópia deste obra sem autorização expressa ou com intuito de lucro constitui crime contra a propriedade intelectual, com sansões previstas no Código Penal.

Apresentação

Este livro apresenta um amplo estudo sobre a qualidade de *software*. Escrito para estudantes de diversas áreas de informática, entre as quais sistemas de informação, processamento de dados e computação. Utilizado como um primeiro texto de qualidade de *software* que exige como pré-requisito conhecimentos básico de informática e desenvolvimento de programas.

Obter qualidade nos processos e produtos de *software* não é uma tarefa trivial. São vários os fatores que dificultam atingir os objetivos de qualidade. No entanto, nada mais decepcionante do que produzir *software* que não satisfaça as necessidades dos clientes. Acreditamos que ao atacarmos a definição e gerência de requisitos estaremos colaborando de maneira fundamental para a qualidade geral do *software*.

A principal implementação feita neste material é apresentar uma sequência consistente de técnicas e métodos que visam a construção econômica de programas com certo nível de qualidade e bem sucedido – quando satisfaz as necessidades das pessoas que usam, tem desempenho sem falhas. Os métodos e processos apresentados neste livro estão calcados no que vem sendo utilizado por diversos desenvolvedores de *software*.

Todos nós desejamos construir *softwares* que tornem as coisas melhores evitando os problemas que espreitam na sombra dos esforços. Neste material foram escolhidos os métodos e técnicas mais eficazes, melhor fundamentadas e que têm tido uma história de sucesso no uso de desenvolvimento de *software*.

Este livro está organizado em 06 capítulos, a saber: o Capítulo 1 apresenta uma introdução ao Papel Evolutivo do *Software*. O Capítulo 2 trata da Engenharia de *software*: Uma Tecnologia em Camadas; Gestão

de Qualidade Total; Um arcabouço de processo como alicerce para um processo de *software* completo; O modelo em estágios do CMMI: as metas e práticas específicas associadas a um conjunto de áreas de processo.

O Capítulo 3 aborda o Panorama Sobre Modelos Prescritivos de Processo; Problemas do Ciclo de Vida Clássico; Realimentação entre fases, prototipagem - modelo de processo de desenvolvimento de *software*; O modelo espiral para a engenharia de *software*, representação do modelo concorrente através de uma série de atividades e seus estados correspondentes, desenvolvimento baseado em componentes, modelos de métodos formais que levam à especificação matemática formal do *software* de computador.

O Capítulo 4 apresenta a definição de Qualidade de *Software*, os padrões e procedimentos em uma organização; Gerenciamento da qualidade do *software*; Implantação da Função de Qualidade - traduz as necessidades do cliente para requisitos técnicos do *software*; processo de projeto e qualidade de projeto; fatores de qualidade de MCCALL; impacto na qualidade; produtividade e custo.

O Capítulo 5, por sua vez, trata das Métricas de Qualidade de *Software*, eficiência na remoção de defeitos - métrica de qualidade, que fornece benefício tanto no nível de projeto quanto de processo, integração de métricas no processo de *software*; argumentos favoráveis a métricas de *software*_medir o processo de engenharia de processo e o produto (*software*) que ele produz; estabelecimento de uma referência para as métricas, os benefícios obtidos nos níveis (técnicos) de processo, projeto e produto.

O Capítulo 6 aborda a análise e os Conceitos de Qualidade; controle de qualidade; garantia de qualidade - consiste de um conjunto de funções para auditar e relatar que avalia a efetividade e completeza das atividades do controle de qualidade, o custo da qualidade decorrente da busca da qualidade ou da execução das atividades relacionadas à qualidade, além de discutir as revisões de *software* e o impacto nos custos de defeitos de *software*.

Neste livro, os conceitos básicos envolvidos são enfatizados na medida do possível na apresentação das matérias. Demonstrações da qualidade de *software* são fornecidas à medida que contribuem para a compreensão do tema apresentado. Todo o material é organizado visando a um desenvolvimento gradual da teoria de controle.

Ao longo do livro, vários exercícios são apresentados no final

de cada capítulo. Esses exercícios constituem uma parte importante do texto. Dessa maneira, cabe ao estudante resolver cuidadosamente esses exercícios e observar os temas para discussão para compreender bem a matéria.

Sumário

13 **UNIDADE 1**
O PAPEL EVOLUTIVO DO SOFTWARE

Software 17
Software não "se desgasta" 18
A Natureza Mutável do *Software* 21
Software Legado 24
A Qualidade do *Software* Legado 25
Evolução de *Software* 26
Mitos do *Software* *27*
Como Tudo Começa 30
Processo: Uma Visão Genérica 31

35 **UNIDADE 2**
ENGENHARIA DE SOFTWARE – UMA TECNOLOGIA EM CAMADAS

Um Arcabouço de Processo 39
O CMMI (*Capatibility Maturity Model Integration*) 42
Padrões de Processo 46
Avaliação de Processo 49
Modelos de Processo Pessoal e de Equipe 50
PSP (*Personal Software Process* – Processo Pessoal de *Software*) 51
TSP (*Team Software Process* – Processo de Equipe de Processo) 52
Tecnologia de Processo 54
Ferramentas de Modelagem de Processo 55
Produto e Processo 56

61 **UNIDADE 3**
PANORAMA SOBRE MODELOS PRESCRITIVOS DE PROCESSO

Modelo Clássico / Em Cascata 64

Problemas do Ciclo de Vida Clássico 67
Realimentação entre fases 68
RAD (Rapid Application Development)..... 69
Prototipagem..... 70
O modelo de Prototipagem..... 72
Espiral ou Evolutivo 73
Concorrente..... 76
Desenvolvimento baseado em componentes 78
O modelo de métodos formais..... 78
Desenvolvimento de Software orientado a aspectos 79
Processo Unificado (PU) 79

89 **UNIDADE 4**
QUALIDADE DE SOFTWARE

Gerenciamento da qualidade do software consiste 92
Implantação da Função de Qualidade..... 96
Processo de Projeto e Qualidade de Projeto..... 97
Qualidade de Software..... 100
Fatores de Qualidade de McCall..... 100
Impacto na Qualidade, Produtividade e Custo..... 102

107 **UNIDADE 5**
MÉTRICAS DE QUALIDADE DE SOFTWARE

Medição de Qualidade 109
Eficiência na Remoção de Defeitos..... 111
Integração de Métricas no Processo de Software 112
Argumentos Favoráveis a Métricas de Software 112
Estabelecimento de uma Referência 113
Coleta, Cálculo e Avaliação de Métricas..... 114
Métricas para Pequenas Organizações..... 115
Estabelecimento de um Programa de Métricas de Software 116

121 **UNIDADE 6**
CONCEITOS DE QUALIDADE

Qualidade 124
Controle de Qualidade 125
Garantia de Qualidade 125
Custo da Qualidade 125
Garantia de Qualidade de Software 127
Panorama Historico 127
Atividades de SQA 128

Prepara um plano SQA para um projeto 129
Revisões de Software 130
Impacto nos Custos de Defeitos de Software 131
Amplificação e Remoção de Defeitos 131
Modelo de amplificação de defeitos 132
Revisões Técnicas Formais 133
Abordagens Formais para SQA 139
Garantia Estatística de Qualidade de Software 139
Um Exemplo Genérico 140
Seis Sigma para Engenharia de Software 142
Confiabilidade de Software 143
Medidas de Confiabilidade e Disponibilidade 144
Segurança de Software 145
As Normas de Qualidade ISO 9000 146
Os Planos de SQA 147

UNIDADE 01

O PAPEL EVOLUTIVO DO *SOFTWARE*

OBJETIVOS:

- **Compreender o papel evolutivo do *software*;**
- **Estabelecer uma relação entre as categorias de *software* de computadores;**
- **Identificar as principais características do *software*;**
- **Conceituar o processo de análise do *software*;**
- **Conhecer alguns mitos sobre *software*.**

1 O PAPEL EVOLUTIVO DO *SOFTWARE*

O Papel Evolutivo do *Software*

Hoje em dia, o *software* assume um duplo papel. Ele é o produto e, ao mesmo tempo, o veículo para entrega do produto. Como produto ele disponibiliza o potencial de computação presente no *hardware* do computador ou, mais amplamente, por uma rede de computadores acessível pelo *hardware* local. Quer resida em um telefone celular, quer opere em um computador de grande porte, o *software* é um transformador de informações – produzindo, gerindo, adquirindo, modificando, exibindo ou transmitindo informações que podem ser tão simples como um *púnico bit* ou tão complexas quanto uma apresentação multimídia. Como veículo usado para entrega do produto, o *software* age como base para o controle do computador (sistemas operacionais), apara a comunicação da informação (rede) e para a criação e controle de outros programas (ferramentas e ambientes de *software*).

O *software* entrega o mais importante do produto da nossa época – a informação. Ele transforma dados pessoais (por exemplo, as transações financeiras de uma pessoa) de modo que os dados possam ser mais úteis em um determinado contexto; organiza informações comerciais para melhorar a competitividade; fornecem um portal para as redes de informação de âmbito mundial (por exemplo, a *Internet*) e proporciona os meios para obter informação em todas as suas formas.

A importância do *software* de computadores tem passado por mudanças significativas em pouco mais de 50 anos. Melhora surpreendente o desempenho do *hardware*, profundas modificações na arquitetura de computadores, aumento significativo na memória e na capacidade de armazenamento, e uma variedade de opções incomuns de entrada e saída levaram a sistemas baseados em computadores mais

sofisticados e complexos, produzindo resultados magníficos quando um sistema é bem-sucedido, mas também podem causar enormes problemas para quem precisa construir sistemas complexos.

Livros populares publicados nas décadas de 1970 e 1980 fornecem uma visão histórica útil da percepção de mudanças dos computadores e do *software*, e do seu impacto em nossa cultura. Osborne (1979) caracterizou como uma "nova revolução industrial". Toffler (1980) chamou o advento da microeletrônica de "terceira onda de mudança" da história humana e Naisbitt (1982) previu a transformação de uma sociedade industrial para uma "sociedade de informação". Feingebaum e McCorduck (1983) sugeriram que a informação e o conhecimento (controlados por computadores) seriam o centro do poder no século XXI e Stoll (1989) ponderou que a "comunidade eletrônica", criada por redes e *software*, era a chave para a troca de conhecimento em todo o mundo.

No início dos anos 90, Toffler (1990) descreveu uma "mudança de poder" na quais antigas estruturas de poder (governamental, educacional, industrial, econômica e militar) se desintegravam, visto que computadores e *software* levam a uma "democratização do conhecimento". Yourdon (1992) temeu que as empresas norte-americanas pudessem perder sua vantagem competitiva nos negócios relacionados a *software* e previu "o declínio e a queda do programador norte-americano". Hammer e Champy (1993) previram que as tecnologias da informação desempenhariam um papel fundamental na "reengenharia das empresas". Em meados dos anos 90, a difusão dos computadores e do software provocou uma onda de livros "anticomputadores" (por exemplo, *Resisting the Virtual Life*, editado por James Brook e Ian Boal, e The Future Does Not Compute por Stephen Talbolt). Esses autores demonificaram o computador, enfatizando preocupações legítimas, mas ignorando os profundos benefícios que deles já haviam sido obtidos [LEV95].

Na segunda metade da década de 1990, Yourdon (1996) reavaliou as perspectivas para o profissional de *software* e sugeriu "a ascensão e a ressurreição" do programador norte-americano. À medida que a *Internet* cresceu em importância, a mudança de tendência de Yourdon mostrou ser correta. Ao final do século XX, o foco deslocou-se mais uma vez, agora sob o impacto da "bomba-relógio" do ano 2000 (por exemplo, Yordon (1998a), e [KAR99]. Apesar de as previsões de catástrofe do ano 2000 terem sido exageradas, sua popularização concorreu para a invasão do *software* em nossas vidas.

Conforme o segundo milênio progredia, Johnson (2001) discutiu

o poder do “surgimento” – fenômeno que explica o que acontece quando as interconexões entre entidades relativamente simples resultam em um sistema que “se auto-organiza para formar comportamento mais inteligente e mais adaptativo”. Yourdon (2002) revisitou os trágicos eventos de 11 de setembro para discutir o impacto contínuo do terrorismo global na comunidade de TI. Wolfman (2002) apresentou um tratado sobre “uma nova espécie de ciência”, que postula uma teoria unificadora baseada principalmente em simulações de *software* sofisticadas. Daconta *et al* (2003) discutiram a evolução da “*Web* semântica” e os modos pelos quais ela vai mudar a maneira pela qual as pessoas interagem com as redes globais.

Hoje em dia uma enorme indústria de *software* tornou-se fator dominante nas economias do mundo industrializado. O programador solitário de antigamente foi substituído por uma equipe de especialistas em *software*, cada um se concentrando numa parte da tecnologia necessária para produzir uma aplicação complexa. No entanto, as questões que eram feitas ao programador solitário são as mesmas questões que são feitas quando os modernos sistemas baseados em computador são constituídos:

- Por que leva tanto tempo para concluir um *software*?
- Por que os custos de desenvolvimento são tão altos?
- Por que não podemos achar todos os erros antes de entregar o *software* aos clientes?
- Por que gastamos tanto tempo e esforço mantendo programas existentes?
- Por que continuamos tendo dificuldade em avaliar o progresso enquanto o *software* é desenvolvido e mantido?

Essas e muitas outras questões demonstram a preocupação da indústria com o *software* e a maneira pela qual ele é desenvolvido – preocupação que tem levado à adoção da prática de engenharia de *software*.

Software

Em 1970, menos de 1% do público poderia ter definido o que “*software* de computadores” significava. Hoje, a maioria dos profissionais e muitos membros do público em geral pensam que entendem de

software. Mas será verdade?

A definição de *software* em um livro-texto poderia ter a seguinte forma: *Software* são (1) instruções (programas de computadores) que quando executadas fornecem as características, função e desempenho desejados; (2) estruturas de dados que permitem aos programas manipular adequadamente a informação; e (3) documentos que descrevem a operação e o uso dos programas. Não há dúvida de que definições mais completas poderiam ser oferecidas, mas nós precisamos mais do que uma definição formal.

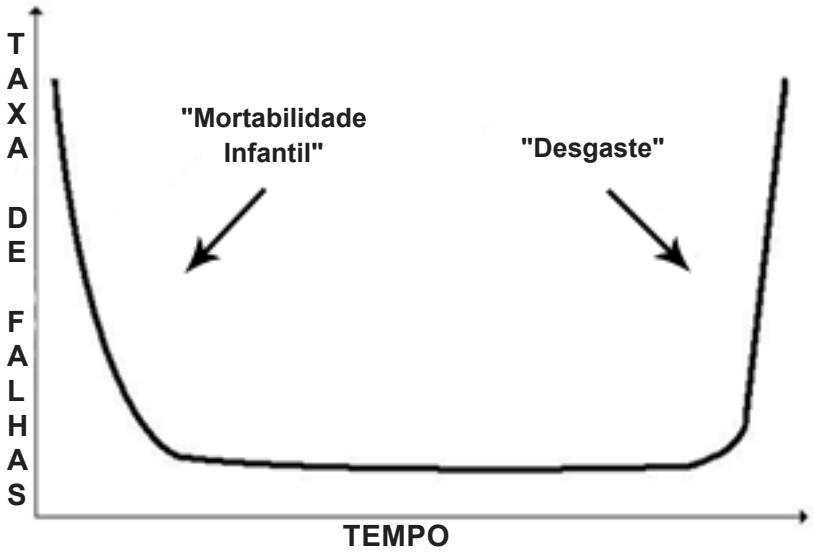
Para entendermos de *software* (e, em última análise, de engenharia de *software*), é importante examinar as características do *software* que o tornam diferente de outras coisas que os seres humanos nos produzem. O *software* é um elemento de um sistema lógico e não de um sistema físico. Assim, ele possui características que são consideravelmente diferentes daquelas do *hardware*.

"O *software* é desenvolvido ou passa por um processo de engenharia; não é fabricado no sentido clássico"

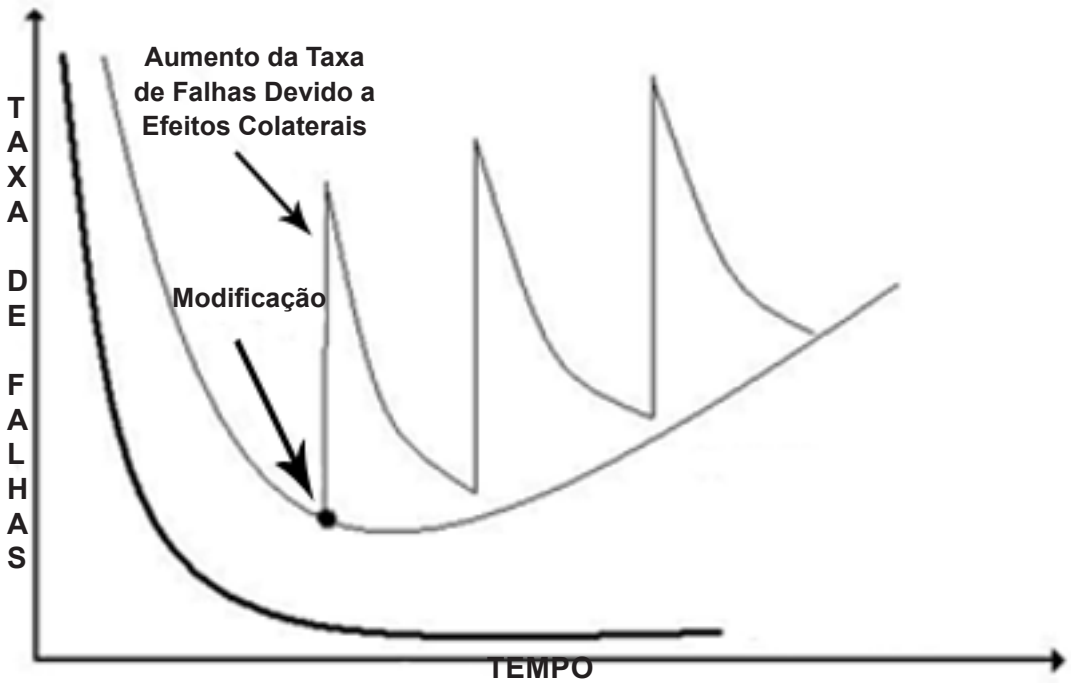
Apesar de existirem algumas semelhanças entre o desenvolvimento de *softwares* e a fabricação de *hardware*, as duas atividades são fundamentalmente diferentes. Em ambas, a alta qualidade é conseguida por um bom objeto, mas a fase de fabricação do *hardware* pode gerar problemas de qualidade que são inexistentes (ou facilmente corrigidos) para o *software*. Ambas as atividades são dependentes de pessoas, mas a relação entre as pessoas envolvidas e o trabalho realizado é inteiramente diferente. As duas atividades requerem a construção de um "produto", porém as abordagens são diferentes. Os custos do *software* são concentrados na engenharia. Isso significa que os projetos de *software* não podem ser geridos como se fossem projetos de fabricação.

Software não "se desgasta"

O relacionamento, frequentemente chamado de "curva da banheira", indica que o *hardware* exibe taxas de falha relativamente altas no início de sua vida (essas falhas são, em sua maioria, atribuíveis a defeitos de projeto ou de fabricação).



Fonte : Pressman, Roger S. Engenharia de Software / 2010.



Fonte : Pressman, Roger S. Engenharia de ***Software*** / 2010.

Os defeitos são depois corrigidos e a taxa de falhas de um projeto cai para um nível constante (idealmente bastante baixo) por certo período de tempo. Com o passar do tempo, no entanto, a taxa de falhas aumenta

novamente, visto que os componentes de *hardware* sofrem o efeito cumulativo de poeira, vibração, maus-tratos, gradientes de temperatura e muitos outros males ambientais. Falando simplesmente, o *hardware* começa a se desgastar.

O *software* não é suscetível aos males ambientais que causam o desgaste do *hardware*. Teoricamente, entretanto, a curva da taxa de falhas para o *software* deveria tomar a forma de "curva idealizada". Defeitos não detectados causarão altas taxas de falhas no início da vida de um programa. Todavia, eles são corrigidos (idealmente sem a introdução de outros erros) e a curva se achata. A curva idealizada é uma simplificação grosseira dos modelos reais de falhas para *softwares*. Todavia, a implicação é clara – o *software* não se desgasta, mas se deteriora.

Essa aparente contradição poderá ser bem explicada considerando a "curva real". Durante sua vida, o *software* passará por modificações. Conforme as modificações feitas, é provável que erros sejam introduzidos causando um dente na curva da taxa de falhas. Antes que a curva possa voltar ao seu estado original de taxa de falhas estável, outra modificação pode é solicitada causando novo dente na curva. A taxa de falhas mínima começa a subir lentamente – o *software* está se deteriorando em razão das modificações.

Outro aspecto do desgaste ilustra a diferença entre *hardware* e *software*. Quando um componente de *hardware* se desgasta, é substituído por uma peça sobressalente. Não há peças sobressalentes de *software*. Toda falha indica um erro no projeto ou no processo pelo qual o projeto foi traduzido para código de máquina executável. Assim, a manutenção de *software* envolve consideravelmente mais complexidade do que a manutenção de *hardware*.

"Apesar de a indústria estar se movendo em direção à montagem baseada em componentes, a maior parte dos *softwares* continua a ser construída sob encomenda".

Considere a maneira pela qual o *hardware* de controle para um produto baseado em computador é projetado e construído. O engenheiro de projeto desenha um esquema simples do circuito digital, faz uma análise fundamental para assegurar a função pretendida e, depois consulta os

catálogos de componentes digitais. Cada circuito integrado tem um código de componente, uma função definida e validada, uma interface bem delineada e um conjunto padrão de diretrizes de integração. Depois que cada componente é selecionado pode ser requisitado do estoque.

À medida que uma disciplina da engenharia evolui, uma coleção de componentes de projeto padronizados é criada. Parafusos padronizados e circuitos integrados de linha são apenas dois dos milhares de componentes-padrões que são usados por engenheiros mecânicos e eletricistas quando projetam novos sistemas. Os componentes reutilizáveis foram criados para que o engenheiro possa se concentrar nos elementos realmente inovadores de um projeto, isto é, as partes do projeto que representam algo novo no mundo do *hardware*, o reuso de componentes é natural no processo de engenharia. No mundo do *software*, é algo que está apenas começando a ser obtido em ampla escala.

Um componente de *software* deve ser projetado e implementado de modo que possa ser reusado em muitos programas diferentes. Componentes modernos reutilizáveis encapsulam tanto os dados quanto o processamento aplicado aos dados, permitindo ao engenheiro de *software* criar novas aplicações a partir de partes reusáveis. Por exemplo, as atuais interfaces gráficas com os usuários são construídas com componentes reutilizáveis que permitem a criação de janelas gráficas, menus retráteis (*Pull-down*) e uma ampla variedade de mecanismos de interação. Os detalhes de estruturas de dados e de processamento necessários para construir a interface estão contidos em uma biblioteca de componentes reutilizáveis para a construção de interface.

A Natureza Mutável do *Software*

Hoje em dia, sete amplas categorias de *software* de computadores apresentam desafios contínuos para os engenheiros de *software*:

***Software* de sistemas:** é uma coleção de programas. Alguns - por exemplo: compiladores, editores e utilitários para gestão de arquivos – processam estruturas de informação complexas, mas determinadas. Outras aplicações de sistemas (por exemplo, componentes de sistemas operacionais, acionadores, *software* de rede e processadores de telecomunicações) processam dados amplamente indeterminados. Em ambos os casos, a área se *software* de sistemas é caracterizada por interação intensa com o *hardware* do computador; uso intenso

por múltiplos usuários; operação concorrente que requer ordenação, compartilhamento de recursos e sofisticada gestão de processo; estruturas de dados complexos e interfaces externas múltiplas.

***Software* de aplicação:** consiste em programas isolados que resolvem uma necessidade específica do negócio. Aplicações nessa área processam dados comerciais ou técnicos de um modo que facilita as operações ou gestão/tomada de decisões técnicas do negócio. Além de aplicações convencionais de processamento de dados, o *software* de aplicação é usado para controlar funções do negócio em tempo real (por exemplo, processamento de transições no ponto-de-venda, controle de processo de fabricação em tempo real).

***Software* científico e de engenharia:** Antigamente caracterizado por algoritmos "*number crunching*" (que processam números), as aplicações de *software* científico e de engenharia vão da astronomia à vulcanologia, da análise automotiva de tensões à dinâmica orbital do ônibus espacial, e da biologia molecular à manufatura automatizada. Todavia, as aplicações modernas na área científica de engenharia estão se afastando dos algoritmos numéricos convencionais. Projeto apoiado por computadores, simulação de sistemas e outras aplicações interativas começaram a adquirir características de tempo real e até de *software* de sistemas.

***Software* embutido:** reside de um produto ou sistema e é usado para implementar e controlar características e funções para o usuário final e para o próprio sistema. Ele pode realizar funções muito limitadas e particulares (por exemplo, o controle de teclado para um forno de microondas) ou fornecer função significativa e capacidade de controle (por exemplo, funções digitais em um automóvel, tais como controle de combustível, mostradores do painel e sistemas de frenagem, etc.).

Software para linhas de produtos: Projetado para fornecer capacidade específica de ser usado por muitos clientes diferentes, pode focalizar um mercado limitado e especial (por exemplo, produtos de controle de estoque) ou dirigir-se ao mercado de consumo de massa (por exemplo, processamento de texto, planilhas, gráficos por computador, multimídia, entretenimento, gestão de banco de dados, aplicações financeiras pessoais e empresariais).

Aplicações da Web: "*ApsWeb*", cobrem uma ampla gama de aplicações. Na sua forma mais simples, *ApsWeb* podem ser pouco mais que um conjunto de arquivos ligados por hipertexto que apresentam informações usando texto e poucos gráficos. No entanto, conforme

as aplicações de comércio eletrônico (*e-commerce*) e B2B crescem em importância, as *ApsWeb* evoluem para sofisticados ambientes computacionais que fornecem não apenas características isoladas, funções de computação e conteúdo para o usuário final, mas também estão integradas ao banco de dados da empresa e às aplicações do negócio.

***Software* para inteligência artificial:** Faz uso de algoritmos não-numéricos para resolver problemas complexos que não são passíveis de computação ou análise direta. Aplicações nessa área incluem robótica, sistemas especialistas, reconhecimento de padrões (de imagem e de voz), redes neurais artificiais, prova de teoremas e jogos.

Milhões de engenheiros de *software* em todo o mundo estão trabalhando duro em projetos de uma ou mais dessas categorias. Em alguns casos, novos sistemas estão sendo construídos, mas em outras aplicações existentes são corrigidas, adaptadas e aperfeiçoadas. É comum para um jovem engenheiro de *software* trabalhar em um programa que é mais velho do que ele! Gerações passadas de pessoal de *software* deixaram um legado em cada uma dessas categorias que foram discutidas. Tomara que o legado deixado por esta geração torne mais fácil o encargo de futuros engenheiros de *software*. No entanto, novos desafios apareceram no horizonte.

Computação ubíqua: O rápido crescimento de redes sem fio pode levar logo à verdadeira computação distribuída. O desafio para os engenheiros de *software* será desenvolver sistemas e *softwares* de aplicação que permitam que pequenos dispositivos, computadores pessoais e sistemas empresariais comuniquem através de grandes redes.

NetSourcing: A *World Wide Web* (www) está rapidamente se transformando em uma ferramenta da computação e também em um fornecedor de conteúdo. O desafio para os engenheiros de *software* é arquitetar aplicações simples (como planejamento financeiro pessoal) e sofisticadas que forneçam benefícios para um mercado global de usuários finais visados.

Software* aberto*: uma tendência crescente que resulta na distribuição de código-fonte para sistemas de aplicação (Por exemplo: sistemas operacionais, bancos de dados e ambientes de desenvolvimento), de modo que os clientes possam fazer modificações locais. O desafio para os engenheiros de *software* é construir código-fonte que seja autodescritível e, mais importante, desenvolver técnicas que permitam ao cliente e aos desenvolvedores saber que modificações

foram feitas e como essas modificações se manifestam no *software*.

A "nova economia": A insanidade ponto-com que tomou os mercados financeiros durante o final da década de 1990 e o surto que se seguiu no início de 2000 levaram muitas pessoas de negócio a acreditar que a nova economia está morta. A nova economia está viva e muito bem, mas vai evoluir lentamente. Ela está caracterizada pela comunicação e distribuição em massa. Andy Lippman (2002, 44) nota isso quando escreve:

> Estamos entrando em uma era caracterizada por comunicações entre máquinas distribuídas e pessoas dispersas, em vez de ser principalmente uma conexão entro dois indivíduos ou entre um indivíduo e uma máquina. A abordagem antiga para a telefonia era sobre "conexões para"; a onda seguinte é sobre "conexões entre". Os sistemas Napster, de mensagens instantâneas, de mensagens curtas e BlackBerries são exemplos disso.

O desafio parta os engenheiros de *software* é construir aplicações que facilitem a comunicação de massa e a distribuição de produtos em massa usando conceitos que estão agora apenas se formando.

Cada um desses "novos desafios" vai, sem dúvida, obedecer à lei das consequências não-pretendidas e terá efeitos (para empresários, engenheiros de *software* e usuários finais) que não podem ser hoje previstos. No entanto, os engenheiros de *software* podem se preparar para instanciar um processo que seja suficientemente ágil e adaptável para acomodar grandes modificações em tecnologia e regras de negócio que certamente virão na próxima década.

Software Legado

Centenas de milhares de programas de computador caem em um dos sete amplos domínios de aplicação – *software* de sistemas, *software* de aplicação, *software* científico e de engenharia, *software* embutido, produtos de *software*, *ApsWeb* e aplicações para inteligência artificial – discutidos na Seção anterior. Alguns deles são *softwares* do estado da arte – recém-lançados para indivíduos, indústria e governo. Mas outros programas são mais antigos, em alguns casos muito mais antigos.

Esses programas antigos – frequentemente referidos como *software* legado – têm sido foco de atenção e preocupação contínuas

desde a década de 1960. Dayani-Fard e seus colegas (1999) descrevem o software legado do seguinte modo:

> Sistemas de software legado (...) foram desenvolvidos décadas atrás e têm sido continuamente modificados para satisfazer a mudanças nos requisitos do negócio e nas plataformas de computação. A ploriferação de tais sistemas está causando dor de cabeça para grandes organizações que os consideram dispendiosos de manter e arriscados de evoluir.

Liu et al. (1998) estendem essa descrição observando que “muitos sistemas legados permanecem dando suporte a funções importantes do negócio e são indispensáveis ao negócio”. Assim, *software* legado é caracterizado por longevidade e com críticas para o negócio.

A Qualidade do *Software* Legado

Infelizmente, existe uma característica adicional que pode estar presente no *software* legado – má qualidade. Sistemas legados, algumas vezes, têm projetos não-extensíveis, código complicado, documentação pobre ou inexistente, casos de teste e resultados que nunca foi arquivado, um histórico de modificações mal gerido - a lista pode ser bem longa. E, no entanto, esses sistemas apoiam “funções importantes do negócio e são indispensáveis ao negócio” (LIU et al, 1998). O que pode ser feito?

A única resposta razoável pode ser não fazer nada, pelo menos até que o sistema legado precise passar por modificações significativas. Se o *software* legado satisfaz às necessidades de seus usuários e funciona confiavelmente, não está danificado e não precisa ser consertado. No entanto, à medida que o tempo passa os sistemas legados frequentemente evoluem, por uma ou mais das seguintes razões:

- O *software* deve ser adaptado para satisfazer às necessidades do novo ambiente ou tecnologia computacional;
- O *software* precisa ser aperfeiçoado para implementar novos requisitos do negócio;
- O *software* precisa ser estendido para torná-lo interoperável com os sistemas ou bancos de dados mais modernos.
- O *software* precisa ser rearquitetado para torná-lo viável em um ambiente de rede.

Quando esses modos de evolução ocorrem, um sistema legado precisa ser ”reengenheirado” de modo que permaneça viável no futuro. O objetivo da engenharia de *software* moderna é conceber metodologias que sejam fundamentadas na noção de evolução, ou seja, a noção de que sistemas de *software* modificam-se continuamente, novos sistemas de *software* são construídos a partir de antigos e todos precisam interoperar e cooperar uns com os outros (DAV, 1995).

Evolução de *Software*

Independentemente do tamanho, complexidade ou domínio de aplicação, o *software* de computador vai evoluir como o tempo. As modificações (frequentemente denominadas manutenção de *software*) orientam esse processo e ocorrem quando erros são corrigidos, quando o *software* é adaptado a um novo ambiente, quando o cliente solicita novas características ou funções e quando a aplicação é “reengenheirada” para fornecer benefícios em um contexto moderno. San Williams (2002) descreve isso quando escreve:

> À medida que os programas de larga escala como o *Windows* e o *Solaris* expandem-se bastante no intervalo de 30 a 50 milhões de linhas de código, gerentes de projetos bem-sucedidos aprenderam a se dedicar tanto tempo para desatar os nós de código legado quanto para adicionar novo código.

Colocado de modo simples, em uma década que viu o desempenho médio do microchip de PC aumentar cem vezes, a inabilidade do *software* de aumentar na mesma velocidade foi de um pequeno segredo a um embaraço de toda a indústria.

Durante os últimos 30 anos, Manny Lehman e seus colegas (1997a) realizaram análises detalhadas de *software* e sistemas de qualidade industrial em um esforço para desenvolver uma teoria unificada para evolução de *software*. Os detalhes desse trabalho vão além do escopo deste livro, mas as leis subjacentes que foram derivadas são dignas de nota (LEHMAN, 1997b).

- Lei da Modificação Contínua (1974): Sistemas tipo-E devem ser continuamente adaptados ou então eles se tornam

progressivamente menos satisfatórios.

- Lei da Complexidade Crescente (1974): À medida que um sistema tipo-E evolui, sua complexidade aumenta, a menos que seja realizado esforço para mantê-lo ou reduzi-la.
- Lei da Auto-Regulação (1974): A velocidade média da atividade global efetiva em um sistema evolutivo tipo-E é invariante ao longo do tempo de vida do produto.
- Lei da Conservação da Familiaridade (1980): À medida que um sistema tipo-E evolui, todos os que estão a ele associados, desenvolvedores, pessoal de vendas e usuários, por exemplo, devem manter o domínio de seu conteúdo e comportamento para conseguir evolução satisfatória. O crescimento excessivo diminui esse domínio. Assim, o crescimento médio incremental permanece à medida que o sistema evolui.
- Lei do Crescimento Contínuo (1980): O conteúdo funcional de sistemas tipo-E deve ser continuamente aumentado para manter a satisfação do usuário ao longo do tempo de vida do sistema.
- Lei da Qualidade Declinante (1996): A qualidade de sistemas tipo-E parecerá estar declinando a menos que eles sejam rigorosamente mantidos e adaptados às modificações no ambiente operacional.
- Lei de Realimentação do Sistema (1996): Os processos de evolução tipo-E constituem sistemas de realimentação em multiníveis por multiagentes e precisam ser tratados como tal para conseguir aperfeiçoamento significativo sobre qualquer base razoável.
- As leis de Lehman e seus colegas definiram uma parte inerente da realidade de um engenheiro de *software*.

Mitos do *Software*

Mitos de *Software* são crenças sobre *software* e sobre o processo usado para construí-los – podem ser encontrados desde os primeiros dias da computação. Os mitos têm uma qualidade de atributos que os têm tornado traiçoeiros. Por exemplo, os mitos parecem ser afirmações de fatos razoáveis (algumas vezes contendo elementos verdadeiros), têm um aspecto intuitivo e são frequentemente divulgados por profissionais experientes que “entendem do assunto”.

Hoje, a maioria dos profissionais especializados em engenharia de *software* reconhece os mitos pelo que eles são – atitudes enganosas que causaram sérios problemas tanto para gerentes quanto para pessoal técnico. No entanto, atitudes e hábitos antigos são difíceis de modificar e remanescentes de mitos de *software* ainda são acreditados.

Mito da gerência: Os gerentes com responsabilidade sobre o *software*, como os gerentes na maioria das atividades, estão frequentemente sobre pressão para obedecer a orçamentos, manter cronogramas no prazo e melhorar a qualidade. Como uma pessoa que está se afogando e se agarra em um pedaço de madeira, um gerente de *software* frequentemente se agarra à crença em um mito de *software*, se isso puder diminuir a pressão (ainda que temporariamente).

Mito: Já temos um livro que será cheio de padrões e procedimentos para elaborar o *software*. Isso não fornece ao meu pessoal tudo o que ele precisa saber?

Realidade: O livro de padrões pode muito bem existir, mas será que é usado? Os profissionais de *software* sabem de sua existência? Ele reflete as práticas modernas da engenharia de *software*? É completo? É adaptável? Está voltando para melhorar o prazo de entrega, mantendo o foco na realidade? Em muitos casos, a resposta de todas essas perguntas é não.

Mito: Se nos atrasarmos no cronograma, podemos adicionar mais programadores e ficar em dia? Algumas vezes denominado conceito da Horda mongólica.

Realidade: O desenvolvimento de *softwares* não é um processo mecanizado como o de fabricação. Nas palavras de Brooks (1975): "Adicionar pessoas a um projeto de *software* atrasado, demora mais ainda". A princípio, essa afirmação pode parecer contraintuitiva. Todavia, sempre que novas pessoas são adicionadas, o pessoal que estava trabalhando precisa gastar tempo orientando os recém-chegados, reduzindo assim a qualidade de tempo empregado no esforço de desenvolvimento produtivo. Pessoas podem ser adicionadas, mas apenas de forma bem planejada e bem coordenada.

Mito: Se eu decidir terceirizar um projeto de *software*, vou poder relaxar e deitar que aquela firma o elabore?

Realidade: Se uma organização não sabe como gerir e controlar projetos de *software* internamente, certamente terá problemas quando terceirizar esses projetos.

Mito do cliente: Um cliente que encomenda *software* para

computador pode ser uma pessoa na mesa vizinha, um grupo técnico em outra sala, o departamento de promoção/vendas ou outra empresa que encomendou o *software* sob contato. Em muitos casos, o cliente acredita em mitos de *software* porque os gerentes e profissionais deste ramo pouco corrigem essa desinformação. Mitos levam a falsas expectativas (pelo cliente) e, em última análise, à insatisfação com o desenvolvedor.

Mito: O estabelecimento geral de objetos é suficiente para iniciar a escrita de programas – podemos fornecer os detalhes posteriormente.

Realidade: Embora uma declaração abrangente e estável dos requisitos nem sempre seja possível, uma definição inicial malfeita é a principal causa de esforços malsucedidos de *software*. Uma descrição formal e detalhada do domínio da informação, da função, do comportamento, do desempenho, das interfaces, das restrições de projeto e dos critérios de avaliação essência pode ser determinada somente depois de intensa comunicação entre o cliente e o desenvolvedor.

Mito: Os requisitos de um projeto mudam continuamente, mas as mudanças podem ser facilmente acomodadas porque o *software* é flexível.

Realidade: É verdade que os requisitos de um *software* mudam, mas o impacto da mudança varia com a época em que é introduzida. Quando mudanças são solicitadas antecipadamente (antes que o projeto e a codificação tenham começado), o impacto no custo cresce rapidamente baixo. No entanto, à medida que o tempo passa, o impacto de custo cresce rapidamente – recursos foram comprometidos, a estrutura do projeto foi estabelecida e a mudança pode causar consequências que exijam recursos adicionais e grandes modificações no projeto.

Mito do profissional: Os mitos que ainda têm crédito entre os profissionais de *software* sobreviveram a mais de 50 anos de cultura de programação. Durante os primeiros dias do *software*, a programação era vista como uma forma de arte. Maneiras e atitudes antigas custam a morrer.

Mito: Quando escrevemos um programa e o fazemos funcionar, nosso trabalho está completo.

Realidade: Alguém disse um dia que “quanto mais cedo você começar a escrever código, mais demora a acabar”. Dados da indústria indicam que entre 60 e 80% de todo o esforço despendido em *software* vai ser despendido depois de ele ser entregue ao cliente pela primeira vez.

Mito: Até que eu esteja com o programa “rodando” não tenho

como avaliar a sua qualidade.

Realidade: Um dos mecanismos mais eficazes de garantia de qualidade de *software* pode ser aplicado a partir do início do projeto – a revisão técnica formal. Revisões de *software* é um "filtro de qualidade" que descobriu ser mais eficaz que testes para encontrar alguns tipos de erros de *software*.

Mito: O único produto de trabalho que pode ser entregue para um projeto de *software* bem-sucedido é o programa executável.

Realidade: Um programa executável é apenas uma parte de uma configuração de *software* que inclui vários elementos. A documentação fornece a base para uma engenharia bem-sucedida e, mais importante, orienta para o suporte ao *software*.

Mito: A engenharia de *software* vai nos fazer criar documentação volumosa e desnecessária que certamente nos atrasará.

Realidade. A engenharia de *software* não se relaciona à criação de documentos. Refere-se à criação de qualidade. Melhor qualidade leva à redução de repetição do trabalho. E menor repetição resulta em tempos de entrega mais rápidos.

Muitos dos profissionais de *software* reconhecem a falácia dos mitos descritos. Lamentavelmente, ações e métodos habituais levam a práticas de gestão e técnicas de baixa qualidade, mesmo quando a realidade exige uma melhor abordagem.

O reconhecimento da realidade do *software* é o primeiro passo em direção à formulação de soluções práticas para a engenharia deste.

Como Tudo Começa

Todo projeto de *software* é iniciado por alguma necessidade do negócio, ou seja, a necessidade de corrigir algum defeito em uma aplicação existente; a necessidade de adaptar um sistema legado para mudar o ambiente do negócio; a necessidade de estender as funções e características de uma aplicação existente; ou a necessidade de criar um novo produto, serviço ou sistema.

No início de um projeto de engenharia de *software*, a necessidade do negócio é muitas vezes expressa informalmente, como parte de uma simples conversa. A conversa apresentada em CasaSegura é típica.

Com exceção de uma referência passageira, o *software* praticamente não é mencionado como parte da conversa. No entanto, ele será tudo ou nada para a linha de produtos CasaSegura. O esforço

da engenharia terá sucesso somente se o *software* CasaSegura tiver sucesso. O mercado aceitará o produto somente se o *software* embutido dentro dele atender às necessidades dos clientes (ainda que não fixadas).

Processo: Uma Visão Genérica

Em um livro fascinante que fornece uma visão da economia do software e da sua engenharia, Baetjer Jr. (1998, p.85) faz os seguintes comentários sobre o processo de *software*:

> O *software*, como todo capital, é conhecimento incorporado, e como esse conhecimento está inicialmente disperso, tácito, latente e incompleto na sua totalidade, o seu desenvolvimento é um processo de aprendizado social.

O processo é um diálogo no qual o conhecimento que se deve transformar em *software* é reunido e incorporado a este. O processo fornece interação entre usuários e projetistas, entre usuários e ferramentas em desenvolvimento e entre projetistas e ferramentas em desenvolvimento [tecnologia].

É um processo interativo no qual a própria ferramenta serve como meio de comunicação e com cada nova rodada de diálogo explicitado, mais conhecimento útil do pessoal envolvido.

Efetivamente, a elaboração de *software* de computador é um processo interativo de aprendizado, e o resultado, algo que Baetjer chamaria de "essência do *software*" é uma incorporação de conhecimentos coletados, destilados e organizados, à medida que o processo é conduzido.

Mas o que exatamente é um processo de *software* do ponto de vista técnico? Definimos processo de *software* como um arcabouço para as tarefas que são necessárias para construir *softwares* de alta qualidade.

Processo é sinônimo de engenharia de *software*? A resposta é "sim" e "não". Um processo de *software* também inclui tecnologias que constituem o processo – métodos técnicos e ferramentas automatizadas.

Mais importante, a engenharia de *software* é realizada por pessoas criativas, com amplos conhecimentos, que devem adaptar um processo de *software* maduro apropriado para os produtos que elas constróem e para as demandas do seu mercado.



Exercícios Propostos

1. Pesquise sobre a evolução do *Software*.
2. O que caracteriza *Software* Legado.
3. Conceitue *Software*.
4. Faça uma relação sobre alguns mitos e realidades do *software*.



Saiba Mais

http://www.dryjapat.com/look.htm
http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=5484112
http://www.batebyte.pr.gov.br/modules/conteudo/conteudo.php?conteudo=299



Resumindo

O capítulo mostrou que hoje em dia o *software* assume um duplo papel. Ele é o produto e, ao mesmo tempo, o veículo para entrega do produto. Que o *software* de computadores tem passado por mudanças significativas em pouco mais de 50 anos, produzindo, gerindo, adquirindo, modificando, exibindo ou transmitindo informações que podem ser tão simples como um púnico *bit* ou tão complexas quanto uma apresentação multimídia. Hoje em dia, uma enorme indústria de *software* tornou-se fator dominante nas economias do mundo industrializado. O programador solitário de antigamente foi substituído por uma equipe de especialistas em *software*, cada um se concentrando numa parte da tecnologia necessária para produzir uma aplicação complexa. Existem sete amplas categorias de *software* de computadores que apresentam desafios contínuos para os engenheiros deste.

UNIDADE 02

ENGENHARIA DE *SOFTWARE:* UMA TECNOLOGIA EM CAMADAS

OBJETIVOS:

- **Identificar os principais parâmetros da engenharia de *software*;**
- **Compreender as ações nos projetos de *software*;**
- **Identificar as principais características dos projetos de *software*;**
- **Conhecer as metas do processo de análise do software.**

2 ENGENHARIA DE *SOFTWARE:* UMA TECNOLOGIA EM CAMADAS

Apesar de centenas de autores terem desenvolvido definições pessoais de engenharia de *software*, uma definição proposta por Fritz Bauer [NAU69] na conferência pioneira sobre o assunto ainda serve de base para a discussão:

> Engenharia de software é a criação de sólidos princípios de engenharia a fim de obter softwares econômicos que sejam confiáveis e que trabalhem em máquinas reais.

Ela diz pouco sobre os aspectos técnicos na qualidade do *software*; não inclui diretamente a necessidade de satisfação do cliente ou pontualidade; não faz menção à importância de medidas e unidades; não fala sobre a importância de um processo amadurecido. Ainda assim, a definição de Bauer nos dá uma linha básica. Quais são os "sólidos princípios da engenharia" que podem ser aplicados ao desenvolvimento de *softwares* de computador? Como construímos *softwares* "economicamente" e de modo que eles sejam "confiáveis"? O que é necessário para criar programas de computador que trabalhem "eficientemente" em não apenas uma, mas em várias "máquinas reais"? Essas são questões que continuam a desafiar os engenheiros de *software*.

O IEEE (1993) desenvolveu uma definição mais abrangente ao estabelecer:

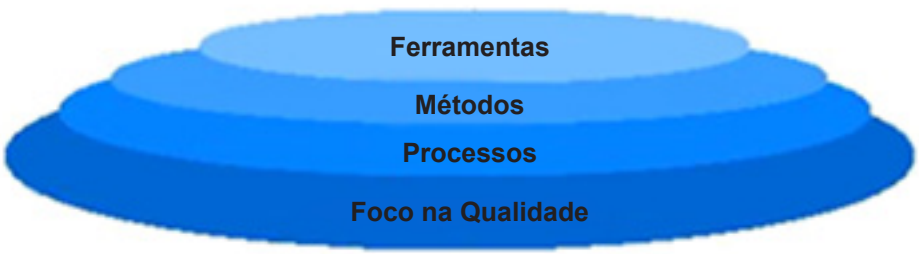
Engenharia de *software*: (1) aplicação de uma abordagem sistemática, disciplinada e quantificável para o desenvolvimento, operação e manutenção do *software*; isto é, a aplicação da engenharia ao *software*. (2) O estudo desta abordagem.

No entanto, o que é "sistemático, disciplinado e quantificável" para uma equipe de *software* pode ser cansativo para outra. Precisamos de

disciplina, mas precisamos também de adaptabilidade e agilidade.

A engenharia de *software* é uma tecnologia em camadas. Gestão de Qualidade Total, Seis Sigmas (*Six Sigma*) e filosofias análogas levam à cultura de um processo contínuo de aperfeiçoamento, e é essa cultura que, em última análise, leva ao desenvolvimento de abordagens cada vez mais efetivas para a engenharia de *software*, o qual é a base em que se apóia o foco na qualidade.

O alicerce da engenharia de *software* é a camada de processo que mantém unidas as camadas de tecnologia e permite o desenvolvimento racional e oportuno de *softwares* de computador. O processo define um arcabouço [PAU93] que deve ser estabelecido para a efetiva utilização da tecnologia de engenharia de *software*. Os processos de *software* formam a base para o controle gerencial de projetos de *software* e estabelece o contexto no qual os métodos técnicos são aplicados, os produtos de trabalho (modelos, documentos, dados, relatórios, formulários, etc.) são produzidos, os marcos são estabelecidos, a qualidade é assegurada e as modificações são adequadamente geridas.



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Os métodos de engenharia de *software* fornecem a técnica de "como fazer" para construí-los. Eles abrangem um amplo conjunto de tarefas que incluem comunicação, análise de requisitos, modelagem de projeto, construção de programas, testes e manutenção. Os métodos de engenharia de *software* repousam num conjunto de princípios básicos que regem cada área da tecnologia e incluem atividades de modelagem e outras técnicas descritivas. As ferramentas de engenharia de *software* fornecem apoio automatizado ou semiautomatizado para o processo e para os métodos. Quando ferramentas são integradas de modo que a informação criada por uma ferramenta posa ser usada por outra, um sistema de apoio ao desenvolvimento de *software*, chamado engenharia de *software* apoiada por computador, é estabelecido.

Um Arcabouço de Processo

Um arcabouço de processo estabelece o alicerce para um processo de *software* completo pela identificação de um pequeno número de atividades de arcabouço aplicáveis a todos os projetos deste, independentemente de seu tamanho ou complexidade. Além disso, o arcabouço de processo engloba um conjunto de atividades guarda-chuva que são aplicáveis durante todo o processo de *software*.

Cada atividade de arcabouço é preenchida por um conjunto de ações de engenharia de *software* – uma coleção de tarefas relacionadas que produz um produto importante do trabalho de engenharia de *software* (por exemplo, o projeto é uma ação de engenharia de *software*). Cada ação é preenchida por tarefas de trabalho Individuais que realizam alguma parte do trabalho determinado pela ação.

O seguinte arcabouço de processo genérico (usado como base para a descrição dos modelos de processo nos capítulos subsequentes) é aplicável à grande maioria dos projetos e *software*:

Comunicação: Essa atividade de arcabouço envolve alta comunicação, colaboração com o cliente (e os interessados) e abrange o levantamento de requisitos e outras atividades relacionadas.

Planejamento: Estabelece um plano para o trabalho de engenharia de *software* que se segue. Descrevem as tarefas técnicas que serão conduzidas, os riscos prováveis, os recursos que serão necessários, os produtos de trabalho a serem produzidos e um cronograma de trabalho.

Modelagem: Essa atividade inclui a criação de modelos que permitam ao desenvolvedor e ao cliente entendem melhor os requisitos do *software* e o projeto que vai satisfazer a esses requisitos.

Construção: Essa atividade combina geração de código (quer manual ou automática) e os testes necessários para revelar erros no código.

Implantação: O *software* (como entidade completa ou incremento parcialmente completo) é entregue ao cliente que avalia o produto entregue e fornece *feedback* com base na avaliação.

Essas cinco atividades genéricas de arcabouço podem ser usadas durante o desenvolvimento de pequenos programas, durante a criação de grandes aplicações para a *Internet* e para a engenharia de grandes e complexos sistemas baseados em computador. Os detalhes do processo de *software* serão muito diferentes em cada caso, mas as atividades de arcabouço permanecem as mesmas.

Usando um exemplo derivado do arcabouço de processo genérico, a atividade de modelagem é composta de duas ações de engenharia de *software* – análise e projeto. A análise engloba um conjunto de tarefas de trabalho (por exemplo, levantamento, elaboração, negociação, especificação e validação de requisitos) que conduzem à criação de modelos de análise (e/ou especificação de projeto).

Cada ação de engenharia de *software* é representada por um número diferente como uma coleção de tarefas de trabalho de engenharia de *software*, produtos de trabalho relacionados, pontos de garantia de qualidade e marcos de projeto. O conjunto de tarefas escolhido é o que melhor acomoda as necessidades do projeto e as características da equipe. Isso implica que uma ação de engenharia de *software* (por exemplo, o projeto) pode ser adaptada às necessidades específicas do projeto de *software* e às características da equipe do projeto.

O arcabouço descrito na visão genérica de engenharia de *software* é complementado por várias atividades guarda-chuva. Atividades típicas dessa categoria incluem:

- **Acompanhamento e controle de projeto de *software*** – permite à equipe progresso com base no plano de projeto e tomar ação necessária para manter o cronograma.
- **Gestão de risco** – avalia os riscos que podem afetar o resultado do projeto ou a qualidade do produto.
- **Garantia de qualidade de *software*** – define e conduz as atividades necessárias para garantir a qualidade do *software*.
- **Revisões técnicas formais** – avaliam os produtos de trabalho da engenharia de *software*, num esforço para descobrir e remover erros antes que eles sejam propagados para a próxima ação ou atividade.
- **Medição** – define e reúne medidas de processo, projeto e produto que ajudam a equipe a entregar um *software* que satisfaça às necessidades do usuário; pode ser usada conjugada com todas as outras atividades de arcabouço e guarda-chuva.
- **Gestão de configurações de *software*** – gerencia os efeitos das modificações ao longo de todo o processo de *software*.
- **Gestão de reusabilidade** – define critérios para a reutilização dos produtos de trabalho (inclusive componentes de *software*) e estabelece mecanismos para obter componentes recusáveis.

Preparação e produção do produto do trabalho abrange as atividades necessárias para criar produtos do trabalho como modelos, documentos, registros, formulários e listas.

As atividades guarda-chuva são aplicadas ao longo de todo o processo de *software*.

A aplicação inteligente de qualquer modelo de processo de *software* deve reconhecer que a adaptação (ao problema, ao projeto, à equipe e à cultura organizacional) é essencial para o sucesso. Mas os modelos de processo diferem fundamentalmente:

- No fluxo geral de atividades e tarefas e as interdependências entre atividades e tarefas;
- No grau em que as tarefas de trabalho são definidas dentro de cada atividade de arcabouço;
- No grau em que os produtos do trabalho são identificados e solicitados;
- Na maneira como as atividades de garantia de qualidade são aplicadas;
- No modo como o monitoramento de projeto e as atividades de controle são aplicados;
- No grau geral de detalhes e rigor em que processo é descrito;
- No grau em que clientes e outros interessados estão envolvidos no projeto;
- No nível de autonomia da equipe de projeto de *software*;
- No grau em que a organização da equipe e os papéis são prescritos.

Modelos de processo que enfatizam a definição, identificação e aplicação detalhada de atividades e tarefas de processo têm sido aplicadas na comunidade de engenharia de *software* durante os últimos trinta anos. Quando esses modelos prescritivos de processo são aplicados, o objetivo é melhorar a qualidade do sistema para tornar os projetos mais gerenciáveis, as datas de entrega e os custos mais previsíveis e para guiar equipes de engenheiros de *software* à medida que eles realizam o trabalho necessário para construir um sistema.

Infelizmente, tem havido épocas em que esses objetivos não têm sido alcançados. Se os modelos prescritivos forem aplicados dogmaticamente e sem adaptação, eles podem aumentar o nível de burocracia associado à construção de sistemas baseados em computador

e, inadvertidamente, criar dificuldades para desenvolvedores e clientes.

Modelos de processo que enfatizam a agilidade do projeto e seguem uma série de princípios que levam a uma abordagem mais informal (mas, segundo seus proponentes, não menos efetivas) do processo de *software* que foram propostos nos últimos anos. Esses modelos de processo ágil enfatizam a as manobras e a adaptabilidade. Eles são adequados a muitos tipos de projeto e são particularmente úteis quando aplicações *Web* passam por engenharia.

Que filosofia de processo de *software* é melhor? Essa questão tem provocado um debate emocional entre engenheiros de *software*. Por hora, é importante notar que essas duas filosofias de processo têm um objetivo comum – criar *softwares* de alta qualidade que satisfaçam às necessidades do cliente -, mas abordagens diferentes.

O CMMI (*Capatibility Maturity Model Integration*)

O SEI (*Software Engineering Institute*) desenvolveu um abrangente metamodelo de processo baseado em um conjunto de capacidades de engenharia de *software* que devem estar presentes à medida que as empresas alcançam diferentes níveis de capacidade e maturidade de processo. Para conseguir essas capacidades, o SEI alega que uma organização deve desenvolver um modelo-processo que siga as diretrizes do CMMI.

O CMMI representa um modelo de processo de dois modos diferentes: (1) como um modelo contínuo e (2) como um modelo de estágios. O CMMI contínuo descreve um processo em duas dimensões. Cada área de processo (por exemplo, planejamento de projeto ou gestão de requisitos) é avaliada formalmente com base em metas e práticas específicas e é classificada de acordo com os seguintes níveis de capitação:

- **Nível 0**: Incompleto. A área de processo (por exemplo, gestão de requisitos) não é realizada e não atinge todas as metas e objetivos pelo CMMI para o nível 1 de capacitação.´
- **Nível 1**: Realizado. Todas as metas específicas da área de processo (como definidas pelo CMMI) foram satisfeitas. As tarefas de trabalho necessárias para produzir os produtos de trabalho definidos estão sendo conduzidas.
- **Nível 2**: Gerido. Todos os critérios do nível 1 foram satisfeitos.

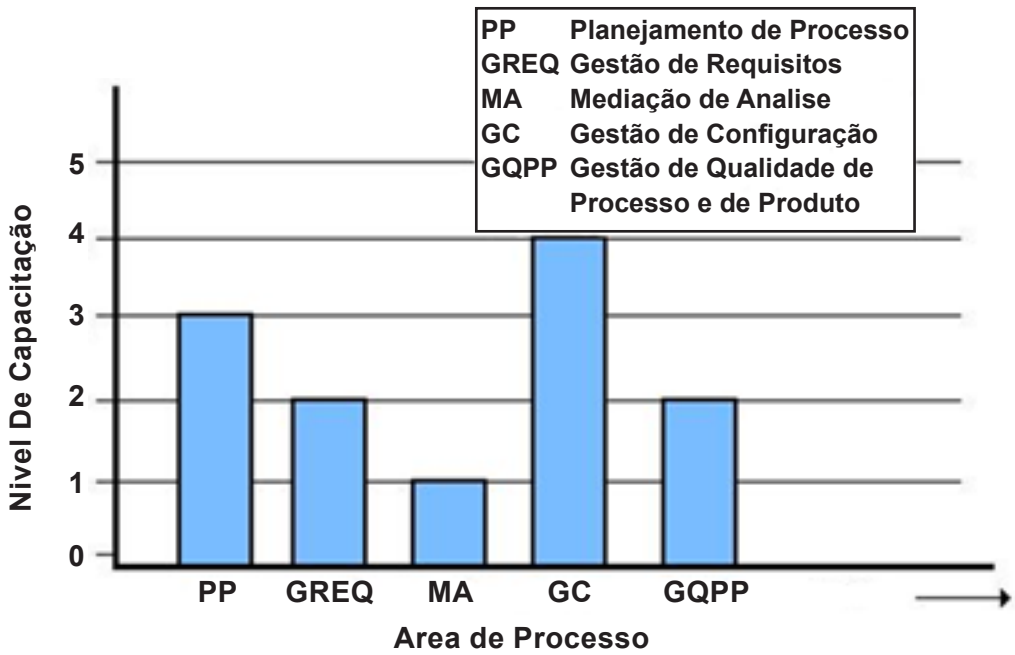
Além disso, todo trabalho associado à área de processo está de acordo com a política defendida pela organização; todo o pessoal que está fazendo o trabalho tem acesso aos recursos adequados para fazer o serviço; os interessados estão ativamente envolvidos nas áreas de processo, conforme necessário; todas as tarefas e produtos de trabalho são "monitorados, controlados e revisados e são avaliados quanto à aderência à descrição do processo".

- **Nível 3**: Definido. Todos os critérios do nível 2 foram alcançados. O processo é "feito sob medida para o conjunto-padrão de processos de organização, de acordo com as suas diretrizes quanto a fazer coisas sob medida e contribui com produtos de trabalho medições e outras informações de aperfeiçoamento de processo de patrimônio de processos da organização".
- **Nível 4**: Quantitativamente gerido. Todos os critérios do nível 3 foram alcançados. E ainda, as áreas de processo são controladas e aperfeiçoadas usando medições e avaliação quantitativa.
- "Objetivos quantitativos para qualidade e desempenho de processo são estabelecidos e utilizados como critério na gestão de processo".
- **Nível 5**: Otimizado. Todas as capacidades do nível 4 foram alcançadas. Assim como, a área de processo é adaptada e otimizada usando meios quantitativos (estatísticos) para satisfazer às alterações de necessidade do cliente e continuamente aperfeiçoar a eficácia da área de processo em consideração.

O CMMI define cada área de processo em termos de "metas específicas" e das "práticas específicas" necessárias para atingis tais metas. As específicas estabelecem as características que devem existir se as atividades determinadas por uma área de processo tiverem de ser efetivas. Já as práticas específicas refinam uma meta num conjunto de atividades relacionadas ao processo.

Por exemplo, o planejamento de projeto é uma das oito áreas de projeto definidas pelo CMMI para a categoria "gestão de processos". As metas específicas (ME) e as práticas específicas (PE) associadas ao planejamento de projeto são:

ME 1 Estabelecer estimativas



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

PE 1.1-1 Estime o corpo do projeto;
PE 1.2-1 Estabeleça estimativas de produto do trabalho e atributos de tarefas;
PE 1.3-1 Defina o ciclo de vida do projeto;
PE 1.4-1 Determine estimativas de esforço e de custo.

ME 2 Desenvolver um plano de projeto
PE 2.1-1 Estabeleça o orçamento e o cronograma;
PE 2.2-1 Identifique os riscos do projeto;
PE 2.3-1 Planeje a gestão de dados;
PE 2.4-1 Planeje os recursos do projeto;
PE 2.5-1 Planeje as habilidades e conhecimentos necessários;
PE 2.6-1 Planeje o envolvimento dos interessados;
PE 2.7-1 Estabeleça o plano de projeto.

ME 3 Obter comprometimento com o plano
PE 3.1-1 Revise planos que afetam o projeto;
PE 3.2-1 Reconcilie os níveis de trabalho e os recursos;
PE 3.3-1 Obtenha comprometimento com o plano.

Além de metas e práticas específicas, o CMMI também define um

conjunto de cinco metas genéricas e práticas relacionadas para cada área de processo. Cada uma das cinco metas genéricas corresponde a um dos cinco níveis de capacitação. Assim, para alcançar um determinado nível de capacitação, a meta genérica para aquele nível e as práticas genéricas que correspondem àquela meta precisa ser alcançada. Para ilustrar, as metas genéricas (MG) e as práticas genéricas (PG) para a área de processo de planejamento de projeto são:

MG 1 Alcançar metas específicas;
PG 1.1 Realize as práticas básicas;
MG 2 Institucionalizar um processo gerido;
PG 2.1 Estabeleça uma política organizacional;
PG 2.2 Planeje o processo;
PG 2.3 Forneça os recursos;
PG 2.4 Atribua responsabilidades;
PG 2.5 Treine o pessoal;
PG 2.6 Gerencie configurações;
PG 2.7 Identifique e envolva os interessados relevantes;
PG 2.8 Monitore e controle o processo;
PG 2.9 Avalie objetivamente a aderência;
PG 2.10 Revise o estado com a gestão de nível superior;
MG 3 Institucionalizar um processo definido;
PG 3.1 Estabeleça um processo definido;
PG 3.2 Reúna informações de aperfeiçoamento;
MG 4 Institucionalizar um processo quantitativamente gerido;
PG 4.1 Estabeleça objetivos quantitativos para o processo;
PG 4.2 Estabilize o desempenho dos subprocessos;
MG 5 Institucionalizar um processo otimizado;
PG 5.1 Garanta aperfeiçoamento contínuo do processo;
PG 5.2 Corrija as causas básicas dos problemas.

O modelo em estágios do CMMI define as mesmas áreas de processo, metas e práticas que o modelo contínuo. A principal diferença é que o modelo em estágios define em cinco níveis de maturidade em vez de cinco níveis de capacitação. Para atingir um nível de maturidade, as metas e práticas específicas associadas a um conjunto de áreas de processo precisam ser alcançadas.

Padrões de Processo

O processo de *software* pode ser definido como uma coleção de padrões que definem um conjunto de atividades, ações, tarefas de trabalho, produtos de trabalho e/ou comportamentos relacionados necessários ao desenvolvimento de *softwares* de computadores. Dito em termos gerais, um padrão de processos nos fornece um gabarito – um método consiste para descrever uma característica importante do processo de *software*. Pela combinação de padrões, uma equipe de *software* pode construir um processo que melhor satisfaça às necessidades de um projeto.

Nivel	Foco	Area De Processo
Otimizado	Aperfeiçoamente Contínuo de Processo	Inivação e Implantação Organizacional Análise e Resolução Casual
Quantitativamente Gerido	Gestão Quantitativa	Desempenho de Processo Organizacional Gestão Quantitativa de Projeto
Definido	Padronização de Processo	Desenvolvimento de Requisitos Solução Técnica Integração de Produtos Verificação Validação Foco do Processo Organizacional Definição do Processo Organizacional Treinamento Organizacional Gestão Integrada de Projeto Gestão Integrada de Fornecedor Gestão de Risco Análise e Resolução de Decisão Ambiente Organizacional para Integração Formação Integrada de Equipe
Gerido	Gestão Básica de Processo	Gestão de Requisistos Planejamento de Projeto Monitoramento e Controle de Projeto Gestão de Acordo com o Fornecedor Mediação e Análise Garantia de Qualidade de Processo e de Produto
Realizado		

Fonte: adaptado de Curtis *et al*. (1995, p. 50).

Padrões podem ser definidos em qualquer abstração. Em alguns casos, um padrão pode ser usado para descrever um processo completo (por exemplo, prototipação). Em outras situações, eles podem ser usados para descrever uma importante atividade de arcabouço (por exemplo,

estimativa de projeto).

Ambler (1998) propôs o seguinte gabarito para descrever um padrão de processo.

Nome padrão: Ao padrão é dado um nome significativo que descreva sua função dentro do processo de *software* (por exemplo, comunicação-com-o-cliente).

Intenção: O objetivo do padrão é descrito brevemente. Por exemplo, a intenção de comunicação-com-o-cliente é "estabelecer um relacionamento colaborativo com o cliente esforçando-se para definir o escopo do projeto, os requisitos do negócio e outras restrições do projeto". A intenção pode ser mais expandida com texto explanatório adicional e diagramas apropriados, se necessários.

Tipo. O tipo de padrão é especificado. Ambler (1998) sugere três tipos:

- Os padrões de tarefa definem uma ação de engenharia de *software* ou tarefa de trabalho que é parte do processo relevante para a prática bem-sucedida da engenharia de *software* (por exemplo, levantamento de requisitos é um padrão de tarefa);
- Os padrões de estágio definem uma atividade de arcabouço para o processo. Como uma atividade de arcabouço abrange várias tarefas de trabalho, um padrão de estágio incorpora vários padrões de tarefa que são relevantes para o estágio (atividade de arcabouço). Um exemplo de um padrão de estágio poderia ser comunicação. Esse padrão incorpora o padrão de tarefa de levantamento de requisitos e outros.
- Os padrões de fase definem a sequência de atividades de arcabouço que ocorrem no processo, mesmo quando o fluxo global de atividades é a natureza interativa. Um exemplo de padrão de fase poderia ser um modelo espiral ou prototipação.

Contexto inicial: As condições sobre a quais o padrão se aplica são descritas. Antes da iniciação do padrão, pergunta-se: (1) Que atividades relacionadas à organização ou à equipe já ocorreram? (2) Qual é o estado das entradas para o processo? (3) Que informações de engenharia de *software* ou informações de projeto já existem?

Por exemplo, o padrão planejamento (um padrão de estágio) exige que (1) os clientes e os engenheiros de *software* tenham estabelecido uma comunicação colaborativa; (2) a conclusão bem-sucedida de

certo número de padrões de tarefa (especificados) para que o padrão comunicação-com-o-cliente tenha ocorrido; (3) escopo do projeto, os requisitos do negócio e as restrições de projeto sejam conhecidos.

Problema: O problema a ser resolvido pelo padrão é descrito. Por exemplo: o problema a ser resolvido por comunicação-com-o-cliente poderia ser descrito da seguinte maneira: a comunicação entre o desenvolvedor e o cliente é frequentemente inadequada porque um formato efetivo para fazer o levantamento de informações não é estabelecido, um mecanismo útil para registrá-las não é criado e uma revisão significativa não é realizada.

Solução: A implementação do padrão é descrita. Esta seção discute como o estado inicial do processo (que existe antes do padrão ser implementado) é modificado em consequência da iniciação do padrão. Descreve também como informação de engenharia de *software* ou a informação de projeto que está disponível antes da iniciação do padrão é transformada como consequência bem-sucedida do padrão.

Contexto Resultante: As condições que resultarão quando o padrão tiver sido implementado com sucesso são descritas. Ao término do padrão pergunta-se: (1) Que atividades relacionadas à organização ou à equipe devem ter ocorrido? (2) Qual é estado de saída do processo? (3) Que informações de engenharia de *software* ou informações de projeto foram desenvolvidas?

Padrões Relacionados: Uma lista de todos os padrões de processo diretamente relacionados a este é fornecida – como uma hierarquia ou em alguma outra forma diagramática. Por exemplo: o padrão de estágio comunicação inclui padrões de tarefa montagem da equipe de projeto, definição colaborativa de diretrizes, isolamento do escopo, levantamento de requisitos, descrição de restrições e criação de miniespecies/modelos.

Usos Conhecidos/Exemplos: São indicadas as instâncias específicas nas quais o padrão é aplicável. Por exemplo, a comunicação é obrigatória no início de cada projeto de *software*; recomendada ao longo do projeto deste; é obrigatória quando a atividade implantação está no curso.

Os padrões de projeto fornecem um mecanismo efetivo para descrever qualquer processo de *software*. Eles permitem a organização de engenharia de *software* para desenvolver uma descrição hierárquica do processo começando em um alto nível de abstração (um padrão de fase). A descrição é refinada em um conjunto de padrões de estágio que descrevem atividades de arcabouço e ainda mais refinada de modo

hierárquico em padrões de tarefa mais detalhados para cada padrão de estágio. Uma vez que os padrões de processo tenham sido desenvolvidos, eles podem ser reutilizados para a definição de variantes de processo – isto é, um modelo de processo personalizado pode ser definido por uma equipe de software usando padrões como blocos construtivos para o modelo de processo.

Avaliação de Processo

A existência de um processo de *software* não é garantia de que o *software* será entregue no prazo, de que ele satisfaça às necessidades do cliente ou de que ele exiba as características técnicas que levarão a características de qualidade no longo prazo. Padrões de processo precisam ser acoplados à sólida prática da engenharia de *software*. Além disso, o processo em si deve ser avaliado para garantir que ele satisfaça a um conjunto de critérios básicos de processo que demonstraram ser essenciais para uma engenharia de *software* bem-sucedida. Várias e diferentes abordagens de avaliação de processo e *software* têm sido propostas durante as últimas décadas.

O SCAMPI (*Standard CMMI Assessment Method for Process Improvement*_ Método de Avaliação Normalizado do CMMI para Aperfeiçoamento de Processo) fornece um modelo de processo de avaliação com cinco passos que incorpora a iniciação, o diagnóstico, o estabelecimento, a ação e o aprendizado. O método SCAMPI usa o CMMI da SEI (*Software Engineering Institute*) como base para a avaliação.

A CBA (*CMM-Based Appraisal for Internal Process Improvement*_ Avaliação do Processo de Aperfeiçoamento Interno Baseado no CMM) fornece uma técnica de diagnóstico para avaliar a maturidade relativa de uma organização de *software* usando o CMM da SEI (Precursor do CMMI) como base para a avaliação [DUN82].

A norma SPICE (ISSO/IEC 15504) define um conjunto de requisitos para avaliação de processo de *software*. A intenção da norma é assistir às organizações no desenvolvimento de uma avaliação objetiva da eficácia de qualquer processo definido de *software* [SPI92].

A ISSO 9001:2000 para *Software* é uma norma genérica que se aplica a qualquer organização que deseje aperfeiçoar a qualidade global dos produtos, sistemas ou serviços que ela fornece. Assim, a norma é diretamente aplicável a organizações e empresas de *software*.

A ISSO (*Internacional Organization for Standardization*)

desenvolveu a norma ISSO 9001:2000 [ISO00] para definir os requisitos de um sistema de gestão de qualidade que servirá para produzir produtos de alta qualidade e, conseqüentemente, aumentar a satisfação do cliente.

A estratégia subjacente sugerida pela ISSO 9001:2000 é descrita da seguinte maneira :

A ISSO 9001:2000 enfatiza a importância de uma organização para identificar, implementar, gerir e aperfeiçoar continuamente a efetividade dos processos necessários ao sistema de gestão de qualidade e gerir as interações desses processos a fim de alcançar os objetivos da organização.

A ISSO 9001:2000 adotou um ciclo “planejar-fazer-verificar-agir” que é aplicado aos elementos de gestão de qualidade de um processo de *software*. Dentro do contexto de *software*, “planejar” estabelece os objetivos do processo, suas atividades e as tarefas que são necessárias para a obtenção de *software* de alta qualidade e a resultante satisfação do cliente. “Fazer” implementa o processo de *software* (incluindo atividades de arcabouço e guarda-chuva). “Verificar” monitora e mensura o processo a fim de garantir que todos os requisitos estabelecidos para a gestão de qualidade tenham sido atingidos. “Agir” inicia as atividades de aperfeiçoamento do processo de *software* que trabalham continuamente para aperfeiçoar o processo.

Para uma discussão detalhada, consultar as próprias normas ISSO.

Modelos de Processo Pessoal e de Equipe

O melhor processo de *software* é aquele que se aproxima do pessoal que fará o serviço. Se um modelo de processo tiver sido desenvolvido no nível da empresa ou organização, ele pode ser efetivo apenas se for suscetível a adaptações significativas de modo a satisfazer às necessidades da equipe de projeto que está realmente fazendo o trabalho de engenharia de *software*.

Numa instalação ideal, cada engenheiro de *software* criaria um processo que melhor satisfizesse às suas necessidades e, ao mesmo tempo, às necessidades mais globais da equipe e da organização. Alternativamente, a própria equipe poderia criar seu próprio processo e, ao mesmo tempo, satisfazer às necessidades mais restritas dos indivíduos e às necessidades mais amplas da organização. Watts Humphrey (1997 e 2000) argumenta que é possível criar um “processo

pessoal de *software*" e/ou um "processo de equipe de *software*". Ambos requerem trabalho árduo, treinamento e coordenação, mas ambos são passíveis de obtenção.

PSP (*Personal Software Process* – Processo Pessoal de *Software*)

Todo desenvolvedor usa algum processo para construir *softwares* de computador. O processo pode ser aleatório ou ad hoc, pode modificar-se diariamente, pode não ser eficiente, efetivo ou, mesmo, bem-sucedido, mas existe um processo. Watts Humphrey (1997) sugere que para modificar um processo pessoal não efetivo, o indivíduo precisa passar por quatro fases, cada uma exigindo treinamento e cuidadosa instrumentação. O PSP (Processo pessoal de *software*) enfatiza a medição pessoal tanto do produto do trabalho que é produzido quanto à qualidade resultante do produto (por exemplo: fazer orçamentos e cronogramas) e dá poder ao profissional para controlar a qualidade de todos os produtos do trabalho de *software* que são desenvolvidos.

O modelo de processo de PSP define cinco atividades de arcabouço: planejamento, o projeto de alto nível, revisão de projeto de alto nível, desenvolvimento e pós-conclusão.

Planejamento: Essa atividade isola os requisitos e, com base neles, desenvolvem estimativas tanto de tamanho quanto de recursos. Além disso, é feita uma estimativa de defeitos (o número de defeitos projetado para o trabalho). Toda a métrica é registrada em planilhas ou gabaritos. Finalmente, as tarefas de desenvolvimento são identificadas e um cronograma de projeto é criado.

Projeto de alto nível: São desenvolvidas especificações externas para cada componente a ser construído e é criado um projeto dos componentes. Protótipos são construídos quando existe incerteza. Todos os itens são registrados e monitorados.

Revisão do projeto de alto nível: Métodos de verificação formal são aplicados para descobrir erros no projeto. A métrica é mantida para todas as tarefas e resultados importantes do trabalho.

Desenvolvimento: O projeto em nível de componente é refinado e revisado. O código é gerado, revisado, compilado e testado. A métrica é mantida para todas as tarefas e resultados importantes do trabalho.

Pós-conclusão: Usando as medias e a métrica coletadas (uma quantidade substancial de dados que devem ser analisados

estatisticamente), a efetividade do projeto é determinada. As medidas e a métrica devem fornecer diretrizes para a modificação do processo de modo a aperfeiçoar a sua efetividade.

O PSP enfatiza a necessidade de cada engenheiro de *software* identificar logo os erros e, igualmente importante, entender os tipos de erro que ele tende a fazer. Isso é conseguido por meio de uma atividade de avaliação rigorosa desenvolvida em todos os produtos de trabalho produzidos pelo engenheiro.

O PSP representa uma abordagem disciplinada, baseada na métrica da engenharia de *software*, que pode levar a um choque de culturas em muitos profissionais. No entanto, quando o PSP é adequadamente apresentado aos engenheiros de *softwares* (HUMPREY, 1996), o aperfeiçoamento resultante na produtividade da engenharia de *software* e na qualidade deste é significativo. No entanto, O PSP não tem sido amplamente adotado pela indústria. As razões, infelizmente, têm mais a ver com a natureza humana e a inércia organizacional que com os pontos fortes e fracos da abordagem do PSP.

O PSP é intelectualmente desafiador e exige um nível de comprometimento (dos profissionais e de seus gerentes) que nem sempre é possível ser obtido. O treinamento é relativamente demorado e os custos do treinamento são altos. O nível de medição exigido é culturalmente difícil para muitos entre o pessoal de *software*.

TSP (Team *Software* Process – Processo de Equipe de Processo

Como muitos projetos de *software* de nível industrial são desenvolvidos por uma equipe de profissionais, Watts Humphrey estendeu as lições aprendidas com a introdução do PSP e propôs o TSP (Processo de equipe de *software*). O objetivo do TSP é construir uma equipe de projeto “autodirigida” que se organize para produzir *softwares* de alta qualidade. Humphrey (1998) define os seguintes objetivos para o TSP:

- Construir equipes autodirigidas que planejem e monitorem seu trabalho, estabeleçam metas e possuam seus próprios processos e planos. Podem ser equipes essencialmente de *softwares* ou equipes de produto integrado (IPT – *Integrated Product Team*) de três a aproximadamente 20 engenheiros;
- Mostrar aos gerentes como acompanhar e motivar suas equipes e como ajudá-las a manter desempenho de pico;

- Acelerar o aperfeiçoamento do processo de *software* tornando o comportamento de nível 5 do CMM normal e esperado;
- Fornecer diretrizes de aperfeiçoamento para organizações de alta maturidade;
- Facilitar o ensino universitário das habilidades de equipe de nível industrial.

Uma equipe autodirigida tem um entendimento consistente de suas metas e objetivos gerais. Ela define papéis e responsabilidades para cada membro da equipe; monitora dados de projetos quantitativos (sobre produtividade e a qualidade); identifica um processo de equipe apropriado para o projeto e uma estratégia para implementação do processo; define normas locais aplicáveis ao trabalho de engenharia de *software*; avalia continuamente o risco e reage a isso; além de monitorar, gerenciar e relatar o estado do projeto.

O TSP define as seguintes atividades de arcabouço: lançamento, projeto de alto nível, implementação, integração e teste, e pós-conclusão. Como suas contrapartidas no PSP (note que a terminologia é um tanto diferente), essas atividades permitem que a equipe planeje, projetem e construam *software* de modo disciplinado e, ao mesmo tempo, meça quantitativamente o processo e o produto. A pós-conclusão prepara o cenário para o aperfeiçoamento do processo.

O TSP usa uma grande variedade de documentos, formulários e normas que servem para guiar os membros da equipe no trabalho deles. Os documentos definem as atividades específicas do processo (isto é, lançamento, projeto, implementação, integração e teste, e pós-conclusão do projeto) e outras funções de trabalho mais detalhadas (por exemplo, planejamento e desenvolvimento, desenvolvimento de requisitos, gestão de configuração de *software* e testes unitários) que fazem parte do processo de equipe. Para ilustrar, considere a atividade inicial do processo – lançamento do projeto.

Cada projeto é “lançado” usando-se uma sequência de tarefas (definidas como um documento) que permite à equipe estabelecer uma base sólida para iniciar o projeto. Recomenda-se o seguinte documento de lançamento (apenas o esboço) (HUMPREY, 2000):

- Revise os objetivos do projeto com a gerência, entre em acordo e documente os objetivos da equipe;
- Estabeleça os papéis da equipe;
- Defina o processo do desenvolvimento da equipe;

- Faça um planejamento de qualidade e estabeleça metas de qualidade;
- Planeje as instalações de apoio necessárias;
- Produza uma estratégia global de desenvolvimento;
- Faça um plano de desenvolvimento para todo o projeto;
- Faça planos detalhados para cada engenheiro na fase seguinte;
- Integre os planos individuais num plano de equipe;
- Refaça o balanceamento da carga de trabalho da equipe para obter um cronograma mínimo geral;
- Avalie os riscos de projeto e atribua responsabilidades de monitoramento para cada risco-chave.

Deve-se notar que a atividade de lançamento pode ser aplicada antes de cada atividade de arcabouço TSP mencionada anteriormente. Isso acomoda a natureza interativa de muitos projetos e permite à equipe adaptar-se às necessidades mutantes do cliente e às lições aprendidas nas atividades anteriores.

O TSP reconhece as melhores equipes de *softwares* que são autodirigidas. Os membros da equipe estabelecem os objetivos do projeto, adaptam o processo para satisfazer às suas necessidades, tem controle sobre o cronograma e, por meio da medição e análise da métrica coletada, trabalham continuamente para melhorar a abordagem rigorosa da engenharia de *software*.

Como o PSP, o TSP é uma abordagem rigorosa da engenharia de *software* que fornece benefícios distintos e quantificáveis em produtividade e qualidade. A equipe deve estabelecer um total comprometimento com o processo e deve passar por treinamento para garantir que a abordagem seja propriamente aplicada.

Tecnologia de Processo

Os modelos de processos genéricos discutidos nas seções precedentes devem ser adaptados para o uso de uma equipe de projeto de *software*. Para conseguir isso, foram desenvolvidas ferramentas de tecnologia de processo a fim de ajudar as organizações e analisar seu processo atual, organizar tarefas de trabalho, controlar e monitorar o progresso e gerenciar a qualidade técnica.

As ferramentas de tecnologia de processo permitem a uma

organização de *software* que constroem um modelo automatizado de arcabouço comum de processo, dos conjuntos de tarefas e das atividades guarda-chuva. O modelo, normalmente representado por uma rede, pode então ser analisado para determinar o fluxo típico de trabalho e examinar estruturas de processo alternativas que possam reduzir o tempo do desenvolvimento ou o custo.

Uma vez criado um processo aceitável, outras ferramentas de tecnologia de processo podem ser usadas para atribuir, monitorar e até mesmo controlar todas as tarefas de engenharia de *software* definidas como parte do modelo de processo. Cada membro de uma equipe de *software* pode usar tais ferramentas para desenvolver um checklist das tarefas de trabalho a serem desenvolvidas, dos produtos de trabalho a ser produzidos e das atividades de garantia de qualidade a serem conduzidas. A ferramenta de tecnologia de *software* de processo também pode ser usada para coordenar o uso de outras ferramentas de engenharia de *software* apoiadas por computador que sejam adequadas a uma tarefa específica.

Ferramentas de Modelagem de Processo

Objetivo: Se uma organização trabalha para aperfeiçoar um processo de negócio (ou *software*), ela deve primeiramente entendê-lo. As ferramentas de modelagem de processo (também chamadas de ferramentas de tecnologia de processo ou gestão de processo) são usadas para representar os elementos-chave de um processo de modo que ele possa ser mais bem entendido. Tais ferramentas podem também fornecer *links* para descrições do processo que ajudem os que estão envolvidos no processo a entender as ações e tarefas de trabalho necessárias para realizá-lo. Ferramentas de modelagem de processo fornecem *links* para outras ferramentas que fornecem apoio a atividades definidas do processo.

Mecânica: As ferramentas dessa categoria permitem a uma equipe definir os elementos de um modelo de processo específico (ações, tarefas, produtos de trabalho, pontos de garantia de qualidade), fornecer diretrizes detalhadas sobre o conteúdo ou descrição de cada elemento do processo e, depois, gerir o projeto à medida que ele é conduzido. Em alguns casos, as ferramentas de tecnologia de processo incorporam tarefas-padrão de gestão de projeto tais como orçar, montar cronogramas, monitorar e controlar.

Ferramentas Representativas: As ferramentas *Igrafx Process Tools*, distribuídas pela *Corel Corporation* (www.igrafx.com/products/process), são um conjunto de ferramentas que permitem a uma equipe mapear, medir e modelar o processo de *software*.

O *Objexis Team Portal*, desenvolvido pela *Objexis Corporation* (www.objexis.com), fornece definição e controle para todo o fluxo de trabalho do processo.

Produto e Processo

Se o processo for fraco, o produto final sofrerá inevitavelmente, mas uma ênfase exagerada no processo também é perigosa. Em um ensaio resumido, Margaret Davis [DAV,95] comenta a dualidade do produto e do processo:

> A cada dez anos, avançando ou recuando cinco, a comunidade de *software* redefine "o problema" mudando o foco de aspectos do produto para aspectos do processo. Assim, adotamos linguagens de programação estruturada (produto), seguidas de métodos de análise estruturada (processo) e, depois, do encapsulamento de dados (produto), da ênfase atual no Modelo de Maturidade da Capacidade de Desenvolvimento de *Software*, do SEI (*Software Enginnering Institute*) (processo) [seguido por métodos orientados a objetos e desenvolvimento ágil de *softwares*].

Enquanto um pêndulo tende a ficar naturalmente em repouso no ponto intermediário entre os dois extremos, o foco da comunidade de *software* desloca-se constantemente, pois uma nova força é aplicada quando a última oscilação falha. Essas oscilações são prejudiciais em si e por si porque confunde o profissional de *software* médio modificado radicalmente o significado de realizar o trabalho ou de realizá-lo bem. As oscilações também não resolvem "o problema", pois estão fadadas a falhar, enquanto o produto e processo forem tratados como uma dicotomia em vez de uma dualidade.

Há precedência na comunidade científica para sugerir noções de dualidade quando contradições nas observações não podem ser totalmente explicadas por uma ou outra teoria competitiva. A natureza dual da luz, que parece ser simultaneamente partícula e onda, tem sido aceita desde 1920, quando Louis de Broglie a propôs. Acredito

que as observações que podemos fazer das ferramentas de *software* e do seu desenvolvimento demonstraram uma dualidade fundamental entre o produto e o processo. Não se pode deduzir ou entender toda a ferramenta, seu contexto, uso, significado e valor se a virmos somente como um processo ou somente como um produto.

Toda a atividade humana pode ser um processo, mas cada um de nós obtém um senso de autovalor daquelas atividades que resultam numa representação ou instância que pode ser usada ou apreciada por mais de uma pessoa, utilizada repetidamente ou empregada em algum outro contexto não considerado. Isto é, obtemos um sentimento de satisfação pelo reuso de nossos produtos por nós mesmos ou pelos outros.

Assim, enquanto a rápida assimilação de metas de reutilização no desenvolvimento de *software* aumenta potencialmente a satisfação dos profissionais de *software* com seu trabalho, ela aumenta também a urgência da aceitação da dualidade entre o produto e processo. Pensando em um artefato somente como produto ou somente como um processo, ou obscurece o contexto ou os modos de usá-lo, ou obscurece o fato de que cada uso resulta num produto que, por sua vez, será usado como entrada para alguma outra atividade de desenvolvimento de *software*. Aumentar uma visão ao invés de outra reduz substancialmente as oportunidades de reutilização e, consequentemente, as oportunidades de satisfação crescente com o trabalho.

As pessoas obtêm tanta (ou mais) satisfação de um processo criativo quanto de um produto final. Um artista aprecia tanto as pinceladas quanto o resultado emoldurado. Um escritor aprecia tanto a busca de uma metáfora adequada quanto um livro pronto. Um profissional de *software* criativo também deveria obter tanta satisfação do processo quanto do produto final.

O trabalho do pessoal de *software* vai se modificar nos próximos anos. A dualidade entre produto e processo é um elemento importante para manter pessoas criativas engajadas enquanto a transição da programação para a engenharia de *software* é concluída.



Exercícios Propostos

1. Conceitue Engenharia de *Software*.
2. Pesquise sobre a Tecnologia em camadas.
3. Quais são os elementos considerados no arcabouço de processo genérico?
4. Qual o objetivo das Ferramentas de Modelagem de Processo.
5. Faça uma relação sobre avaliação e padrões do processo do *software*.



Saiba Mais

Sobre Engenharia de *Software* (Tecnologia em camadas), leia o capitulo 2 do livro: Engenharia de *Software*, de Roger S. Prissman. Analise e discuta com seus colegas de turma os seguintes questionamentos:



Resumindo

Neste capítulo, estudamos os conceitos de engenharia de *software*, os aspectos técnicos da sua qualidade, a importância de medidas e unidades. Foram vistos os “sólidos princípios da engenharia” que podem ser aplicados ao desenvolvimento de *softwares* de computador e como construir *softwares* “economicamente” e de modo que eles sejam “confiáveis”.

O processo de engenharia de *software* é o adesivo que mantém unidas as camadas de tecnologia e permite o desenvolvimento racional e oportuno de *softwares* de computador. Vimos os modelos de processo que enfatizam a definição, identificação e aplicação detalhada de

atividades e tarefas de processo que têm sido aplicadas na comunidade de engenharia de *software* durante os últimos trinta anos.

Por fim, o capitulo exibiu os chamados níveis e metas dos processos para criação do *software*.

UNIDADE 03

PANORAMA SOBRE MODELOS PRESCRITIVOS DE PROCESSO

OBJETIVOS:

- **Definir modelos prescritivos de processo;**
- **Classificar os modelos prescritivos de processo;**
- **Determinar as atividades que serão realizadas durante o desenvolvimento do *software*;**
- **Compreender as ações do modelo prescritivo de processo nos projetos de *software*.**

3 PANORAMA SOBRE MODELOS PRESCRITIVOS DE PROCESSO

Um modelo de processo de *software* define o que deve ser realizado em cada fase do desenvolvimento e dá as instruções de como realizar essas atividades. Ele serve como um guia, um roteiro para a execução de um processo de desenvolvimento. Um modelo descritivo retrata como um processo é executado em um ambiente em particular. Já um "modelo prescritivo" retrata como um processo deveria ser executado. Sendo assim um modelo prescritivo é uma espécie de recomendação que pode ser adaptada ou melhorada pela empresa de *software* que for adotá-la.

- **O que é?** Modelos prescritivos de processo definem um conjunto distinto de atividades, ações, tarefas, marcos e produtos de trabalho que são necessários para fazer engenharia de *software* com alta qualidade. Esses modelos de processo não são perfeitos, mas efetivamente fornecem um roteiro útil para o trabalho da engenharia de *software*;

- **Quem faz?** Engenheiros de *software* e seus gerentes adaptam um modelo de processo prescritivo a suas necessidades e depois o seguem. Além disso, o pessoal que solicitou o *software* tem um papel a desenvolver, à medida que o modelo de processo é seguido;

- **Por que é importante?** Porque fornece estabilidade, controle e organização a uma atividade que pode tornar-se se bastante caótica, se deixada sem controle. Alguns têm referido a modelos prescritivos de processo como “modelos de processo rigorosos”, porque eles frequentemente incluem as capacitações sugeridas pelo CMMI. No entanto, cada

modelo de processo deve ser adaptado para que seja usado efetivamente em um projeto de *software* específico;

- **Quais são os passos?** O processo dirige uma equipe de *software* por meio de um arcabouço de atividades guarda-chuva que são organizadas em um fluxo de processo que pode ser linear, incremental ou evolutivo. A terminologia e os detalhes de cada modelo de processo diferem, mas as atividades genéricas de arcabouço permanecem razoavelmente consistentes;

- **Qual é o produto do trabalho?** Do ponto de vista de um engenheiro de *software*, os produtos de trabalho são os programas, documentos e dados que são produzidos em consequência das atividades e tarefas definidas pelo processo;

- **Como tenho certeza de que fiz corretamente?** Existem diversos mecanismos de avaliação de processo de *software* que permitem às organizações determinar a "maturidade" do seu processo de *software*. Todavia, a qualidade, pontualidade e viabilidade no longo prazo do produto que você constrói são os melhores indicadores da eficácia do processo usado.

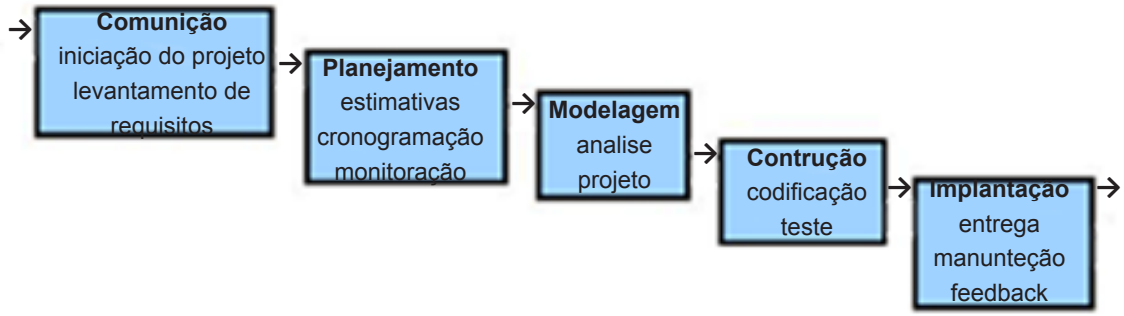
Se os modelos prescritivos de processo buscam estrutura e ordem, serão apropriados ao mundo do *software* que busca modificações? No entanto, se rejeitarmos modelos de processos convencionais (e a ordem que eles implicam) e os substituirmos por algo menos estruturado, tornam-se impossível obter coordenação e coerência no mundo de *software*?

Não há respostas fáceis para essas questões, mas existem alternativas disponíveis para os engenheiros de *software*.

O Modelo Clássico / Em Cascata

Este modelo é classificado como sendo um Modelo Prescritivo de Processo. Apesar de todos os seus problemas, o modelo em cascata é utilizado há muitos anos, além de ser o mais antigo dos modelos. Atualmente, devido à complexidade cada vez maior dos sistemas, ele é pouco utilizado.

Suas principais características:



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

- Método sistemático e sequencial;
- O resultado de uma fase se constitui na entrada de outra - cada atividade é uma fase distinta;
- Só após o seu total término é que a próxima atividade começa;
- O nível de abstração vai diminuindo à medida que se avança no processo enquanto o formalismo aumenta;
- Cada fase é estruturada como um conjunto de atividades que podem ser executadas por pessoas diferentes;
- Os requisitos do problema devem estar bem compreendidos;
- Diferentes autores podem considerar diferentes atividades a serem trabalhadas (mesmo que eles tenham as características em comum).

No entanto o que está acima descrito se aplica a qualquer conjunto de atividades consideradas.

Atividades do Ciclo de Vida Clássico:

- Análise de Engenharia de Sistemas;
- Análise;
- Projeto;
- Implementação;
- Testes;
- Manutenção.

Sobre as atividades de análise:

- Envolve a coleta dos requisitos para todos os elementos do sistema;
- Análise em alto nível. Pouco projeto;
- Visão global do sistema: tarefas, interface com o usuário, interface com *hardware*, interface com banco de dados, etc.

A análise envolve diversas tarefas:

- Identificar as necessidades dos usuários;
- Realizar a análise dos custos envolvidos;
- Realizar a análise dos recursos necessários tanto de ferramentas quanto de pessoal;
- Estabelecer restrições de prazo e custo;
- Realizar um projeto inicial e global do sistema, incluindo sua divisão em módulos;

Como realizar a análise:

- Entrevistas entre o analista e o cliente;
- Questionários;
- O analista deve saber fazer as perguntas certas, orientar, esclarecer e dar conselhos;
- O cliente deve ser capaz de esclarecer suas expectativas e metas de projeto;
- Técnicas: análise estruturada, análise orientada a objeto, métodos formais;

Resultado: Especificação dos Requisitos. E pode acrescentar também o plano de Projeto.

Sobre a atividade de projeto de sistema:

- Refinar a especificação global do sistema gerada na fase de análise;
- O objetivo é realizar uma especificação mais detalhada do sistema, de forma que seja possível avaliar a qualidade prevista, antes de iniciar a codificação;
- Definições realizadas nesta fase;
- Estrutura de dados que será utilizada;
- Arquitetura do *Software*;
- Detalhes Procedimentais;
- Interface entre módulos e com o usuário.

Sobre a atividade de Codificação / Implementação:

- Consiste em implementar o que foi definido no projeto;
- Quando o projeto é bem feito e os desenvolvedores são experientes e/ou competentes, esta etapa é simplificada;
- Em outras palavras, esta etapa consiste da tradução do projeto para uma linguagem artificial, resultando em instruções

executáveis pelo computador.

Sobre a atividade de Teste de Sistema:

- Assim que o código for gerado, inicia-se a realização de testes do programa. Este processo concentra-se nos: aspectos lógicos interno do *software*, garantindo que todas as instruções tenham sido testadas; aspectos funcionais externos, realizando testes para descobrir erros e garantir que a entrada definida produza resultados reais.

Sobre a atividade de manutenção:

- Se eventualmente o *software* sofrer mudanças depois que for entregue ao cliente, a manutenção de *software* reaplica todas as etapas do ciclo de vida;

Tipos de Manutenção:

- Manutenção corretiva: consiste em diagnosticar e corrigir erros;
- Manutenção adaptativa: adaptar o software devido a mudanças no *hardware* ou *software* (por exemplo, novo sistema operacional);
- Manutenção perfectiva: melhorar o desempenho do *software* ou adicionar funcionalidades;
- Manutenção preventiva: melhorar a confiabilidade ou garantir possibilidade de manutenção futura.

Problemas do Ciclo de Vida Clássico

- Os projetos reais raramente seguem o fluxo sequencial que o modelo propõe. Alguma interação sempre ocorre e traz problemas na aplicação do paradigma;
- Muitas vezes é difícil para o cliente declarar todas as exigências explicitamente. O ciclo clássico tem dificuldade de acomodar a incerteza natural que existe no começo de muitos projetos;
- Uma versão de trabalho do(s) programa(s) só estará disponível em um ponto muito tardio do cronograma do projeto. O cliente deve ter paciência;
- No modelo em cascata, os subprocessos são executados em restrita sequência, o que permite demarcá-los com pontos de controle bem definidos. Essa característica facilita a gestão

dos projetos. Por outro lado, caso interpretado literalmente, é um processo rígido e burocrático, em que as atividades devem ser muito bem dominadas, pois não são permitidos erros (retornar após uma etapa ter sido concluída);

- "[...] A natureza linear do modelo em cascata leva a 'estados de bloqueio', nos quais alguns membros da equipe de projeto precisam esperar que outros membros completem as tarefas dependentes" (PRESSMAN, 2010).

Realimentação entre fases

Na prática, é sempre necessário permitir que, em fases posteriores, haja revisão e alteração de resultados das fases anteriores. Por exemplo, os modelos e documentos de projeto podem ser alterados durante a implementação, à medida que os problemas vão sendo descobertos. Uma variante que permite superposição entre fases e a realimentação de correções é um modelo mais realista, com realimentação entre as fases. Esta realimentação dificulta gerenciar projetos baseados neste modelo de processo.

Incremental

Este modelo é classificado como sendo um Modelo Prescritivo de Processo.

O modelo incremental e interativo foi proposto como uma resposta aos problemas encontrados no modelo em cascata.

Um processo de desenvolvimento segundo essa abordagem divide o desenvolvimento de um produto de *software* em ciclos, sendo desenvolvido em partes (incrementos).

Em cada ciclo de desenvolvimento, podem ser identificadas todas as fases do projeto.

Essa característica contrasta com a abordagem clássica, na qual as fases do projeto são realizadas uma única vez.

Cada um dos ciclos considera um subconjunto de requisitos. Os requisitos são desenvolvidos uma vez que sejam alocados a um ciclo de desenvolvimento.

No próximo ciclo, outro subconjunto dos requisitos é considerado para ser desenvolvido, o que produz um novo incremento do sistema que contém extensões e refinamentos sobre o incremento anterior.

Assim, o desenvolvimento evolui em versões, através da

construção incremental e interativa de novas funcionalidades até que o sistema completo esteja construído.

Vale destacar que apenas uma parte dos requisitos é considerada em cada ciclo de desenvolvimento. Na verdade, um modelo interativo e incremental pode ser visto como uma generalização da abordagem em cascata: o *software* é desenvolvido em incrementos, o qual é desenvolvido em cascata.

A abordagem incremental e interativa somente é possível se existir um mecanismo para dividir os requisitos do sistema em partes, para que cada parte seja alocada a um ciclo de desenvolvimento.

Essa alocação é realizada em função do grau de importância atribuído cada requisito.

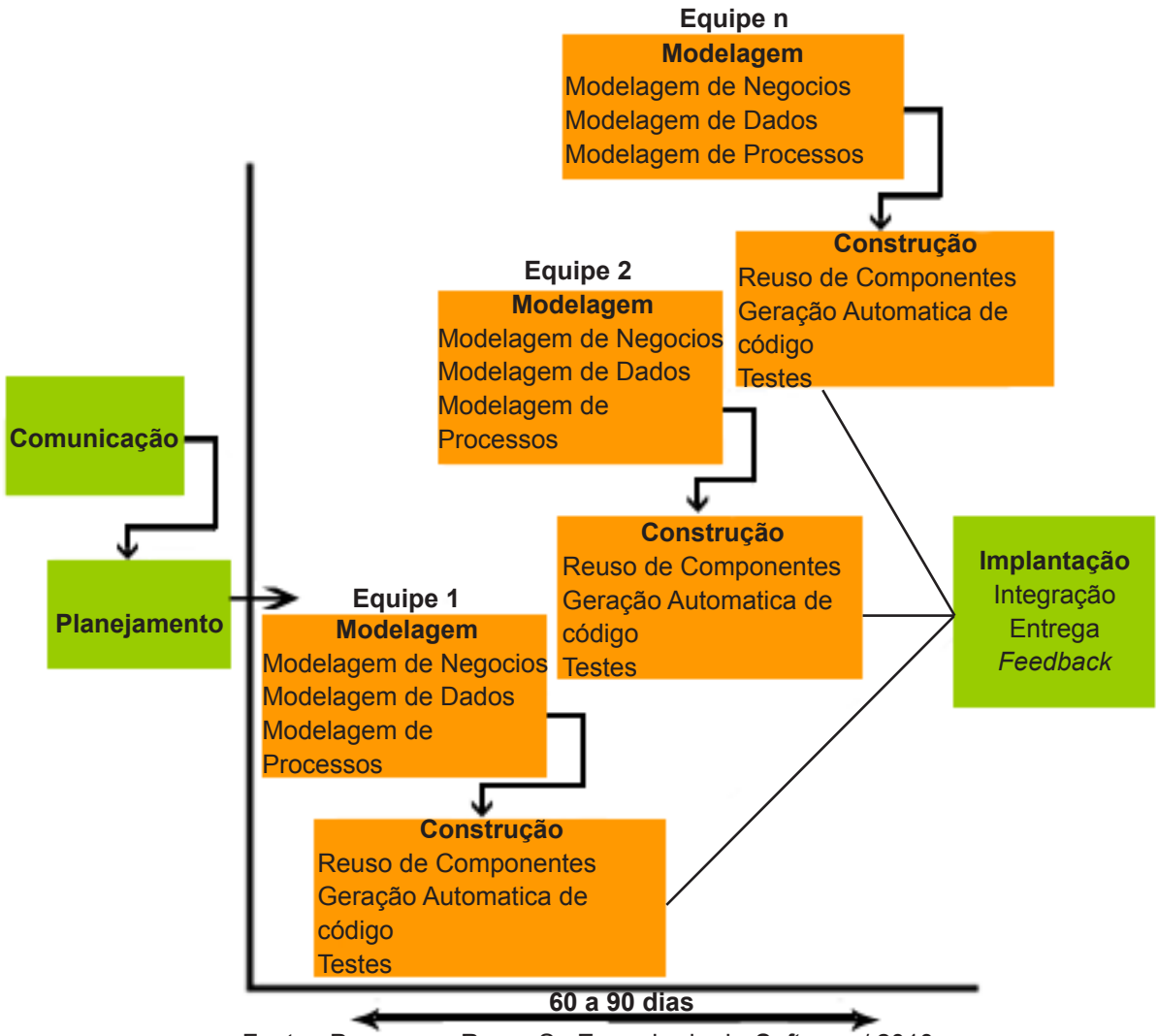
RAD (*Rapid Application Development*)

Este modelo é classificado como sendo um Modelo Prescritivo de Processo e incremental.

O desenvolvimento rápido de aplicação é um modelo de processo de desenvolvimento de *software* incremental que enfatiza um ciclo de desenvolvimento extremamente curto (por exemplo, de 60 a 90 dias).

É uma adaptação do modelo cascata, no qual o desenvolvimento rápido é conseguido com o uso de uma abordagem de construção baseada em componentes, requerendo que os requisitos sejam bem compreendidos, os objetivos propostos e haja diferentes equipes de desenvolvimento envolvidas:

- Tamanho das equipes envolvidas, a medida que o escopo do problema aumenta;
- Necessidade de maior envolvimento e comprometimento, para cumprir os prazos curtos;
- O problema precisa ser adequadamente modularizado para que haja atuação das diferentes equipes;
- Em projetos de alto desempenho com interfaces de módulos muito interligadas há maior probabilidade de erros;
- Pode não ser a abordagem mais adequada quando os riscos técnicos são altos, como por exemplo, na utilização de uma nova tecnologia.



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Prototipagem

Este modelo é classificado como sendo um Modelo Prescritivo de Processo e evolucionário.

Sobre modelos de processo evolucionários, Pressman (2006) destaca o seguinte: O *software*, como todo sistema complexo, evolui com o passar do tempo.

Os requisitos do negócio e do produto mudam frequentemente à medida que o desenvolvimento prossegue, dificultando um caminho direto para um produto final; prazos reduzidos de mercado tornam impossível completar um produto de *software* abrangente, mas uma versão reduzida

pode ser elaborada para fazer frente à competitividade ou às pressões do negócio; um conjunto de requisitos básicos de um produto ou sistema é bem entendido, mas os detalhes das extensões do produto ou sistema ainda precisam ser definidos.

Nesse caso, e em situações semelhantes, os engenheiros de *software* precisam de um modelo de processo que tenha sido explicitamente projetado para acomodar um produto que evolui com o tempo.

O mesmo autor ainda prossegue, destacando a necessidade dos modelos evolucionários: "[...] o *software* de computador moderno é caracterizado por modificações contínuas, prazos muito curtos e por uma enfática necessidade de satisfação do cliente/usuário.

Em muitos casos, o período até a colocação no mercado é o requisito gerencial mais importante. Se um nicho de mercado é perdido, o projeto de *software* em si pode ser insignificante."

- Dificulta o planejamento de projeto, pois há um número incerto de ciclos para construir o produto;
- A velocidade da evolução pode acarretar problemas no próprio produto;
- São processos focados na flexibilidade e extensibilidade, em detrimento da alta qualidade.

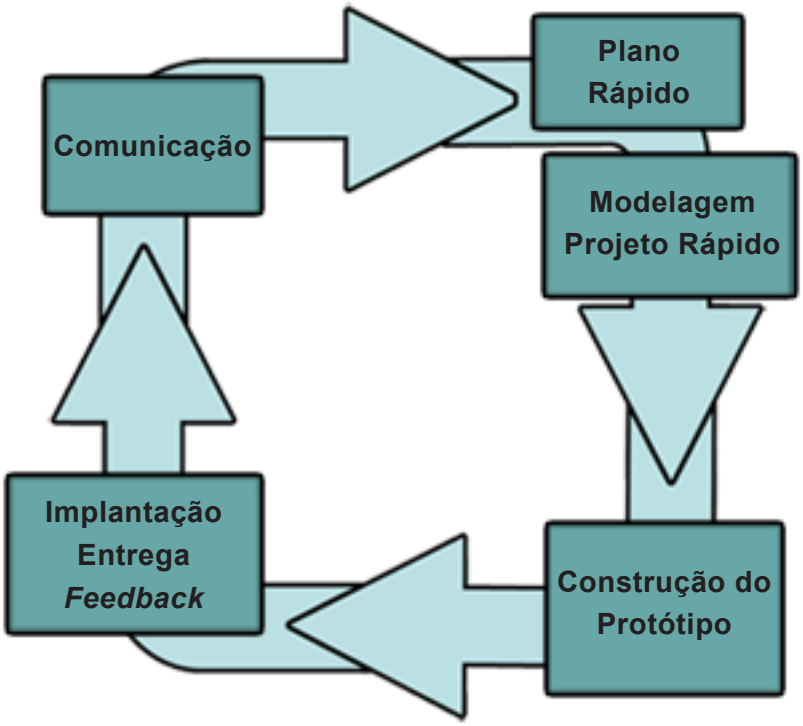
A prototipagem pode também ser considerada como um modelo de processo de desenvolvimento de *software*, mas que pode funcionar concomitantemente com qualquer outro tipo de modelo.

No desenvolvimento de um *software*, a prototipagem é um esboço de uma parte do sistema.Protótipos são construídos para telas de entrada, de saída, para subsistemas ou mesmo para todo um sistema.

A construção de protótipos costuma utilizar as denominadas linguagens de programação visual. Exemplos são o *Dclphi*, o *PowerBuilder* e o *Visual Basic* que, na verdade, são ambientes com facilidades para a construção da interface gráfica (telas, formulários, etc.).

Além disso, muitos sistemas de gerência de bancos de dados também fornecem ferramentas para a construção de telas de entrada e saída de dados.

O modelo de Prototipagem



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Esse modelo constrói uma versão descartável (ou protótipo). Esse protótipo visa a testar conceitos e requisitos e será usado para demonstrar o comportamento aos clientes.

Após a concordância dos clientes, o *software* é desenvolvido seguindo as mesmas fases do modelo linear sequencial.

O esforço despendido no protótipo geralmente é compensado pelo não desenvolvimento de características desnecessárias.

Pode assumir 3 formas:

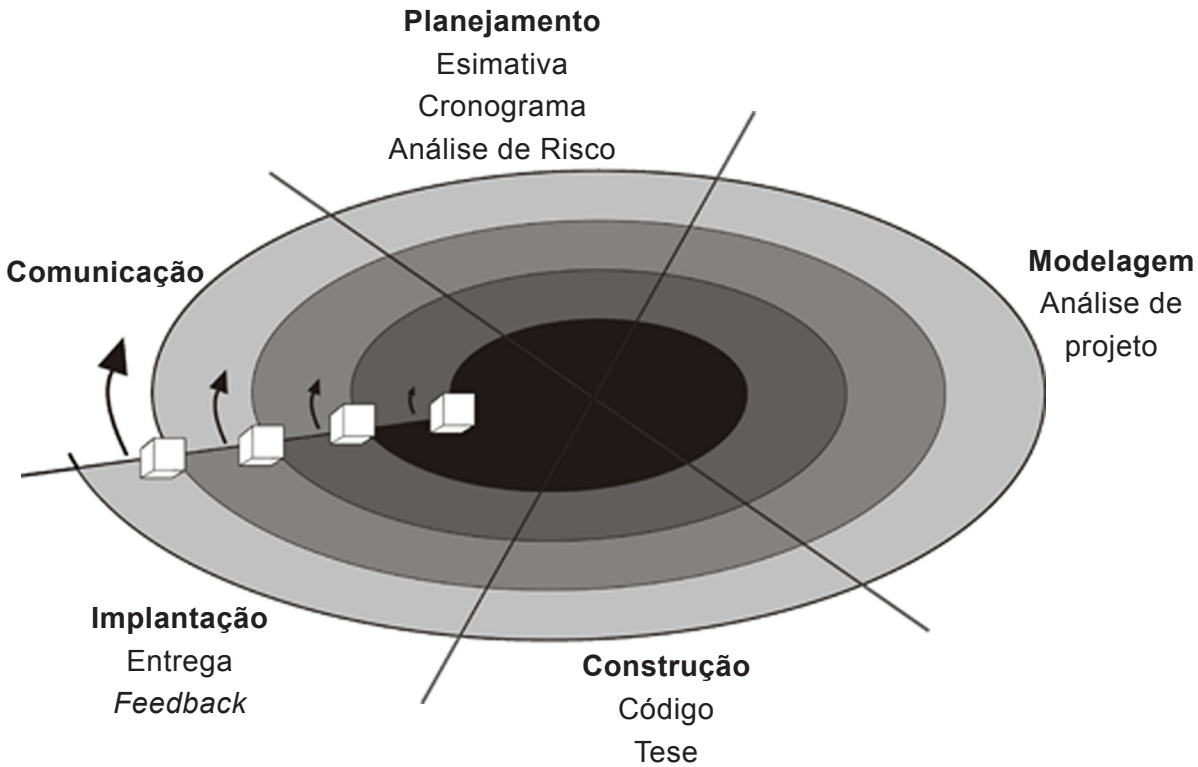
- Um protótipo em papel ou modelo baseado em PC;
- Um protótipo de trabalho que implementa algum subconjunto da função exigida do *software* desejado;
- Um programa existente que executa parte ou toda a função desejada;
- O funcionamento básico é o seguinte: A prototipação inicia com a coleta dos requisitos. O desenvolvedor e o cliente reúnem-

se e definem os objetivos globais para o *software*. Ocorre então a elaboração de um "projeto rápido". O projeto rápido leva a construção de um protótipo que é avaliado pelo cliente/usuário e é usado para refinar os requisitos para o *software* a ser desenvolvido;

- Idealmente, o protótipo serve como um mecanismo para identificar os requisitos de *software*, quando há muitas dúvidas entre estes. A prototipação é fundamental à medida que cresce a importância da interface com o usuário;
- O protótipo pode servir como "o primeiro sistema" – sistema esse que se recomenda que joguemos fora. O problema é que alguns usuários tendem a encarar o protótipo como algo já confiável, ou seja, como se já fosse parte do *software*;
- Na maioria dos projetos, o primeiro sistema construído dificilmente é usável. Ele pode ser muito lento, muito grande, desajeitado em uso, ou todos os três. Não resta alternativa senão começar tudo de novo, mas com mais habilidade. O desenvolvedor pode não utilizar as técnicas de programação adequadas, pensando apenas na rapidez. Um protótipo normalmente preocupa-se apenas com a interface, não tratando dos algoritmos e regras de negócio. Esse é, portanto mais um motivo para que ele seja descartado, ao invés de servir de base para o *software* final;
- Ainda que possam ocorrer problemas, a prototipação é um paradigma eficiente da engenharia de *software*. A chave é definir as regras do jogo logo no começo, ou seja, o cliente e o desenvolvedor devem ambos concordar que o protótipo seja construído para servir um mecanismo a fim de definir os requisitos. Ele será depois descartado (pelo menos em parte) e o *software* real será projetado.

Espiral ou Evolutivo

Este modelo é classificado como sendo um Modelo Prescritivo de Processo, e evolucionário.



Este é um modelo que atende os seguintes casos:

- Problema a ser resolvido não está totalmente entendido;
- A realidade pode mudar enquanto o sistema está sendo desenvolvido;
- A própria solução adotada pode ter algum efeito colateral desconhecido;
- A preocupação está centrada mais na qualidade e funcionalidade do que se produz.

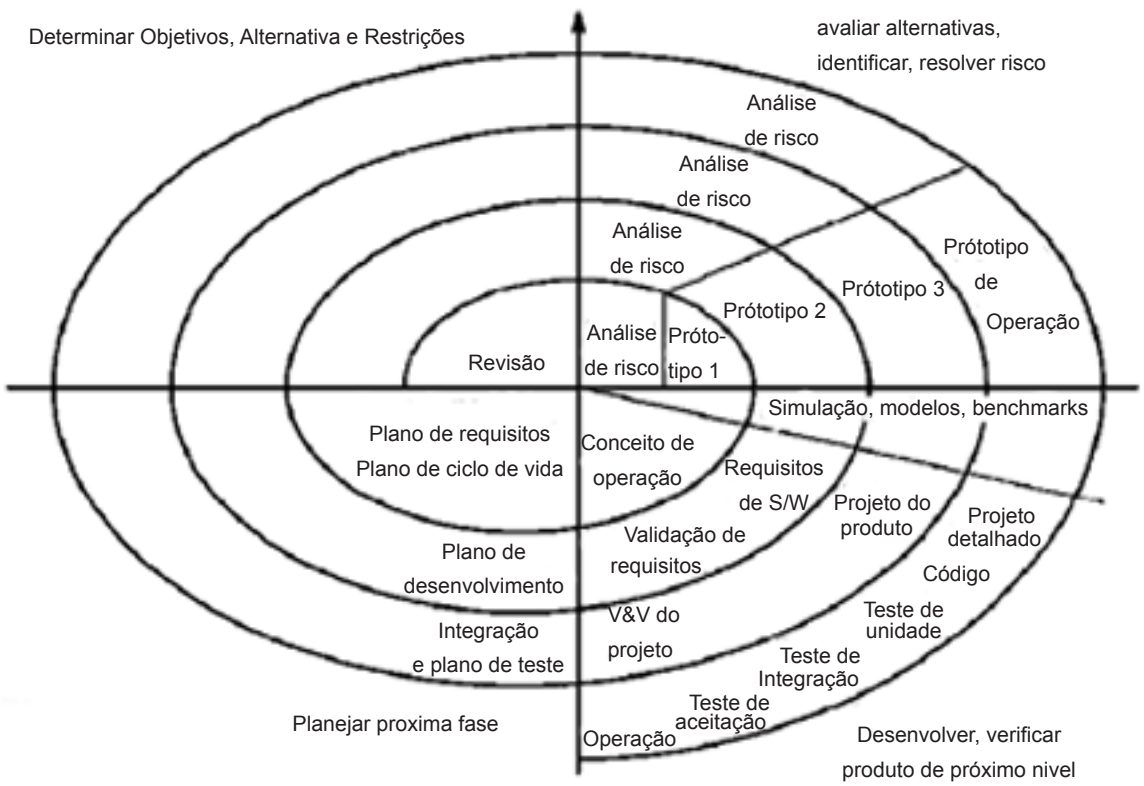
O modelo espiral para a engenharia de *software* foi desenvolvido para abranger as melhores características tanto do ciclo de vida clássico (no que se refere à sistematização e controle) como da prototização (no que se refere à interatividade), acrescentando, ao mesmo tempo, um novo elemento – a análise dos riscos – que falta a esses paradigmas. O modelo, representado pela espiral, define quatro importantes atividades representadas pelos quatro quadrantes da figura:

- Planejamento: determinação dos objetivos, alternativas e restrições;

- Análise dos riscos: análise de alternativas e identificação/ resolução dos riscos;
- Engenharia: desenvolvimento do produto no “nível seguinte”;
- Avaliação feita pelo cliente: avaliação dos resultados da engenharia;
- No modelo em Espiral o produto é desenvolvido em uma série de interações. Cada nova iteração corresponde a uma volta na espiral. Isso permite construir produtos em prazos curtos com novas características e recursos que são agregados à medida que a experiência descobre sua necessidade. As atividades de manutenção são usadas para identificar problemas; seus registros fornecem dados para definir os requisitos das próximas liberações;
- As versões progressivamente mais completas dos *softwares* são construídas ao redor de cada interação da espiral;
- Durante o início do giro, os objetivos, alternativas e restrições são definidas e os riscos são identificados e analisados. A conclusão da análise dos riscos resulta numa decisão de “prosseguir ou não”. Agora, se os riscos forem muito grandes, o projeto pode ser encerrado.
- Há o desenvolvimento de parte do programa que é disponibilizado para testes. Então, o cliente avalia o trabalho de engenharia e apresenta sugestões para modificações;
- Cada passagem pela região de planejamento resulta em ajustes no plano de projeto. O custo e o cronograma são ajustados com base no *feedback* derivado do cliente após a entrega.
- Deve-se notar que o número de atividades de desenvolvimento que ocorre no quadrante inferior direito eleva-se à medida que as atividades se afastam do centro da espiral. O paradigma do modelo espiral usa a prototização como um mecanismo de redução dos riscos, possibilitando que o desenvolvedor aplique a abordagem de protótipo em qualquer etapa da evolução do produto;
- O modelo espiral exige uma consideração direta dos riscos técnicos em todas as etapas do projeto e, se adequadamente aplicado, deve reduzir os riscos antes que eles se tornem problemáticos. Ela exige considerável experiência na avaliação dos riscos e fia-se nessa experiência para o sucesso. Se um

grande risco não for descoberto, indubitavelmente ocorrerão problemas;

- O principal problema do ciclo de vida em espiral é que ele requer gestão muito sofisticada para ser previsível e confiável. E pode ser difícil convencer os clientes sobre a realização de controle em tal forma de trabalho, visto os constantes ajustes que são realizados;
- Vale destacar que o modelo em espiral pode ser continuamente utilizado, após a conclusão inicial do *software*, para a realização das manutenções que o mesmo deverá sofrer ao longo do tempo. Outra figura interessante é apresentada por Sommerville (2006) e está abaixo destacada.



Modelo em espiral do processo de *software* Boehm. Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Concorrente

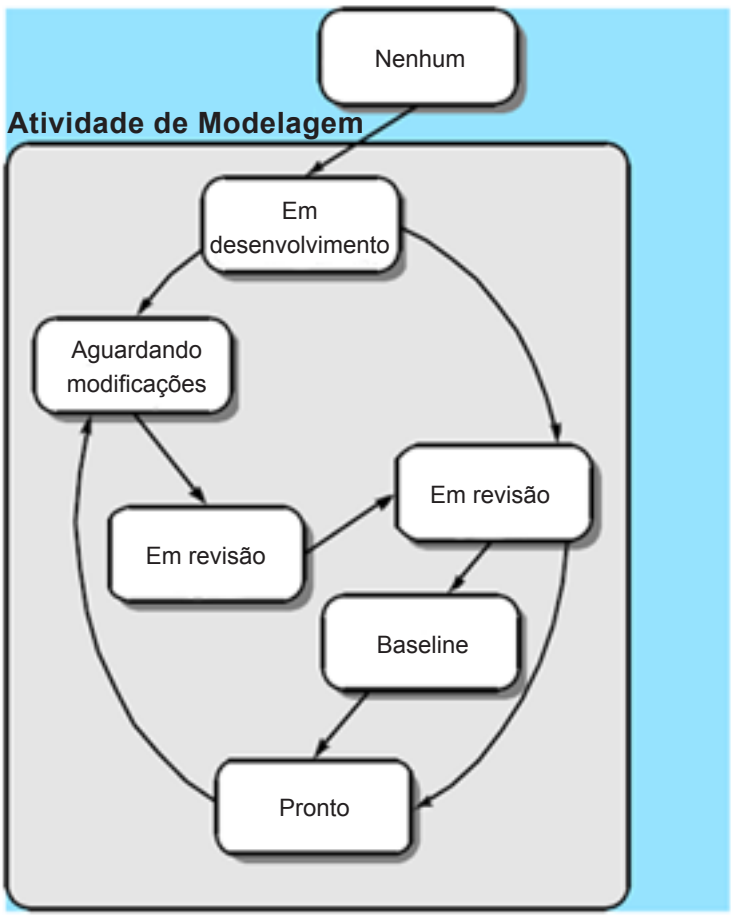
Este modelo é classificado como sendo um Modelo Prescritivo de Processo e evolucionário.

O modelo concorrente é representado através de uma série de atividades e seus estados correspondentes.

Sobre ele pode ser dito o seguinte:

- As atividades podem ocorrer em paralelo, mas estão em diferentes estados;
- O modelo define uma série de eventos que vão disparar transições de estado para estado, para cada uma das atividades;
- Em vez de usar uma sequência como o modelo cascata, ele define uma rede de atividades;
- Pode ser aplicado a todo tipo de desenvolvimento de *software* e fornece uma visão exata de como está o estado do projeto.

Um exemplo de uma rede de atividades pode ser como o da figura abaixo:



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Desenvolvimento baseado em componentes

É um modelo desenvolvido para apoiar a reusabilidade necessária ao atual desenvolvimento de *software* por suas principais consequências: redução de prazo e custo.

- Utiliza componentes de *software* comercial de prateleira, isto é, componentes que apresentam funcionalidade e interface bem definidas e que podem ser utilizados integrados no *software* em desenvolvimento;
- Incorporam muitas das características do modelo espiral. Evolucionário por natureza demanda uma abordagem interativa.

Apresenta os seguintes passos:

- Pesquisa e avaliação de componentes disponíveis no mercado;
- Análise das necessidades de integração;
- Projeto da arquitetura de *software*, considerando os componentes;
- Validação rigorosa.

O modelo de métodos formais

Os modelos de métodos formais abrangem um conjunto de atividades que levam à especificação matemática formal do *software* de computador.

São modelos que se apoiam na aplicação de uma rigorosa notação matemática.

Tem ganhado adeptos quando há necessidade do desenvolvimento de *softwares* críticos em termos de segurança, devido à sua promessa de um produto livre de erros.

- **Vantagens:** Ambiguidade, inconclusão e inconsistência podem ser descobertas e corrigidas mais facilmente por meio de análise matemática;
- **Problemas:** O desenvolvimento de modelos formais é atualmente muito lento e dispendioso.

Como poucos desenvolvedores de *software* têm o preparo necessário para aplicar métodos formais, torna-se necessário um

treinamento extensivo; É difícil usar os modelos como um mecanismo de comunicação, com clientes despreparados tecnicamente.

Desenvolvimento de *Software* Orientado a Aspectos

Certas preocupações transversais cobrem toda a arquitetura de um sistema, não sendo exclusivas de uma funcionalidade, característica ou informação. Por exemplo: segurança, tolerância a falhas, transações, geração de *logs*. Assim, pode-se definir:

Aspectos: mecanismos que transcendem subrotinas e herança para localizar a expressão de uma preocupação transversal, afetando toda a arquitetura de *software*.

Não existe ainda processo orientado a aspectos, mas é provável que se adote características tanto do modelo de processo espiral quanto do concorrente.

Alguns termos que começam a se tornar conhecidos:

- POA – Programação Orientada a Aspectos;
- DSOA – Desenvolvimento de *Software* Orientado a Aspectos.

Processo Unificado (PU)

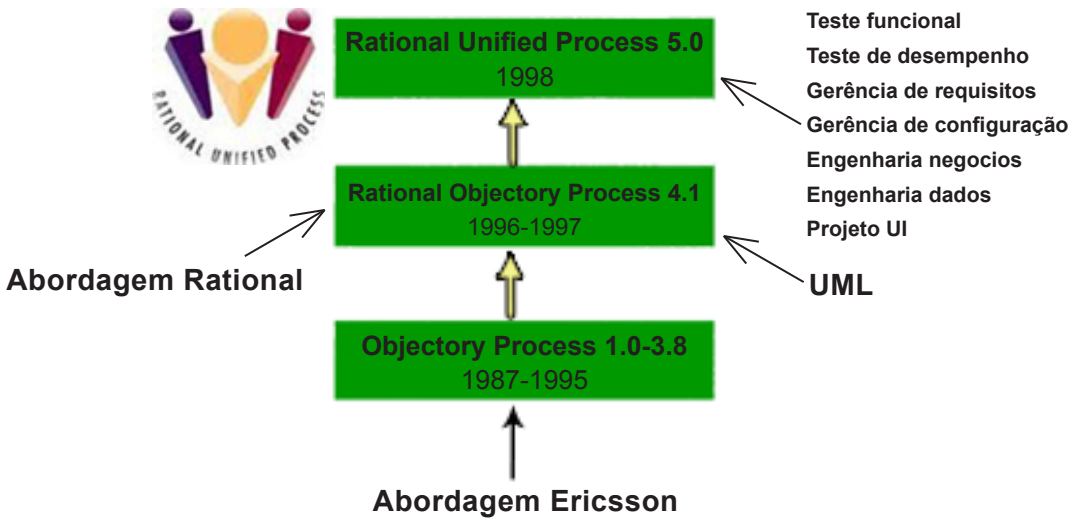
O PU é uma tentativa de se apoiar nos melhores recursos e características dos modelos convencionais de processo de *software*, mas caracterizá-los de um modo que programem muitos dos melhores princípios de desenvolvimento ágil de *softwares*.

O Processo Unificado é um processo de engenharia de *software* desenvolvido por três dos principais gurus da indústria de *software*: Ivar Jacobson, James Rumbaugh e Grady Booch, sendo o resultado de mais de 30 anos de experiência acumulada.

É o primeiro processo de desenvolvimento a explorar integralmente as capacidades do padrão UML e se baseia nas práticas comuns aos projetos de *software* com mais alto ROI do mercado.

Processo de *Software* Unificado (*Rational Unified Process*) = Processo + Métodos + Linguagem (UML)

Histórico:



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

O desenvolvimento de sistemas seguindo o RUP é um processo:

- Dirigido por casos de uso (use *cases*);
- Centrado na arquitetura;
- Iterativo e incremental.

Processo dirigido por casos de uso (use *cases*):

Caso de uso é um modelo que define o que o sistema deve fazer da perspectiva dos usuários, subsistemas ou periféricos;

Ator é algo que interaja com o sistema a ser desenvolvido;

Todos os casos de uso de um sistema compõem a especificação funcional do sistema (modelo de casos de uso), ou seja, definem os requisitos do sistema;

Use cases associam todos os *workflows* de forma conjunta;

Dirigem várias atividades de desenvolvimento: Criação e validação da arquitetura do sistema, Criação de casos de teste, Planejamento das interações, Criação de documentação do usuário, Implantação do sistema;

Sincronizam conteúdo dos modelos criados em cada *workflow*;

Processo centrado na arquitetura:

Fornece uma base sólida para a construção do *software*;

Melhor compreensão do sistema e organização do desenvolvimento;

Descrição da arquitetura envolve elementos mais importantes,

como a coleção de visões dos modelos do sistema;

UP prescreve o refinamento sucessivo da arquitetura;

A arquitetura representa a forma, enquanto que os use cases representam funcionalidade;

Arquitetura e *use cases* devem ser balanceados.

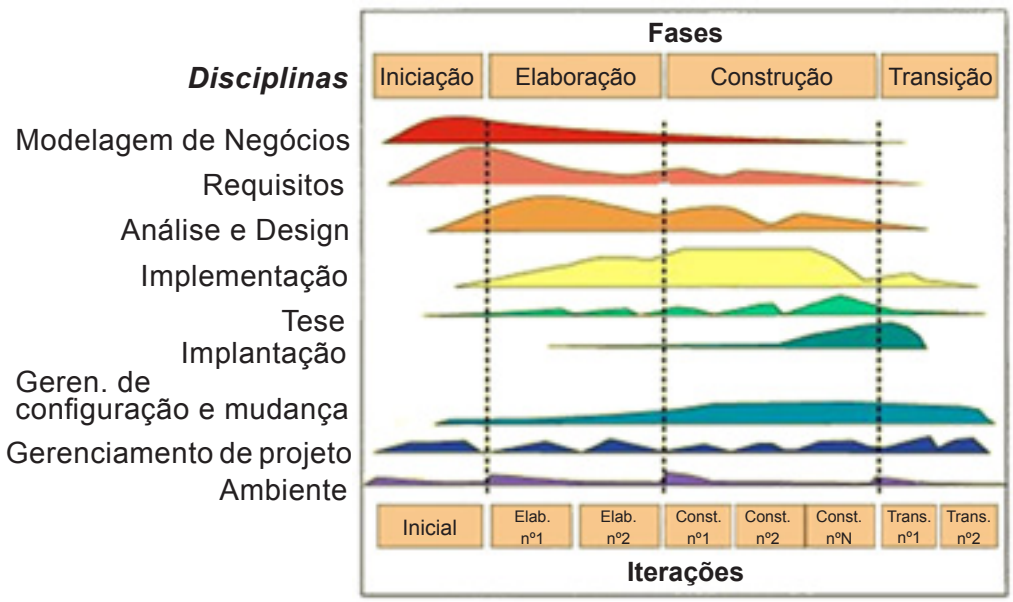
Identificação de riscos é adiantada;

Preparação do Sistema para requisitos que mudam;

Integração contínua (facilita testes) e aprendizado facilitado;

Desenvolvimento em mini-projetos (interações) que incrementam o desenvolvimento;

Modelos evoluem nas interações.



Fonte : Pressman, Roger S. Engenharia de Software / 2010.

Fases do Rup

O RUP define um conjunto de atividades:

- Bem definidas;
- Com responsáveis;
- Com artefatos de entrada e saída;
- Com dependências e ordem de execução;
- Com modelo de ciclo de vida;
- Com uma descrição sistemática de como executá-las;
- Usando linguagem UML.

Conceitos-chave do RUP:

O Processo Unificado proposto pela Rational Unified Process – RUP foi criado para apoiar o desenvolvimento orientado a objetos, fornecendo uma forma sistemática para se obter reais vantagens no uso da Linguagem de Modelagem Unificada (*Unified Modeling Language* – UML). De fato, ele não é exatamente um processo: é uma infraestrutura genérica de processo que pode ser especializada para uma ampla classe de sistemas de *software* para diferentes áreas de aplicação, tipos de organização, níveis de competência e tamanhos de projetos.

O RUP está fundamentado em três princípios básicos: orientação a casos de uso, arquitetura e interação. Ele é dito dirigido a casos de uso, pois são os casos de uso que orientam todo o processo de desenvolvimento. Com base no modelo de casos de uso são criados uma série de modelos de análise, projeto e implementação que realizam estes casos de uso. É centrado em arquitetura, pois defende a definição de um esqueleto para a aplicação (a arquitetura), a ganhar corpo gradualmente ao longo do desenvolvimento. Finalmente, o RUP é interativo e incremental, oferecendo uma abordagem para particionar o trabalho em porções menores ou miniprojetos. Esses três conceitos são igualmente importantes. A arquitetura provê a estrutura para guiar o desenvolvimento do sistema em interações, enquanto os casos de uso definem as metas e conduzem o trabalho de cada interação.

O ciclo de vida adotado no RUP é tipicamente evolutivo. Contudo, uma forma de organização em fases é adotada para comportar os ciclos de desenvolvimento, permitindo uma gerência mais efetiva de projetos complexos.

Ao contrário do tradicionalmente definido como fases na maioria dos modelos de ciclo de vida – planejamento, levantamento de requisitos, análise, projeto, implementação e testes são definidas fases ortogonais a estas, a saber:

- Concepção: nesta fase, é estabelecido o escopo do projeto e suas fronteiras, determinando os principais casos de uso do sistema. Esses casos de uso devem ser elaborados com a precisão necessária para se proceder as estimativas de prazos e custos. As estimativas devem ser globais para o projeto como um todo e detalhadas para a fase seguinte. Assim, a ênfase nesta etapa recai sobre o planejamento e, por conseguinte, é necessário levantar requisitos do sistema e preliminarmente analisá-los. Ao término dessa fase, são

examinados os objetivos do projeto para se decidir sobre a continuidade do desenvolvimento.

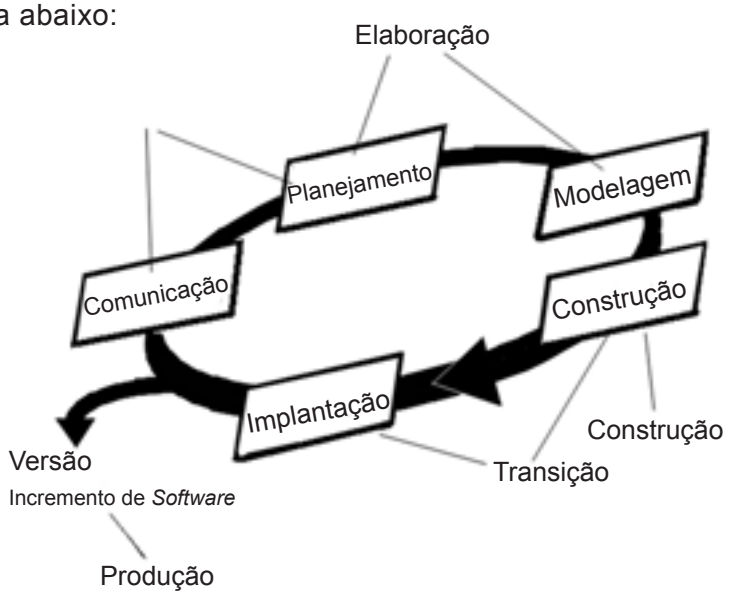
- Elaboração: o propósito desta fase é analisar mais refinadamente o domínio do problema, estabelecer uma arquitetura de fundação sólida, desenvolver um plano de projeto para o sistema a ser construído e eliminar os elementos de projeto que oferecem maior risco. Embora o processo deva sempre acomodar alterações, as atividades da fase de elaboração asseguram que os requisitos, a arquitetura e os planos estão suficientemente estáveis e que os riscos estão suficientemente mitigados, de modo a prever com precisão os custos e prazos para a conclusão do desenvolvimento.
- Construção: durante esta fase um produto completo é desenvolvido de maneira interativa e incremental, para que esteja pronto para a transição à comunidade usuária.
- Transição: nesta fase, o *software* é disponibilizado à comunidade usuária. Após o produto ter sido colocado em uso, naturalmente surgem novas considerações que vão demandar a construção de novas versões para permitir ajustes do sistema, corrigir problemas ou concluir algumas características que foram postergadas. É importante realçar que dentro de cada fase, um conjunto de interações, envolvendo planejamento, levantamento de requisitos, análise, projeto e implementação e testes é realizado. Contudo, de uma interação para outra e de uma fase para a próxima, a ênfase sobre as várias atividades mudam, como mostra a figura 1, em que a cor preta indica grande ênfase, enquanto a cor branca indica pouca ênfase. Na fase de concepção, o foco principal recai sobre o entendimento dos requisitos e a determinação do escopo do projeto (planejamento e levantamento de requisitos). Na fase de elaboração, o enfoque está na captura e modelagem dos requisitos (levantamento de requisitos e análise), ainda que o trabalho de projeto e implementação seja realizado para ser o protótipo da arquitetura, evitando certos riscos técnicos. Na fase de construção, o enfoque concentra-se no projeto e na implementação, visando evoluir e rechear o protótipo inicial, até obter o primeiro produto operacional. Finalmente, a fase de transição concentra-se nos testes, visando garantir que o sistema possui o nível adequado de qualidade.

Além disso, usuários devem ser treinados, características ajustadas e elementos esquecidos adicionados.

Além dos conjuntos de interações em cada fase, as fases em si podem ocorrer de forma interativa.

Como mostra a figura 2, elas não necessariamente têm a mesma duração. O esforço realizado em cada uma varia fortemente em função das circunstâncias específicas do projeto.

As fases propostas no PU podem ser encaradas da mesma forma que as atividades de arcabouço por ele propostas, conforme destacado na figura abaixo:



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Processo de Engenharia de *Software* na visão do RUP



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Um processo é um conjunto de passos ordenados com a intenção de atingir uma meta. Em engenharia de *software*, a meta é criar um

software ou aperfeiçoar um existente; em engenharia de processos, a meta é desenvolver ou aperfeiçoar um processo. No RUP, eles são organizados em um conjunto de disciplinas para posteriormente definirem os fluxos de trabalho e outros elementos do processo.

Em termos de modelagem de negócios, o processo de desenvolvimento de *software* é um processo de negócios; o *Rational Unified Process* (RUP) é um processo de negócios genéricos para engenharia de software orientada a objetos. Ele descreve uma família de processos de engenharia de *software* relacionados que compartilha uma estrutura comum, uma arquitetura de processos comum. Ele proporciona uma abordagem disciplinada para a atribuição de tarefas e de responsabilidades dentro de uma organização de desenvolvimento. Sua meta é garantir a produção de *software* de alta qualidade que atenda às necessidades dos usuários, dentro de uma programação e uns orçamentos previsíveis. O RUP captura muito das melhores práticas do desenvolvimento de *software* moderno, de forma que possam ser adaptadas para uma grande variedade de projetos e de organizações.

Quando um sistema de *software* é desenvolvido começando do zero, o desenvolvimento é o processo de criação de um sistema a partir dos requisitos.

Porém, depois que os sistemas tiverem tomado forma (ou seja, tiverem passado pelo ciclo de desenvolvimento inicial), os desenvolvimentos subsequentes serão o processo de adaptação do sistema aos requisitos novos ou modificados. Isso se aplica durante todo o ciclo de vida do sistema.

Nem toda tarefa identifica para um fluxo de trabalho do PU é conduzida para qualquer projeto de *software*. A equipe adapta o processo (ações, tarefas, subtarefas e produtos de trabalho) para satisfazer às suas necessidades.

Exercícios Propostos



1. Pesquise sobre modelo prescritivo de processo.
2. Qual a importância do ciclo de vida e a realimentação?
3. Conceitue Prototipagem.
4. Faça uma relação sobre algumas preocupações no

Saiba Mais

desenvolvimento de *softwares* orientados a objeto.

PRESSMAN, Roger. Engenharia de *software. McGraw Hill*. Capítulo 3 – Modelos Prescritivos de Processo; Capítulo 4 – Desenvolvimento Ágil, 2006

SOMMERVILLE, Ian. Engenharia de *software*. *Pearson Education*. Capítulo 4 – Processo de *Software*, 2006.

Para Pensar

Discuta com seus colegas sobre a importância das atividades representadas pelos quatro quadrantes do modelo espiral.

Resumindo

Neste capítulo, estudamos o conceito de modelo prescritivo de processo, sua estrutura no desenvolvimento de *software*, os aspectos técnicos e a classificação dos modelos. Foi visto a importância do ciclo de vida e a realimentação entre as fases.

A prototipagem é um modelo de processo de desenvolvimento de *software* que pode funcionar concomitantemente com qualquer outro tipo de modelo. O modelo desenvolvimento baseado em componentes desenvolvido para apoiar a reusabilidade, necessária ao atual desenvolvimento de *software*, por suas principais consequências: redução de prazo e custo.

Foram estudadas as preocupações no desenvolvimento de *software* orientado a objeto. Vimos os modelos de processo no ciclo de vida do desenvolvimento do *software* que enfatizam a definição, identificação e aplicação detalhada de atividades e tarefas.

Por fim, o capítulo exibiu o ciclo de vida adotado no RUP.

UNIDADE 04

QUALIDADE DE *SOFTWARE*

OBJETIVOS:

- **Definir Qualidade de *software*;**
- **Implantar a função qualidade;**
- **Analisar Processo e Qualidade de Projeto;**
- **Compreender as ações do Impacto na Qualidade, Produtividade e Custo.**

4 QUALIDADE DE *SOFTWARE*

O que é qualidade? Existem diversas definições. Algumas pessoas que tentaram uma definição simples chegaram a frases como:

- Qualidade é estar em conformidade com os requisitos dos clientes;
- Qualidade é antecipar e satisfazer os desejos dos clientes;
- Qualidade é escrever tudo o que se deve fazer e fazer tudo o que foi escrito.

Segundo a NBR ISSO 9000:2005, "qualidade é o grau no qual um conjunto de características inerentes satisfaz aos requisitos". Ou seja, pode-se afirmar que se algum produto ou serviço atende aos requisitos especificados, este mesmo produto ou serviço possui a qualidade desejada.

Qualidade significa que o produto a ser desenvolvido deve atender a suas especificações (conceito da manufatura – CROSBY, 1979).

A qualidade pode ser medida através do grau de satisfação em que as pessoas avaliam determinado produto ou serviço. No entanto, esse produto ou serviço pode ter qualidade para algumas pessoas e para outras nem tanto, ou seja, a qualidade é algo subjetivo.

Agora que você já sabe o que é qualidade e como ela pode ser avaliada, vamos tentar aplicar estes conceitos aos produtos de software e ao processo de desenvolvimento de *software*. Inicialmente, vamos encontrar um grande problema: muitas pessoas acham que criar programas é uma arte que não pode seguir regras, normas ou padrões. Isto acontece principalmente porque:

- Produtos de *software* são complexos, até mais do que o *hardware*

onde executam;

- *Software* não tem produção em série. Seu custo está no projeto e desenvolvimento;
- *Software* não se desgasta e nem se modifica com o uso;
- O *Software* é invisível. Sua representação em gráficos e diagramas não é precisa;
- A Engenharia de *Software* ainda não está madura, é uma tecnologia em evolução;
- Não há um acordo entre os profissionais da área sobre o que é Qualidade de *Software*.

Apesar de tudo isso, você precisa entender que o problema não está no *Software* em si, mas na forma como as pessoas tem desenvolvido *software* até os dias de hoje. Você provavelmente já ouviu dizer que "Se os engenheiros construíssem prédios como os analistas constroem *software*, um único pica-pau destruiria a humanidade". Exageros à parte, você precisa se conscientizar que nós precisamos aplicar na indústria de *software* os conceitos de qualidade, urgentemente.

Produto de *software*:

- O cliente determina as características que o produto deve ter;
- Especificação de características de qualidade de maneira ambígua;
- Dificuldade de escrever especificações de *software* completas.

A qualidade pode ser conseguida definindo-se padrões e procedimentos em uma organização e seguindo-os.

- Gerenciamento de qualidade?
- Cultura da qualidade?
- Padrões e Procedimentos?

Gerenciamento da qualidade do *software* consiste:

- Garantia da qualidade: Procedimento organizacional da empresa;
- Planejamento da qualidade: Seleção de procedimentos e padrões apropriados a um projeto específico;
- Controle da qualidade: Confirmar que os procedimentos e padrões foram seguidos;

- O gerenciamento da qualidade deve ser realizado por uma equipe específica e independente.

Qualidade de Processo:

- A qualidade do processo influencia no produto?
- Gerenciar a qualidade do processo envolve:
  - Definições de padrões de processo;
  - Monitoração do processo de desenvolvimento;
  - Relato do processo de *software* para os gerentes e clientes.
- Processos padrões devem ser utilizados independentes do tipo de *software*?

Padrões:

Podemos ter 2 tipos de padrões.

1. Padrões de Produtos

Estabelecem os padrões que se aplicam ao produto de *software* em desenvolvimento.

2. Padrões de Processos:

Estabelece processos a ser seguidos para garantir que os padrões de produtos sejam atingidos.

São importantes por:

- Refletirem as melhores práticas de desenvolvimento de *software*;
- Provêem um *framework* para a implantação de processos de *software*;
- Ajudam a continuidade, quando as mesmas práticas são adotadas por uma organização

Porém:

- Podem ser burocráticos e desnecessários em alguns tipos de projeto;
- Devem ser adotados por todo o grupo de desenvolvedores;
- Devem ser customizados para cada tipo de projeto.

ISSO 9000:

- Conjunto de padrões internacionais que pode ser usado para o gerenciamento da qualidade em todos os tipos de indústrias;

- Baseado nele, pode-se definir os procedimentos de garantia de qualidade através da confecção do manual que definirá o processo de qualidade.

Padrões de Documentações:

São importantes porque é o único modo tangível de representação do *software* e do processo deste.

São 3 os padrões de documentações:

- Padrão do processo de documentação: Define o processo para a produção de documentos;
- Padrão de Documentos: Define a estrutura e apresentação do documento;
- Padrão de Intercâmbio de documentos: Define a compatibilidade de todas as cópias eletrônicas do documento.

Planejamento da Qualidade:

É o processo de desenvolvimento de um plano de qualidade para um projeto de *software*;

Deve estabelecer as qualidades desejadas em um *software* e descrever como elas serão avaliadas.

O Plano de qualidade deve conter:

Apresentação do produto;
Planos de produto;
Descrição do processo;
Metas de qualidade;
Riscos e gerenciamento de riscos.

Planejamento da Qualidade:

São atributos em potenciais da qualidade:

Segurança;
Facilidade de uso;
Confiabilidade;
Eficiência;
Complexidade.

Controle da Qualidade:

Envolve a monitoração do processo de desenvolvimento do *software* para assegurar que os procedimentos e padrões estão sendo seguidos. Pode ser usado para isso:

- Revisões da Qualidade:

Envolve um grupo de pessoas que examinam todo o processo

- Avaliações automáticas de *software*:

O *Software* e documentos produzidos são avaliados por um programa específico.

Medições e Métricas de *Software*:

A Medição de um *software* consiste em derivar um valor numérico para algum atributo do produto de *software* ou de um processo.

O Uso de métricas exige um processo bem definido;

Não existem ferramentas para coleta e análise dos dados referentes às métricas.

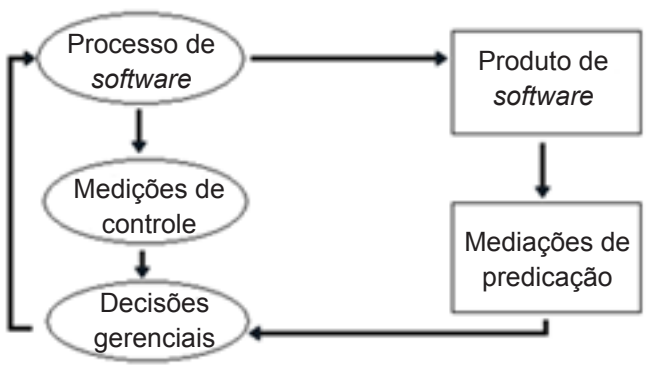
Medições e Métricas de Software:

Métricas de Controle: São associadas ao processo de *software*.

Métricas de Predição: São associadas ao produto de *software*.

É impossível medir os atributos de qualidade de um *software*.

Medições de Predições e Controle



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Métricas de produto de *Software*:

- Dinâmicas:

Coletadas com o programa em execução.

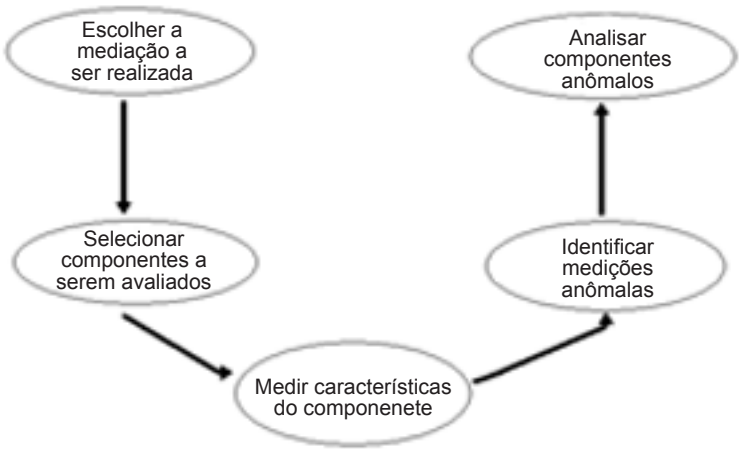
Ex.: Tempo.

- Estáticas:

Coletadas por meio das representações do sistema.

Ex.: complexidade.

Processo de Medição de Produto



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Implantação da Função de Qualidade

A implantação da função de qualidade (IFQ) (*Quality Function Deployment*, QFD) é uma técnica que traduz as necessidades do cliente para requisitos técnicos do *software*. A IFQ "concentra-se em maximizar a satisfação do cliente a partir do processo de engenharia de *software*" [ZUL92]. Para conseguir isso, a IFQ enfatiza o entendimento do tem valor para o cliente e depois implanta esses valores por meio do processo de engenharia. A IFQ identifica três tipos de requisitos [ZUL,92]:

Requisitos normais: Esses requisitos refletem os objetivos e metas estabelecidos para um produto ou sistema durante as reuniões com o cliente. Se esses requisitos estiverem presentes, o cliente fica satisfeito. Exemplos de requisitos normais podem ser os tipos de mostradores gráficos requeridos, funções específicas do sistema e níveis de desempenho definidos.

Requisitos esperados: Esses requisitos estão implícitos no produto ou sistema e podem ser tão fundamentais que o cliente não se refere a eles explicitamente. Sua ausência seria causa de insatisfação significativa. Exemplos de requisitos esperados são: facilidade de interação homem/máquina, correção e confiabilidade operacional global e facilidade de instalação do *software*.

Requisitos excitantes: Esses requisitos refletem características que vão além das expectativas do cliente e mostram ser muito satisfatórias quando presentes. Por exemplo, um *software* de processamento de texto

é solicitado com as características padrão. O produto entregue dispõe de várias facilidades para *layout* de página que são muito agradáveis e inesperados. Na verdade, a IQC cobre todo o processo de engenharia [PAR96]. No entanto, muitos conceitos da IFQ são aplicáveis à atividade de levantamento de requisitos. Apresentamos apenas um panorama desses conceitos (adaptados para *software* de computadores) nos parágrafos seguintes.

Em reuniões com o cliente, a implantação de funções é usada para determinar o valor de cada função necessária ao sistema. A implantação da informação identifica tanto os objetos de dados quanto os eventos que o sistema deve consumir e produzir. Eles são ligados às funções. Finalmente, a implantação de tarefas examina o comportamento do sistema ou produto dentro do contexto do seu ambiente. A análise de valor é conduzida para determinar a prioridade relativa dos requisitos determinados durante cada uma das três implantações.

A IFQ usa entrevistas com o cliente e observação, levantamentos e exame de dados históricos (por exemplo, relatórios de problemas) como dados brutos para a atividade de levantamento de requisitos. Esses dados são então traduzidos em uma tabela de requisitos – chamada de tabela da voz do cliente – que é revisada com o cliente. Diversos diagramas, matrizes e métodos de avaliação são então usados para levantar os requisitos esperados e para tentar derivar os requisitos excitantes [BOS91].

Processo de Projeto e Qualidade de Projeto

Projeto de *software* é um processo interativo por meio do qual os requisitos são traduzidos em um "documento" para construção do *software*. Inicialmente, o documento mostra uma visão holística do *software*.

Ou seja, o projeto é representado em um nível alto de abstração – nível que pode ser diretamente relacionado ao objetivo específico do sistema e a requisitos mais detalhados de dados, funcionais e comportamentais. À medida que ocorrem as interações de projeto, os refinamentos subsequentes levam a representações de projeto em níveis de abstração muito mais baixos. Esses ainda podem ser relacionados aos requisitos, mas a conexão é mais sutil.

Ao longo do processo, a qualidade do projeto em evolução é avaliada com uma série de revisões técnicas formais ou *walkthroughs* de

projeto. McGlaughlin (1991) sugere três características que servem de orientação para a avaliação de um bom projeto.

- O projeto deve implementar todos os requisitos explícitos contidos no modelo de análise e acomodar todos os requisitos implícitos desejados pelo cliente;
- O projeto deve ser um guia legível, compreensível para aqueles que geram código e para os que testam e, subsequentemente, dão suporte ao *software*;
- O projeto deve fornecer um quadro completo de *software*, focalizando os domínios de dados, funcional e comportamental sob perspectiva de implementação.

Cada uma dessas características é na verdade uma meta do processo de projeto. Entretanto, como cada uma dessas metas é atingida?

Diretrizes de qualidade: Para avaliarmos a qualidade de uma representação do projeto, precisamos estabelecer critérios técnicos para um bom projeto. Apresentamos as seguintes diretrizes:

- Um projeto deve exibir uma arquitetura que (a) tenha sido criada por meio de estilos ou padrões arquiteturais reconhecíveis; (b) seja composta de componentes que exibam boas características de projeto, e (c) pode ser implementada de modo evolucionário, facilitando assim a implementação e o teste;
- Um projeto deve ser modular; o *software* deve ser logicamente particionado em elementos ou subsistemas;
- Um projeto deve conter representações distintas dos dados, arquitetura, interfaces e componentes;
- Um projeto deve levar as estruturas de dados adequadas às classes a ser implementadas e baseadas em padrões de dados reconhecíveis;
- Um projeto deve levar a componentes que exibam características funcionais independentes;
- Um projeto deve levar a interfaces que reduzam a complexidade das conexões entre componentes e com o meio ambiente externo;
- Um projeto deve ser derivado por meio de um método passível de repetição que seja pela informação obtida durante a análise

dos requisitos do *software*;
- Um projeto deve ser representado usando uma notação que efetivamente comunique seu significado.

Essas diretrizes de projeto não são satisfeitas por acaso. A engenharia de projeto incentiva um bom projeto por meio da aplicação de princípios fundamentais de projeto, metodologia sistemática e revisões precisas.

Atributos de qualidade: A *Hewlett-Packard* [GRA87] desenvolveu um conjunto de atributos de qualidade de *software* que recebeu a sigla FURPS – *Functionality* (funcionalidade), *Usability* (usabilidade), *Reability* (confiabilidade), *Performance* (desempenho) e *Supportability* (suportabilidade). Os atributos de qualidade FURPS representam uma meta para todo o projeto de *software*:

- Funcionalidade é avaliada pela observação do conjunto de características e capacidades do programa, generalidade das funções entregues e segurança do sistema global.
- Usabilidade é considerada usando fatores humanos, estética, consistência e documentação globais.
- Confiabilidade é avaliada medindo a frequência e a severidade das falhas, a precisão dos resultados da saída, o tempo médio entre falhas (*mean-time-to-failure* – MTTF), a capacidade de recuperação de falhas e previsibilidade do programa.
- Desempenho é medido pela velocidade de processamento, tempo de resposta, consumo de recursos, vazão (*throughput*) e eficiência.
- Suportabilidade combina a capacidade de estender o programa (extensibilidade), adaptabilidade, reparabilidade – esses três atributos representam um termo mais comum, manutenabilidade -, além de testabilidade, compatilibilidade, configurabilidade (a capacidade de organizar e controlar elementos da configuração do *software*; facilidade com a qual o sistema pode ser instalado e facilidade com a qual problemas podem ser localizados.

Nem todo atributo de qualidade de *software* é igualmente ponderado á medida que o projeto de *software* é desenvolvido. Uma aplicação pode salientar funcionalidade com ênfase especial em

segurança. Outra pode exigir desempenho com ênfase particular na velocidade de processamento. Uma terceira pode enfocar a confiabilidade. Indiferentemente da ponderação, é importante notar que esses atributos de qualidade devem ser considerados quando o projeto começa e não depois que o projeto estiver completo e a construção começada.

Qualidade de *Software*

Mesmo os mais sofridos desenvolvedores de *software* concordam que o *software* de alta qualidade é uma meta importante. Mas como definimos qualidade? Em sentido geral, qualidade de *software* é a satisfação de requisitos e de desempenho explicitamente declarados, normas de desenvolvimento explicitamente documentadas e características implícitas que são esperadas em todo o *software* desenvolvido profissionalmente.

Há pouca dúvida de que esta definição poderia ser modificada ou estendida e debatida interminavelmente. A definição serve para enfatizar três pontos importantes:

- Requisitos de *software* é a fundação a partir da qual a qualidade é medida. Falta de conformidade com os requisitos é falta de qualidade.
- Normas especificadas definem um conjunto de critérios de desenvolvimento que guiam o modo pelo qual o *software* é construído. Se os critérios não são seguidos, quase sempre vai resultar em falta de qualidade.
- Há um conjunto de requisitos implícitos que frequentemente não são mencionados (por exemplo, o desejo de facilidade de uso). Se o *software* satisfaz seus requisitos explícitos, mas deixa de satisfazer os requisitos implícitos, a qualidade do *software* é suspeita.

Qualidade em *software* é uma mistura complexa de fatores que variam com cada aplicação diferente e com os clientes que a encomendam.

Fatores de Qualidade de McCall

Os fatores que afetam a qualidade do *software* podem ser categorizados em dois amplos grupos: (1) fatores que podem ser medidos

diretamente (por exemplo, defeitos por ponto por função) e (2) fatores que podem ser medidos apenas indiretamente (por exemplo, usabilidade e manutenabilidade). Em cada caso devem ocorrer medições. Devemos comparar *software* (documentos, programas e dados) a algum valor e chegar a uma indicação de qualidade.



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

McCall, Richards e Walters [MCC77] propõem uma categorização útil de fatores que afetam a qualidade do *software*. Esses fatores de qualidade do *software*, mostrados acima, concentram-se nos três aspectos importantes de um produto de *software*: suas características operacionais, sua habilidade de passar por modificações e sua adaptabilidade a novos ambientes.

Com referência aos fatores mostrados acima, McCall et al (1977), dão as seguintes descrições:

Correção: Quanto um programa satisfaz sua especificação e preenche os objetivos da missão do cliente.

Confiabilidade: Quanto se pode esperar que um programa realize a função pretendida com a precisão exigida.

Eficiência: Quantidade de recursos de computação e código necessários para um programa realizar sua função.

Integridade: Quanto do acesso ao *software* ou dados por pessoas não-autorizadas pode ser controlado.

Usabilidade: O esforço para aprender, operar, preparar entradas e interpretar saídas de um programa.

Manutenabilidade: O esforço necessário para localizar e consertar um erro de programa. Esta é uma definição muito limitada.

Flexibilidade: O esforço necessário para mudar um programa operacional.

Testabilidade: Esforço necessário para testar um programa, a fim de garantir que ele realize a função esperada.

Portabilidade: Esforço necessário para transferir de um ambiente de *hardware* ou *software* para outro.

Reutilização: Quanto de um programa (ou parte dele) pode ser reusado em outras aplicações – relativo ao empacotamento e escopo das funções que o programa realiza.

Interoperabilidade: Esforço necessário para acoplar um sistema a outro.

É difícil, e em alguns casos, impossível desenvolver medidas diretas desses fatores de qualidade. De fato, muitas das métricas definidas por McCall et al (1977) podem ser medidas apenas subjetivamente.

As métricas podem estar em forma de lista de verificação (*checklist*) que é usada para "atribuir grau" e atributos específicos de *software* [CAV78].

Impacto na Qualidade, Produtividade e Custo

Evidências de caos de estudo da indústria, em número considerável (por exemplo, [HEN95], [MCM95]), indicam que substanciais benefícios do negócio podem ser derivados de um reuso de *software* agressivo. A qualidade do produto, a produtividade de desenvolvimento e o custo global são melhorados.

Qualidade: Em condições ideais, um componente de *software* desenvolvido para reuso estaria verificado e não conteria defeitos. Na realidade, a verificação formal não é realizada rotineiramente e defeitos podem ocorrer, e de fato ocorrem. No entanto, com cada recurso são encontrados e eliminados os defeitos e, como resultado, a qualidade do componente melhora. Ao longo do tempo, o componente torna-se virtualmente livre de defeitos.

Em um estudo conduzido na Hewllet Packard, Lim (1994) relata que a taxa de defeitos para código reusado é 0,9 defeito por KLOC, enquanto a taxa para *software* produzido a partir do zero é 4,1 defeitos por KLOC. Para uma aplicação composta por 68% de código reusado, a taxa de defeitos foi de 2,0 defeitos por KLOC – uma melhora de 51% da

taxa esperada, se a aplicação tivesse sido desenvolvida se reuso. Henry e Faller (1995) relatam uma melhora de 35% na qualidade. Apesar de relatos anedóticos sobre percentual de melhora de qualidade espalhar, é justo dizer que o reuso fornece um benefício significativo em termos de qualidade e confiabilidade para o *software* produzido.

Produtividade: Quando são aplicados componentes reusáveis ao longo do processo de *software*, é gasto menos tempo criando planos, modelos, documentos, códigos e dados necessários para produzir um sistema em condições de ser entregue. Daí, decorre que é entregue ao cliente o mesmo nível de funcionalidade, com menos esforço de entrada. Assim sendo, a produtividade serem notoriamente difíceis de interpretar, parece que 30 a 50% de reuso podem resultar em 25 a 45% de melhora de produtividade.

Custo: A economia líquida do custo para reuso é estimada pela projeção do custo do projeto, se tivesse que ser produzido a partir do zero, C^1, subtraindo depois a soma dos custos associados com o reuso, C^2, e os custos reais do *software* na hora de ser entregue, C^3.

C^1 pode ser determinado pela aplicação de uma ou mais técnicas de estimação. Os custos associados com reuso, C^2, incluem [CHR95]: análise e modelagem de domínio, desenvolvimento da arquitetura de domínio, documentação ampliada para facilitar o reuso, suporte e aperfeiçoamento dos componentes do reuso, direitos autorais e licenças para componentes adquiridos externamente, criação ou aquisição e operação de um repositório de reuso, treinamento de pessoal em projeto de construção para reuso. Apesar de os custos associados com a análise de domínio e com a operação do repositório de reuso poder ser substanciais, muitos dos outros custos aqui mencionados referem-se a itens que fazem parte da boa prática de engenharia de *software* sendo ou não o reuso uma prioridade.

Exercícios Propostos



1. Pesquise sobre qualidade de *software*.
2. Qual a importância dos padrões de processo?
3. Caracterize os Fatores de Qualidade de McCall.
4. Determine os padrões de documentações.

Saiba Mais

VON Staa, Arndt. Engenharia de Programas. LTC. Capitulo 2 – Qualidade de *Software*, 1983.

PRESSMAN, Roger. Engenharia de *software*. *McGraw Hill*. Capítulo 8 – Garantia da Qualidade de *Software*, 2006.

Para Pensar

Qualidade e confiabilidade são conceitos relacionados, mas são fundamentalmente diferentes em vários aspectos. Discuta-os.

É possível avaliar a qualidade de *software*, caso o cliente mudar continuamente o que deve ser feito?

Resumindo

O capítulo mostra as definições da qualidade de *software*, onde relacionamos modelos utilizados para definir e avaliar a qualidade. A importância dos métodos e métricas para qualidade de *software*, constante busca pela mesma se tornou uma atividade essencial dentro das empresas. Planejamento adequado para que a qualidade do *software* seja atingida, conforme a definição de qualidade que deverá ser alcançada.

A utilização de modelos de qualidade como referência no processo de desenvolvimento de *software* pode funcionar concomitantemente com qualquer outro tipo de modelo. Para isso, são necessários modelos, padrões, procedimentos e técnicas para atingir essas metas de qualidade propostas. Para tanto, todas as etapas do ciclo de vida de engenharia de *software* devem ser contempladas com atividades que visam garantir a qualidade tanto do processo quanto do produto.

UNIDADE 05

MÉTRICAS DE QUALIDADE DE *SOFTWARE*

OBJETIVOS:

- **Definir Métricas de Qualidade de *Software*;**
- **Identificar e definir as Medidas de Qualidade;**
- **Analisar e Integrar as Métricas no Processo de *Software*;**
- **Compreender as Métricas para Pequenas Organizações.**

5 MÉTRICAS DE QUALIDADE DE *SOFTWARE*

A principal meta da engenharia de *software* é produzir um sistema, aplicação ou produto de alta qualidade em um espaço de tempo que satisfaça uma necessidade de mercado. Para atingir essa meta, engenheiros de *software* devem aplicar métodos efetivos combinados com modernas ferramentas no contexto de um processo de *software* amadurecido. Além disso, um bom engenheiro de *software* (e bons gerentes de engenharia de *software*) precisa medir se a alta qualidade deve ser alcançada.

Métricas privadas, colhidas por engenheiros de *software* individuais, são ajustadas para fornecer resultados do nível do projeto. Apesar de muitas medidas de qualidade poder ser coletadas, a tendência principal do nível de projeto é medir erros e defeitos. Métricas derivadas dessas medidas fornecem indicação e efetividade da engenharia de qualidade de *software*, individual e de grupo e das atividades de controle.

Métricas com erros do trabalho (por exemplo, requisitos ou projeto) por ponto por função, erros descobertos por hora de revisão e erros descobertos na hora de teste são métricas que fornecem compreensão da eficácia de cada uma das atividades por elas determinadas. Dados de erro também podem ser usados para calcular a eficiência de remoção de defeitos (*defect removal efficiency*, DRE), para cada atividade de arcabouço de processo.

Medição de Qualidade

Apesar de existirem várias medidas de qualidade de *software*, a correção, manutenabilidade, integridade e usabilidade fornecem indicadores úteis à equipe de *software*. Gilb (1988) sugere definições e medidas para cada uma.

Correção: Um programa precisa operar corretamente ou é de pouco valor para seus usuários. Correção é o grau em que o *software* desempenha a sua necessária função. A medida mais comum de correção é defeitos por KLOC, em que um defeito é definido como uma falha verificada de obediência aos requisitos. Quando se considera a qualidade geral de um produto de *software*, defeitos são problemas resultados por um usuário do programa, após o programa ter sido entregue para o uso geral. Para fins de avaliação de qualidade, defeitos são contados durante um período de tempo padrão, em geral um ano.

Manutenabilidade: Manutenção de *software* consome mais esforço do que qualquer atividade de engenharia de *software*. Manutenabilidade é a facilidade com que um programa pode ser corrigido, se um erro é encontrado, adaptado, se seu ambiente se modifica ou é aperfeiçoado e se o cliente deseja uma modificação nos requisitos. Não há modo de medir manutenabilidade diretamente; assim, precisamos usar medidas indiretas. Uma métrica simples orientada a tempo é tempo médio para modificação (*mean-time-to-change*, MTCC), que é o tempo despendido para analisar o pedido de modificação. Projetar uma modificação adequada, implementar a modificação, testá-la e distribuí-la para todos os usuários. Em média, programas manuteníveis terão um MTTC mais baixo (para tipos equivalentes de modificação) do que programas não-manuteníveis.

Integridade: A integridade de *software* tornou-se cada vez mais importante na era dos *hackers* e equipamentos *firewalls*. Esse atributo mede a capacidade de o sistema resistir a ataques (tanto acidentais quanto intencionais) à sua segurança. Os ataques podem ser feitos aos três componentes do *software*: programas, dados e documentos.

Para medir a integridade, dois atributos adicionais precisam ser definidos: ameaça e segurança. Ameaça é a probabilidade (que pode ser estimada ou originada de evidência empírica) de um ataque de tipo específico ocorrer dentro de certo período. Segurança é a probabilidade (que pode ser estimada ou originada de evidência empírica) de um ataque ser repetido. A integridade do sistema pode ser então definida como.

$$\text{Integridade} = \sum [1 - (\text{ameaça} \times (1 - \text{segurança}))]$$

Por exemplo, se a ameaça (a probabilidade que um ataque venha a ocorrer) é de 0,25 e a segurança (a probabilidade de repelir o ataque) é de 0,95, a integridade do sistema é de 0,99 (muito alta). Se por outro

lado, a probabilidade de ameaça é de 0,50 e a probabilidade de repelir é de apenas 0,25 e a integridade do sistema é de 0,63 (inaceitavelmente baixa).

Usabilidade: Se um programa não é fácil de usar, está frequentemente fadado ao fracasso, mesmo se as funções que desempenha foram valiosas. Usabilidade é uma tentativa de quantificar facilidade de uso. Os quatro fatores recém-descritos são apenas um exemplo dos que têm sido propostos como medida da qualidade de software.

Eficiência na Remoção de Defeitos

Uma métrica de qualidade, que fornece benefício tanto no nível de projeto quanto de processo, é a eficiência na remoção de defeitos (*defect removal efficiency*, DRE). Em essência, DRE é uma medida da capacidade de filtragem das atividades de controle e garantia de qualidade de *software*, à medida que são aplicadas Às atividades de arcabouço de processo.

Quando considerada para o projeto todo, a DRE é definida da seguinte maneira:

DRE = *E/* (*E* + *D*), em que E é a quantidade de erros encontrados antes da entrega do *software* ao usuário final e D é a quantidade de defeitos encontrados após a entrega.

O valor ideal da DRE é 1, ou seja, nenhum defeito é encontrado no *software*. Na prática, *D* é maior que 0, mas o valor da DRE ainda pode ficar próximo de 1. De fato, à medida que *E* aumenta, é provável que o valor final de *D* diminua (os erros são filtrados antes que se transformem em defeitos). Se usada como métrica que fornece indicador da capacidade de filtragem das atividades de garantia e controle de qualidade, a DRE motiva uma equipe de projeto de *software* a instituir regras para encontrar tentos erros quanto possíveis antes da entrega.

A DRE também pode ser usada no projeto para avaliar a capacidade de uma equipe encontrar erros antes que eles sejam passados para a próxima atividade de arcabouço ou tarefa de engenharia de *software*. Por exemplo, a tarefa de análise de requisitos produz um modelo de análise que pode ser revisado para encontrar e corrigir erros. Os erros que não são encontrados durante a revisão do modelo de análise são passados para a tarefa de projeto (em que eles podem ou não ser encontrados). Quando usada nesse contexto, redefinimos a DRE como:

$DRE_i = \underline{E}_i / (E_i + E_i+1)$, em que E_i, é o número de erros encontrados durante a atividade i de engenharia de *software*, e E_i+1 é a quantidade de erros encontrados durante a atividade de engenharia de *software* i+1, que são atribuíveis a erros que não foram descobertos na atividade de engenharia de *software* i.

Um objetivo de qualidade para uma equipe de *software* (ou para um engenheiro de *software* individualmente) é conseguir que a DRE_i fique próxima de 1. Isto é, os erros devem ser filtrados antes que passem para a próxima atividade.

Integração de Métricas no Processo de *Software*

A maioria dos desenvolvedores de *software* ainda não mede e, infelizmente, muitos têm pouca vontade de começar. Tentativas de coletar medidas, em que nenhuma era anteriormente coletada, frequentemente causam resistência. “Porque precisamos fazer isso?”, pergunta um atarefado gerente de projeto. “Não entendo a razão.”, reclama um profissional sobrecarregado.

Argumentos Favoráveis a Métricas de *Software*

Porque é tão importante medir o processo de engenharia de processo e o produto (*software*) que ele produz? A resposta é relativamente óbvia. Se não medirmos, não haverá forma real de determinar se estamos melhorando. E se não estivermos melhorando, estaremos perdidos.

Pela solicitação e avaliação de medidas de produtividade e qualidade, uma equipe de *software* (e os seus gerentes) pode estabelecer metas significativas para aperfeiçoamento do processo de engenharia de *software*. Se o processo pulo qual é desenvolvido puder ser aperfeiçoado, poderá resultar em impacto direto na base. Mas, para estabelecer metas de aperfeiçoamento, o estado atual do desenvolvimento de *software* deve ser entendido. Assim, a medição é usada para estabelecer um referencial do processo a partir do qual melhorias podem ser avaliadas.

O rigor do dia-a-dia do trabalho de projeto de *software* deixa pouco tempo para raciocínio estratégico. Gerentes de projeto de *software* estão preocupados com assuntos mais comuns (mas igualmente importantes): desenvolver estimativas significativas de projeto, produzir sistemas de alta qualidade e entregar produtos dentro do prazo. Usando-se a medição para estabelecer um referencial de projeto, cada um desses

aspectos torna-se mais gerenciável. Já observamos que um referencial serve como base para estimativas. Além disso, coletar métricas de qualidade permite à organização "sintonizar" seu processo de *software* para remover as "poucas, mas vitais" causas de defeitos, que têm maior impacto no desenvolvimento de *software*.

Estabelecimento de uma Referência

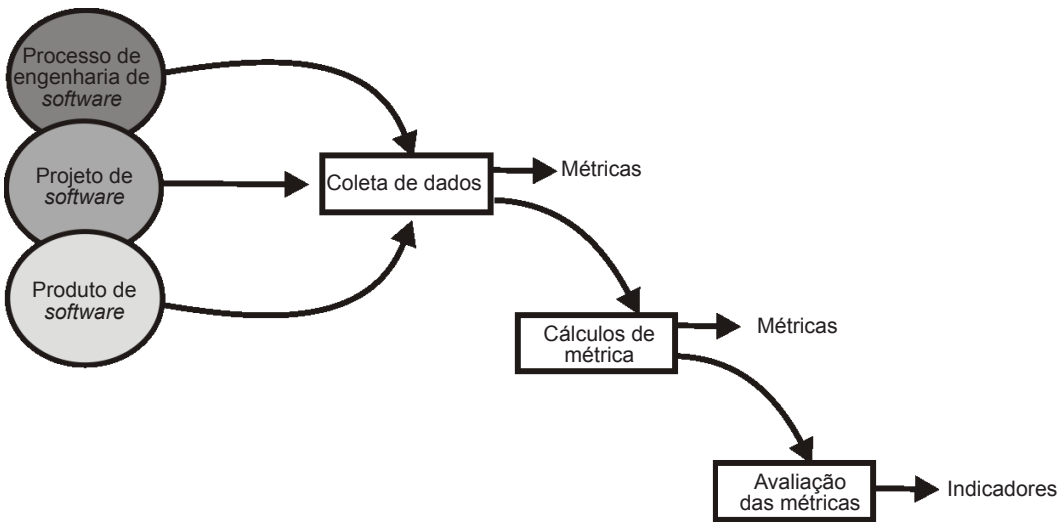
Pelo estabelecimento de uma referência para as métricas, os benefícios podem ser obtidos nos níveis (técnicos) de processo, projeto e produto. Mesmo assim, a informação que é coletada não precisa ser fundamentalmente diferente. As mesmas métricas podem servir a muitos mestres. A referência para as métricas consiste em dados coletados de projetos anteriores de desenvolvimento de *software* e pode ser tão simples como na tabela apresentada a seguir, ou tão complexa como um abrangente banco de dados contendo dezenas de medidas de projeto e as métricas delas derivadas.

Projeto	LOC	Esforço	$ (000)	Pág. Doc.	Erros	Defeitos	Pessoas
alfa	12.100	24	168	365	134	29	3
beta	27.200	62	440	1224	321	86	5
gama	20.200	43	314	1050	256	64	6
...	...	...	...	...	...	...	...

Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Para constituir uma ajuda efetiva na melhoria do processo e/ou na estimativa de custo e esforço, os dados referenciais devem ter os seguintes atributos: (1) os dados precisam ser razoavelmente precisos – "estimativas", sem base sólida, relativas a projetos anteriores, devem ser evitadas; (2) dados devem ser coletados por tantos projetos quanto possível; (3) as medições, por exemplo, deve ser consistente, o qual uma

linha de código deve ser interpretada uniformemente ao longo de todos os projetos os quais os dados são coletados; (4) as aplicações devem ser análogas ao trabalho que precisa ser estimado – faz pouco sentido usar uma referência para um trabalho de sistemas de informação com processamento por lotes (batch), para estimar uma aplicação embutida de tempo real.



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Coleta, Cálculo e Avaliação de Métricas

O processo para estabelecer uma referência está ilustrado na imagem acima. Seria ideal que os dados necessários tivessem sido coletados de modo progressivo. Infelizmente, esse caso é raro. Assim, coletar dados exige uma investigação histórica de projetos anteriores para reconstruir dados necessários. Uma vez coletadas essas medidas (inquestionavelmente, o passo mais difícil), o cálculo das métricas é possível. Dependendo da amplitude das medidas coletadas, as métricas podem abranger um amplo intervalo de métricas orientadas a aplicação (por exemplo, LOC, FP, orientadas a objetos, de *WebbApp*), bem como outras métricas de qualidade e orientadas a projeto. Finalmente, as métricas precisam ser avaliadas e aplicadas durante a estimativa, o trabalho técnico, o controle do projeto e as melhorias do processo. A avaliação de métricas focaliza as razões subjacentes dos resultados obtidos e produz um conjunto de indicadores que dão diretrizes ao projeto ou processo.

Métricas para Pequenas Organizações

A grande maioria das organizações de desenvolvimento de *software* tem menos de 20 pessoas de *software*. É irracional, e na maior parte dos casos inviável esperar que tais organizações desenvolvam programas abrangentes de métricas de *software*. Todavia, é razoável sugerir que organizações de *software* de todos os tamanhos meçam e depois usem as métricas resultantes para ajudar a aperfeiçoar seu processo local de *software* e a qualidade e pontualidade dos produtos que produzem.

Uma abordagem de bom-senso para a implementação de qualquer atividade relacionada e processo de *software* seria: torne simples, ajuste as necessidades locais e certifique-se de que tem valor. Nos parágrafos que seguem, examinamos como essas diretrizes se relacionam a métricas para pequenas empresas. “Torne simples” é uma diretriz que funciona razoavelmente bem em muitas atividades. Contudo, como originamos um conjunto de métricas de *software* “simples”, que ainda tenha valor, e como podemos estar certos de que essas métricas simples vão satisfazer às necessidades de uma organização de *software* específica? Começamos focalizando não a medição, mas os resultados. O grupo de *software* é interrogado para definir um único objetivo que requer aperfeiçoamento. Por exemplo, “reduza o prazo para avaliar e implementar pedidos de modificação”. Uma pequena organização pode selecionar o seguinte conjunto de medidas facilmente coletáveis:

- Tempo (horas ou dias) transcorrido entre o momento em que o pedido é feito até que a avaliação seja completada, t *fila**
- Esforço (pessoa-horas) para realizar a avaliação, *W* aval*
- Tempo (horas ou dias) transcorrido desde o término da avaliação até a atribuição da ordem de modificação ao pessoal, *t aval*˙
- Esforço (pessoas-horas) necessário para fazer a modificação, *W modificação*˚
- Tempo necessário (horas ou dias) para fazer a modificação, *t modificação*˚
- Erros descobertos durante o trabalho para fazer a modificação, *E modificação*˚
- Defeitos descobertos depois que a modificação é entregue ao cliente, *D modificação*˚

Uma vez coletadas essas medidas, para um certo número de pedidos de modificação, é possível calcular o tempo total transcorrido desde o pedido até a implementação da modificação e a porcentagem do tempo transcorrido, absorvida pelo escoamento inicial, avaliação, atribuição e implementação da modificação. Analogamente, a porcentagem do esforço necessário para avaliação e implementação pode ser determinada. Essas métricas podem ser avaliadas no contexto dos dados de qualidade, *E modificação*˚ e *D modificação*˚. As porcentagens permitem discernir onde o processo do pedido de modificação sofre demora, e podem levar a passos de melhoria do processo para reduzir *t fila**, *W aval**, *t aval*˚ e/ou *E modificação*˚. Além disso, a eficiência na remoção de defeitos pode ser calculada como:

DRE = *Emodificação*/(*Emodificação* + *Dmodificação*)

DRE pode ser comparada com o tempo transcorrido e o esforço total para determinar o impacto das atividades de garantia de qualidade pelo tempo e o esforço necessários para fazer uma modificação.

Estabelecimento de um Programa de Métricas de *Software*

O *software Engineering Institute* desenvolveu um manual de diretrizes abrangente [PAR96] para estabelecer um programa de métricas de *software* "orientado a metas", que sugere os seguintes passos:

1. Identifique suas metas de negócio;
2. Identifique o que você deseja saber ou aprender;
3. Identifique suas submetas;
4. Identifique as entidades e atributos relacionados às suas metas;
5. Formalize suas metas de medição;
6. Identifique questões quantificáveis e os indicadores correlacionados que você vai usar para ajudá-lo a atingir suas metas de medição;
7. Identifique os elementos de dados que você vai coletar para construir os indicadores que ajudam a responder a suas perguntas;
8. Defina as medidas a ser usadas e torne essas definições operacionais;

9. Identifique as ações que você executará para programar as medidas;
10. Prepare um plano para programar as medidas.

Deixamos a discussão detalhada desses passos para o livro de diretrizes do SEI. No entanto, uma visão geral dos pontos-chave é aconselhável.

Como o *software* apoia funções do negócio, diferencia sistemas ou produtos baseados em computador ou age como um produto em si mesmo, as metas definidas para o negócio podem, quase sempre, ser rastreadas até metas específicas de engenharia de *software*. Por exemplo, considere uma empresa que produz sistemas avançados de segurança residencial, que tem substancial conteúdo de *software*. Trabalhando em equipe, gerentes de engenharia de *software* e de negócio podem desenvolver uma lista priorizada de metas de negócio:

1. Aperfeiçoar a satisfação de nossos clientes com nossos produtos;
2. Tornar nossos produtos mais fáceis de usar;
3. Reduzir o tempo para colocação de um novo produto no mercado;
4. Facilitar o apoio aos nossos produtos;
5. Aperfeiçoar nossa lucratividade global.

A organização de *software* examina cada meta de negócio e pergunta: Que atividades nós gerimos ou executamos e o que desejamos aperfeiçoar nessas atividades? Para responder a essas questões, o SEI recomenda a criação de uma "lista de questões-entidade" na qual todas as coisas (entidades) dentro do processo de *software*, que são geridas ou influenciadas pela organização de *software*, são anotadas. Exemplos de entidades incluem recursos de desenvolvimento, produtos de trabalho, código-fonte, casos de teste, pedidos de modificação, tarefas de engenharia de *software* e cronogramas.

Para cada entidade relacionada, o pessoal de *software* desenvolve um conjunto de perguntas para avaliar as características quantitativas da entidade (por exemplo, tamanho, custo, prazo para desenvolver). As questões originadas em consequência da criação da lista de questões-entidade levam à origem de um conjunto de submetas, que se relacionam diretamente com as entidades criadas e as atividades desenvolvidas,

como parte do processo de *software*. Considere a quarta meta: “Facilitar o apoio aos nossos produtos”. A seguinte lista de perguntas pode ser originada dessa meta [PAR96]:

- Os pedidos de modificação do cliente contêm a informação de que precisamos para avaliar adequadamente a modificação e depois implementá-la a tempo?
- Qual o tamanho da lista de pedidos de modificação pendentes?
- Nosso tempo de resposta para corrigir defeitos é aceitável em relação às necessidades do cliente?
- Nosso processo de controle de modificação está sendo seguido?
- As modificações com alta prioridade são implementadas a tempo?

Com base nessas perguntas, a organização de *software* pode criar a seguinte submeta: aperfeiçoar o desempenho da gestão do processo de modificação. As entidades e atributos do processo de *software*, que são relevantes para submeta, são identificados, e metas de medição associadas a eles são delineadas.

O SEI [PAR96] oferece diretrizes detalhadas para os passos 6 a 10 de sua abordagem de medição orientada a metas. Em essência, um processo de refinamento passo a passo é aplicado, no qual as metas são refinadas até chegar a perguntas, que são então refinadas até chegar a entidades e atributos, que são depois refinados até chegar a métricas.



Exercícios Propostos

1. Pesquise sobre Métricas de Qualidade de Software.
2. Pesquise sobre Métricas para Pequenas Organizações.
3. Qual a importância das Métricas de Qualidade de Software?
4. Determine o conjunto de medidas para uma pequena organização.
5. Ilustre o processo para estabelecer uma referência de Coleta, Cálculo e Avaliação de Métricas.

Saiba Mais



http://www.bfpug.com.br/Artigos/sistematica_metricas_simoes.htm

http://www.developer.com/tech/article.php/3644656/Software-Quality-Metrics.htm

Para Pensar:

As métricas fornecem a visão aprofundada necessária para criar modelos efetivos de análise e projeto, código sólido e testes rigorosos. Discuta com seus colegas esta colocação.

Resumindo



O capítulo mostrou as definições das métricas de qualidade de *software*. Métricas privadas, colhidas por engenheiros de *software* ajustadas para fornecer resultados do nível do projeto. Medidas de qualidade, tendência principal do nível de projeto para medir erros e defeitos. Uma métrica de qualidade que fornece benefício tanto no nível de projeto quanto de processo e a eficiência na remoção de defeitos.

Tentativas de coletar medidas, em que nenhuma era anteriormente coletada, frequentemente causam resistência. Foi estabelecido metas de aperfeiçoamento, o estado atual do desenvolvimento de *software*. A medição usada para estabelecer um referencial do processo a partir do qual melhorias podem ser avaliadas. Estabelecer uma referência para as métricas, os benefícios podem ser obtidos nos níveis (técnicos) de processo, projeto e produto, coletando dados para uma investigação histórica de projetos anteriores para reconstruir dados necessários.

UNIDADE 06

CONCEITOS DE QUALIDADE

OBJETIVOS:

- **Definir Qualidade;**
- **Determinar Controle de Qualidade;**
- **Compreender Impacto nos Custos de Defeitos de *Software*;**
- **Conhecer as Normas de Qualidade**



6 CONCEITOS DE QUALIDADE

Controle de variação é o âmago do controle de qualidade. Um fabricante deseja minimizar a variação entre os produtos que são produzidos, mesmo quando faz algo relativamente simples como a duplicação de DVDs. Certamente, isso não pode constituir problema – duplicar DVDs é uma operação trivial de fabricação e podemos garantir que duplicatas exatas do *software* são sempre criadas.

Será que podemos? Precisamos garantir que as trilhas estão colocadas nos DVDs dentro da tolerância especificada, de modo que a imensa maioria dos "drives" de DVDs possa ler a mídia. As máquinas de duplicação de disco podem, e de fato se desgastam, e saem da tolerância. Assim, mesmo um processo como a duplicação de um DVD pode encontrar problemas devido à variação entre as amostras.

Mas como isso se aplica ao trabalho de *software*? Como é possível uma organização de desenvolvimento de *software* precisar controlar por variação? De um projeto para outro, desejamos minimizar a diferença entre recursos previstos, necessários para completar um projeto, e os recursos usados de fato, incluindo pessoal, equipamento e tempo no calendário. Em geral, gostaríamos de nos certificar de que nosso programa de teste cobre uma porcentagem conhecida do *software*, de uma versão para outra. Desejamos não apenas minimizar a quantidade de defeitos que vão para o campo, mas também nos certificar que a variância no número de erros seja também minimizada de uma versão para outra. (Nossos clientes provavelmente vão ficar chateados se a terceira versão de um produto tem 10 vezes mais defeitos que a versão anterior).

Gostaríamos de minimizar as diferenças na velocidade e na precisão das nossas respostas, na linha telefônica (*hotline*) de apoio aos problemas dos clientes.

Qualidade

O *American Heritage Dictionary* define qualidade como "uma característica ou atributo de alguma coisa". Como atributo de um item, a qualidade se refere a características mensuráveis – coisas que nós podemos comparar com padrões conhecidos, tais como comprimento, cor, propriedades elétricas e maleabilidade. Todavia, o *software*, que é essencialmente uma entidade intelectual, é mais difícil de caracterizar do que objetos físicos.

No entanto, existem medidas das características de um programa. Essas propriedades incluem a complexidade ciclomática, a coesão, o número de pontos por função, de linhas de códigos e muito outras propriedades. Quando examinados um item baseado em suas características mensuráveis, duas espécies de qualidade podem ser encontradas: Qualidade de projeto e qualidade de conformidade.

Qualidade de projeto refere-se a características que os projetistas especificam para certo item. Qualidade de conformidade é o grau com que as especificações de projeto são seguidas durante a fabricação.

No desenvolvimento de *software*, a qualidade de projeto abrange os requisitos, as especificações e projeto do sistema. A qualidade de conformidade é um assunto concernente, principalmente, à implementação. Se a implementação segue o projeto e o sistema resultante satisfaz os requisitos e metas de desempenho, a qualidade de conformidade é alta.

Mas as qualidades de projetos e de conformidade são os únicos tópicos que os engenheiros de *software* devem considerar? Robert Glass (1998) alega que uma relação mais "intuitiva" é oportuna:

Satisfação do usuário = produto adequado + máxima qualidade + entrega do orçamento e do cronograma

No fundo, Glass argumenta que a qualidade é importante, mas se o usuário não está satisfeito, nada mais realmente importa. DeMarco (1999) reforça essa opinião quando afirma: "A qualidade de um produto corresponde a quanto ele muda o mundo para melhor". Essa visão da qualidade alega que se um produto de *software* fornece substancial benefício para seus usuários finais, eles podem estar dispostos a tolerar problemas eventuais de confiabilidade ou desempenho.

Controle de Qualidade

O controle de variação pode ser equiparado ao controle de qualidade. Mas como se consegue controle de qualidade? Controle de qualidade envolve a série de inspeções, revisões e testes usados ao longo do processo de *software* para garantir que cada produto de trabalho satisfaça os requisitos para ele estabelecidos. O controle de qualidade inclui um ciclo de realimentação no processo de trabalho que criou o produto. A combinação de medida e realimentação nos permite ajustar o processo quando os produtos de trabalho criados deixam de satisfazer suas especificações.

Um conceito-chave do controle de qualidade é que todos os produtos de trabalho têm especificações definidas e mensuráveis com as quais nós podemos comparar o resultado de cada processo. O ciclo de realimentação é essencial para minimizar os defeitos produzidos.

Garantia de Qualidade

Garantia da qualidade consiste em um conjunto de funções para auditar e relatar que avalia a efetividade e completeza das atividades do controle de qualidade. A meta da garantia de qualidade é fornecer à gerência os dados necessários para que fique informada sobre a qualidade do produto, ganhando assim compreensão e confiança de que a qualidade do produto está satisfazendo suas metas.

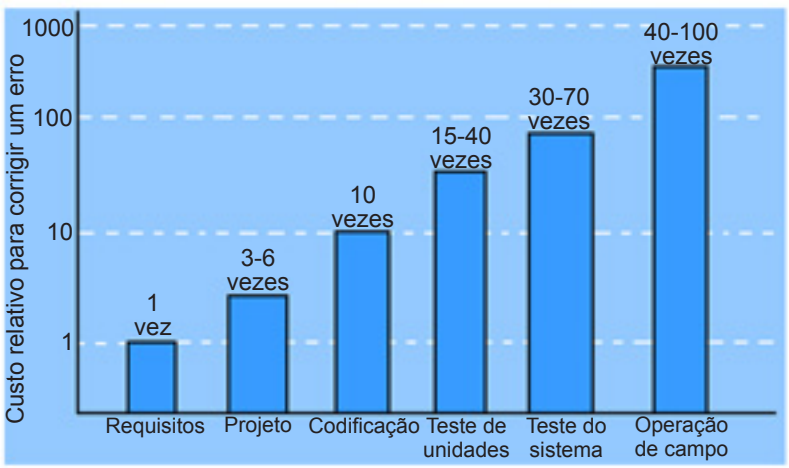
É claro que, se os dados fornecidos por meio da garantia de qualidade identificam problemas, é responsabilidade de a gerência cuidar dos problemas e aplicar os recursos necessários para resolver as questões da qualidade.

Custo da Qualidade

O custo da qualidade inclui todos os custos decorrentes da busca da qualidade ou da execução das atividades relacionadas à qualidade. Estudos de custos de qualidade são conduzidos para obter um referencial para os custos reais de qualidade, identificar as oportunidades para reduzir o custo de qualidade e fornecer uma base de comparação normalizada. A base de normalização é quase sempre a moeda local (ou dólares). Quando temos custos de qualidade normalizados na base do dólar, obtemos os dados necessários para avaliar onde estão as oportunidades

e aperfeiçoar nossos processos. Além disso, podemos avaliar o efeito dos aperfeiçoamentos em termos de dólares.

Os custos da qualidade podem ser divididos em custos associados com a prevenção, com a avaliação e com as falhas. Os custos da prevenção incluem planejamento da qualidade, revisões técnicas formais, equipamento de teste e treinamento. Os custos de avaliação incluem atividades para obter o entendimento da condição do produto na "primeira execução" de cada projeto. Exemplos de custos de avaliação incluem inspeção intra e interprocessos, calibração e manutenção do equipamento e teste.



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Os custos de falha são aqueles que desapareceriam se nenhum defeito aparecesse antes de se entregar um produto ao cliente. Os custos de falha podem ser subdivididos em custos de falhas internas e custo de falhas externas. Os custos de falhas internas ocorrem quando detectamos um defeito no nosso produto antes do embarque. Iincluem refazer, reparar e análise do modo como a falha ocorreu. Os custos de falhas externas são associados com os defeitos encontrados depois que o produto foi enviado ao cliente. Exemplos de custos de falha externa são soluções das queixas, devolução, substituição do produto, manutenção de linha de suporte e trabalho de garantia.

Como esperado os custos relativos para encontrar e reparar um defeito aumenta significativamente à medida que migramos dos custos de prevenção para os de detecção de falhas internas até os de falhas externas. A figura acima, baseada em dados coletados por Boehm e outros (1981), ilustra esse fenômeno.

Garantia de Qualidade de *Software*

Mesmo o mais exausto desenvolvedor de *software* vai concordar que *software* de alta qualidade é uma meta importante. Mas, como definimos qualidade? Um humorista disse certa vez: “Todo programa faz alguma coisa direito, apenas pode não ser a coisa que desejamos que ele fizesse”.

Muitas definições de qualidade de *software* têm sido propostas na literatura. Para o nosso objetivo, a qualidade de *software* é definida como:

Conformidade com requisitos funcionais e de desempenho explicitamente declarados, normas de desenvolvimento explicitamente documentadas e características implícitas, que são esperadas em todo *software* desenvolvido profissionalmente.

Há pouca dúvida de que essa definição poderia ser modificada ou estendida. De fato, uma definição definitiva de qualidade de *software* poderia ser debatida indefinidamente, a definição serve para enfatizar três pontos importantes:

- Os requisitos de *software* são a base pela qual a qualidade é medida. A falta de conformidade com os requisitos é falta de qualidade;
- As normas especificadas definem um conjunto de critérios de desenvolvimento que guia o modo pelo qual o *software* é submetido à engenharia. Se os critérios não são seguidos, irá ocorrer, quase certamente, falta de qualidade;
- Um conjunto de requisitos implícitos frequentemente não é mencionado (por exemplo, o desejo de facilidade de uso e boa manutenabilidade). Se o *software* satisfaz os requisitos explícitos, mas deixa de satisfazer os requisitos implícitos, a qualidade do *software* é suspeita.

Panorama Histórico

A garantia da qualidade é uma atividade essencial para qualquer negócio que faz produtos para serem usados por outros. Antes do século XX, a garantia da qualidade era de responsabilidade tão somente do artesão que construía um produto. A primeira função formal de garantia e controle de qualidade introduzida nos laboratórios Bell, em 1916,

disseminou-se rapidamente por todo o mundo industrializado. Durante os anos 40, abordagens mais formais para o controle de qualidade foram sugeridas. Elas se concentravam na medição e no aperfeiçoamento contínuo do processo [como elementos-chave da gerência de qualidade.

Hoje, toda empresa tem mecanismos para garantir a qualidade de seus produtos. Na realidade, declarações explícitas da preocupação da empresa com qualidade tornaram-se estratégia de mercado durante as ultimas décadas.

A história da garantia de qualidade no desenvolvimento de *software* segue um paralelo com a história da qualidade na fabricação de *hardware*. Durante os primeiros tempos da computação (1950 e 1960), a qualidade era responsabilidade tão somente do programador. Normas para garantia de qualidade de *software* foram introduzidas em contratos militares de desenvolvimento de *software* durante os anos 70 e se disseminaram rapidamente pelo desenvolvimento de *software* pelo mundo comercial [IEE94].

Ampliando a definição antes apresentada, a garantia da qualidade de *software* é um "padrão planejado e sistemático de ações" [SCH98], que são necessárias para garantir a alta qualidade do *software*. A implicação para o *software* é que várias partes diferentes têm responsabilidade sobre a garantia de qualidade de *software* – engenheiros de *software*, gerentes de projeto, clientes, pessoal de vendas e indivíduos que servem em um grupo SQA.

O grupo SQA atua como o representante residente do cliente. Isto é, pessoal que efetua SQA deve olhar o *software* do ponto de vista do cliente. O *software* satisfaz adequadamente os fatores de qualidade? O desenvolvimento do *software* foi conduzido de acordo com padrões pré-estabelecidos? As disciplinas técnicas desempenharam seu papel adequadamente como parte da atividade de SQA? O grupo SQA tenta responder a essas perguntas e a outras questões para garantir que a qualidade de *software* seja mantida.

Atividades de SQA

A garantia da qualidade de *software* é composta de uma variante de tarefas associadas a duas partes diferentes – os engenheiros de *software* que fazem o trabalho técnico e um grupo de SQA, que tem responsabilidade pelo planejamento, supervisão, registro, análise e relato da garantia da qualidade.

Os engenheiros de *software* buscam qualidade (e desenvolvem atividades de garantia da qualidade e de controle de qualidade) aplicando métodos e medidas técnicas sólidas, conduzindo revisões técnicas formais e efetuando teste de software bem planejado.

A missão do grupo de SQA é ajudar a equipe de software a conseguir um produto final de alta qualidade. O *Software* Engineering Institute recomenda um conjunto de atividades de SQA que trata do planejamento, supervisão, registro, análises e relato da garantia de qualidade. Essas atividades são executadas (ou facilitadas) por um grupo independente de SQA que conduz as seguintes atividades:

Prepara um plano SQA para um projeto

O plano é desenvolvido durante o planejamento do projeto e é revisado por todas as partes interessadas. As atividades de garantia de qualidade, realizadas pela equipe de engenharia de *software* e pelo grupo de SQA, são regidas pelo plano. O plano identifica avaliações a serem realizadas, auditorias e revisões a serem realizadas, normas que são aplicáveis ao projeto, procedimentos para o relato e acompanhamento de erros, documentos a serem produzidos pelo grupo de SQA e quantidade de realimentação fornecida à equipe de projeto do software.

Participa no desenvolvimento da descrição do processo do software do projeto.

A equipe de software seleciona um processo para o trabalho a ser realizado. O grupo de SQA revisa a descrição do processo para verificar a sofisticação da política empresarial, padrões internos de *software*, padrões externamente impostos (por exemplo, ISSO-9001) e outras partes do plano de projeto de *software*.

Revê as atividades de engenharia de software para verificar a satisfação do processo de *software* definido.

O grupo de SQA identifica, documenta e acompanha desvios do processo e verifica se correções foram feitas.

Audita os produtos do trabalho de *software* encomendado para verificar a satisfação do que foi definido como parte do processo de software.

O grupo de SQA revê os produtos selecionados do trabalho, identifica, na descrição do processo, nas normas aplicáveis ou nos produtos do trabalho técnico.

Garante que os desvios do trabalho de *software* e dos produtos

do trabalho sejam documentados e manipulados de acordo com um procedimento documentado.

Os desvios podem ser encontrados no plano de projeto, na descrição do processo, nas normas aplicáveis ou nos produtos do trabalho técnico.

Registra qualquer eventual não-satisfação e a relata à gerência superior.

Os itens que não atendem ao padrão são acompanhados até que sejam resolvidos.

Além dessas atividades, o grupo de SQA coordena o controle e a gestão das mudanças e ajuda a coletar e a analisar métricas de *software*.

Revisões de *Software*

As revisões de *software* são um "filtro" para o processo de engenharia de *software*. Isto é, as revisões são aplicadas em vários pontos durante a engenharia de *software* e servem para descobrir erros e defeitos que podem depois ser removidos. As revisões de *software* "purificam" as atividades de engenharia, que chamamos análise, projeto e codificação. Freedman e Weinberg (1990) discutem as necessidades de revisões do seguinte modo:

O trabalho precisa de revisões, pela mesma razão que lápis precisam de borrachas: errar é humano. A segunda razão pela qual precisamos de revisões técnicas é que, apesar de as pessoas serem boas para encontrar alguns dos seus próprios erros, várias classes de erros escapam mais facilmente aos que as originam do que escapam a outras pessoas.

Muitos tipos diferentes de revisões podem ser conduzidos como parte da engenharia de *software*. Cada qual tem seu lugar. Um encontro formal em torno de uma cafeteria é uma forma de revisão, se problemas técnicos são discutidos. Uma apresentação formal do projeto de *software* para uma audiência de clientes, gerência e pessoal técnico é também uma forma de revisão. Todavia, focalizamos a revisão técnica formal, algumas vezes chamada de "walkthrough" ou de inspeção.

Uma revisão técnica formal (*Formal Technical Review* - FTR) é o filtro mais efetivo do ponto de vista de garantia de qualidade. Conduzida por engenheiros de *software* (e outros) para engenheiros de *software*, a FTR é um meio efetivo de descobrir erros e aperfeiçoar a qualidade de *software*.

Impacto nos Custos de Defeitos de *Software*

O objetivo principal das revisões técnicas formais é achar erros durante o processo, de modo que eles não se transformem em defeitos depois da entrega do *software*. O benefício óbvio das revisões técnicas formais é a descoberta antecipada dos erros, de forma que eles não se propaguem para o passo seguinte de processo de *software*.

Vários estudos da indústria (pela TRW, *Nippon Eletric*, *Mitre Corp*, entre outros) indicam que as atividades de projeto introduzem entre 50 e 65% de todos os erros (e em última análise, todos os defeitos) durante o processo de *software*. Todavia, foi mostrado que as revisões técnicas formais são 75% efetivas [JON86] na descoberta de erros de projeto. Detectando ou removendo uma alta porcentagem desses erros, o processo de revisão reduz substancialmente os passos subsequentes nas fases de desenvolvimento e manutenção.

Para ilustrar o impacto do custo da detecção antecipada dos erros, consideramos uma série de custos relativos, que são baseados em dados reais de custos coletados para grandes projetos de *software* [IBM81]. Considere que um erro encontrado durante o projeto vai custar 1,0 (uma) unidade monetária para ser corrigido. Relativamente a esse custo, o mesmo erro, descoberto imediatamente antes do início do teste, vão custar 6,5 unidades; durante o teste, 1,5 unidades; e depois da entrega entre 60 e 100 unidades.

Amplificação e Remoção de Defeitos

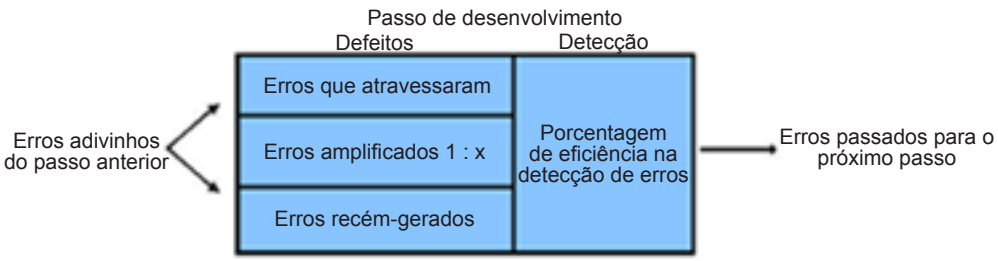
Um modelo de amplificação de defeitos [IBM81] pode ser usado para ilustrar a geração e a detecção de erros durante os passos de projeto preliminar, de projeto detalhado e de codificação do processo de engenharia de *software*.

O modelo é ilustrado esquematicamente na figura a seguir. Uma caixa representa um passo de desenvolvimento de *software*. Durante o passo, erros podem ser gerados inadvertidamente. A revisão pode deixar de descobrir erros recém-gerados e erros advindos de passos anteriores, resultando em certo número de erros que passaram de um passo para outro.

Em alguns casos, os erros que atravessavam vindos de passos anteriores, são amplificados (fator de amplificação, x) pelo trabalho atual. As subdivisões da caixa representam uma dessas características e uma

porcentagem de eficiência na detecção de erros, que é a função do rigor da revisão.

Modelo de amplificação de defeitos



Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

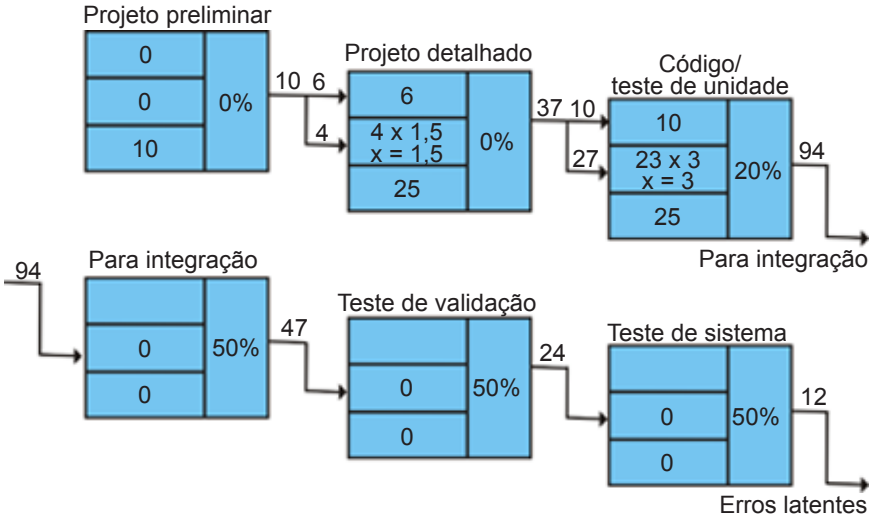
A figura a seguir (1.1) ilustra um exemplo hipotético de amplificação de defeitos para um processo de desenvolvimento de *software* no qual não são feitas revisões.

Com referência à figura, é considerado que cada passo do teste descobre e corrige 50% de todos os erros que chegam, sem introduzir nenhum erro novo (suposição otimista). Dez erros de projeto preliminar são amplificados para 94 erros, antes que o teste comece. Doze erros latentes vão para o campo.

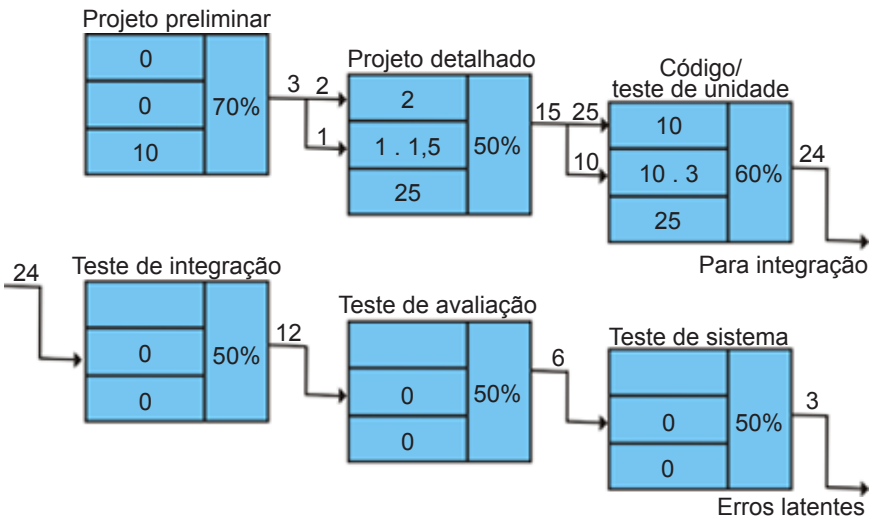
A Figura (1.2) considera as mesmas condições, exceto que revisões de projeto e códigos são conduzidas como parte de cada passo de desenvolvimento. Nesse caso, 10 erros iniciais de projeto preliminar são amplificados para 24 antes que o teste comece.

Apenas três erros latentes existem. Recordando os custos relativos, associados com a descoberta e a correção de erros, o custo total (com e sem revisões para o nosso exemplo hipotético) pode ser estabelecido. O número de erros descoberto durante cada um dos passos mostrados nas figuras 1.1 e 1.2 é multiplicado pelo custo de remoção de um erro (1,5 unidades de custo para projeto, 6,5 unidades de custo antes do teste, 15 unidades de custo durante o teste e 67 unidades de custo após a entrega).

Usando esses dados, o custo total para desenvolvimento e manutenção, quando são feitas as revisões, é de 783 unidades de custo. Quando não são feitas revisões, o custo é de 2.177 unidades – quase três vezes maior.



Fonte : Pressman, Roger S. Engenharia de Software / 2010.



Fonte : Pressman, Roger S. Engenharia de Software / 2010.

Para realizar as revisões, o engenheiro de *software* precisa investir tempo e esforço e a organização de desenvolvimento precisa gastar dinheiro. Todavia, os resultados do exemplo anterior deixam pouca dúvida de que podemos pagar agora ou pagar muito mais depois. As revisões técnicas formais (para o projeto e outras atividades técnicas) fornecem um custo-benefício demonstrável. Elas devem ser realizadas.

Revisões Técnicas Formais

Uma revisão técnica formal é uma atividade de garantia de

qualidade de *software* realizada por engenheiros de *software* (e outros).

Os objetivos das FTR são (1) descobrir erros na função, na lógica ou não implementação, para qualquer representação do *software*; (2) verificar se o *software* sob a revisão satisfaz seus requisitos; (3) garantir que o *software* tenha sido representado de acordo com padrões definidos; (4) conseguir *software* que seja desenvolvido de modo uniforme; e (5) tornar os projetos mais administráveis.

Além disso, a FTR serve como uma oportunidade de treinamento, permitindo a jovens engenheiros observar abordagens diferentes para análise, projeto e implementação de *software*. A FTR também serve para promover retaguarda e continuidade porque algumas pessoas ficam familiarizadas com parte do *software*, que de outra forma poderiam não chegar a ver.

A FTR é na verdade uma classe de revisões que inclui *wlakthroughs*, inspeções, revisões circulares ou "em rodízio" e outras avaliações técnicas de *software* feitas por pequenos grupos.

Cada FTR é conduzida como uma reunião e será bem-sucedida, apenas se for adequadamente planejada, controlada e assistida. Nas seções que seguem diretrizes semelhantes àquelas para um *walkthrough* (por exemplo, [FRE90], [GIL93]) são apresentadas como representativas de uma revisão técnica formal.

A Reunião da Revisão

Independentemente do formato de FTR escolhido, cada reunião de revisão deve atender às seguintes restrições:

- Entre três e cinco pessoas (em geral) devem ser envolvidas na revisão. Preparativos devem ser feitos, os quais não devem exigir mais de duas horas de trabalho de cada pessoa;
- A duração da reunião de revisão deve ser inferior a duas horas;
- À vista dessas restrições, fica óbvio que uma FTR focalizada uma parte específica (e pequena) de todo o *software*. Por exemplo, em vez de tentar revisar todo um projeto, são conduzidos os *walkthroughs* de cada componente ou pequeno grupo de componentes. Restringindo o foco, a FTR tem maior probabilidade de descobrir erros.

O foco de uma FTR é o produto de trabalho (por exemplo, uma parte

da especificação de requisitos, o projeto detalhado de um componente, uma listagem do código-fonte de um componente). O indivíduo que desenvolveu o produto de trabalho – o produtor – informa ao líder do projeto que o produto do trabalho foi completado e que é necessária uma revisão. O líder do projeto contata um líder de revisão que avalia se o produto está efetivamente pronto, gera cópias dos materiais do produto e as distribui a dois ou três revisores para preparativos antecipados.

Espera-se que cada revisor gaste entre uma e duas horas revisando o produto, tomando notas e ganhando familiaridade com o trabalho. Concomitantemente, o líder de revisão também revisa o produto e estabelece uma agenda para a reunião de revisão, que é usualmente marcada para o dia seguinte.

A reunião de revisão tem a participação do líder de revisão, de todos os revisores e do produtor. Um dos revisores assume o papel de registrador, isto é, o indivíduo que registra (por escrito) todas as questões importantes levantadas durante a revisão.

A FTR começa com a introdução da agenda e uma breve introdução pelo produtor. O produtor então prossegue, "percorrendo" ("*walk through*") o produto de trabalho e explicando o material, enquanto os revisores levantam questões baseadas na sua preparação antecipada. Quando problemas ou erros válidos são descobertos, o registrador os anota.

No fim da revisão, todos os participantes da FTR devem decidir se: (1) aceitam o produto sem maiores modificações; (2) rejeitam o produto devido a erros graves (ao ser corrigido, outra revisão deve ser realizada); (3) aceitam o produto condicionalmente (erros triviais foram encontrados e devem ser corrigidos, mas não será nova revisão).

Tomada a decisão, todos os participantes da FTR assinam uma lista na qual indicam sua participação na revisão e sua concordância com os resultados da equipe de revisão.

Relatório e Manutenção de Registros das Revisões

Durante a FTR, um revisor (o registrador) registra ativamente todos os tópicos que foram levantados. Estes são resumidos no fim da reunião de revisão e uma lista de tópicos de revisão é produzida. Além disso, um relatório resumido da revisão técnica formal é completo. Um relatório resumido da revisão responde a três questões:

- O que foi revisado?
- Quem fez a revisão?
- Quais foram as descobertas e conclusões?

O relatório resumido da revisão é um formulário de uma única página (com possíveis anexos). Ele se torna parte do registro histórico do projeto e pode ser distribuído ao líder do projeto e a outras partes interessadas.

A lista de questões da revisão serve a duas finalidades: (1) identificar áreas problemáticas no produto e (2) servir como *checklist* de itens de ação, que orienta o produtor à medida que as correções são efetuadas. Uma lista de tópicos é normalmente anexada ao relatório sumário.

É importante estabelecer um procedimento de acompanhamento para garantir que os itens na lista de tópicos tenham sido adequadamente corrigidos. A menos que isso seja feito, é possível que as questões levantadas possam "escapar pelas frestas". Uma abordagem é atribuir a responsabilidade do acompanhamento ao líder da revisão.

Diretrizes de Revisão

Diretrizes para a condução de revisões técnicas formais devem ser estabelecidas previamente, distribuída a todos os revisores, receber a concordância de todos e depois ser seguidas. Uma revisão fora de controle pode frequentemente ser pior do que nenhuma revisão. A seguir, um conjunto mínimo de diretrizes para revisões técnicas formais:

1. Revise o produto, não o produtor. Uma FTR envolve pessoas e egos. Conduzida adequadamente, a FTR pode assumir a aura de inquisição. Os erros devem ser mostrados gentilmente; o tom da reunião deve ser descontraído e construtivo; o objetivo não deve ser embaraçado ou perturbado;
2. Estabeleça uma agenda e a siga. Uma das doenças-chave das reuniões de todos os tipos é a derivação. Uma FTR deve ser mantida na linha e no cronograma. O líder de revisão é investido da responsabilidade de manter o cronograma da reunião e não deve temer repreender as pessoas quando a reunião fica "à deriva";
3. Limite o debate e a réplica. Quando um tópico é levantado por

um revisor, pode não haver concordância de todos quanto ao seu impacto. Em vez de gastar tempo debatendo a questão, o tópico deve ser registrado para posterior discussão fora da reunião;

4. Enuncie as áreas problemáticas, mas não tente resolver todo problema anotado. Uma revisão não é uma seção de solução de problemas;
5. Tome nota por escrito. Algumas vezes é uma boa ideia para o registrador tomar nota em um quadro de parede, de modo que a redação e as prioridades possam ser avaliadas pelos outros revisores à medida que a informação é registrada;
6. Limite o número de participantes e insista nos preparativos antecipados. Duas cabeças são melhores que uma, mas 14 não são necessariamente melhores do que 4. Mantenha o número de pessoas envolvidas no mínimo necessário. Todavia, todos os membros da equipe de revisão devem se preparar antecipadamente. Comentários escritos devem ser solicitados pelo líder de revisão (indicando que o revisor reviu o material);
7. Desenvolva um *checklist* para cada produto que será provavelmente revisada. Um *checklist* ajuda o líder de revisão a estruturar uma reunião FTR e ajuda cada revisor a se concentrar em tópicos importantes;
8. Aloque recursos e programe tempo para as FTR. Para que as revisões sejam efetivas, elas devem ter cronogramas como uma tarefa durante o processo de engenharia de *software*. Além disso, deve ser reservado tempo para as modificações inevitáveis que irão ocorrer como resultado de uma FTR;
9. Faça treinamento significativo para todos os revisores. Para serem efetivos, todos os participantes de uma revisão devem saber algum treinamento formal. O treinamento deve enfatizar tanto os assuntos relativos ao processo quanto o lado humano, psicológico das revisões;
10. Reveja suas primeiras revisões. A análise de uma reunião de revisão pode ser benéfica para a descoberta de problemas no próprio processo de revisão. O primeiro produto a ser revisado deve ser as próprias diretrizes de revisão.

Com muitas variáveis (por exemplo, número de participantes, tipo de produtos de trabalho, horário e duração, abordagem específica de

revisão) tem impacto sobre uma revisão bem-sucedida, uma organização de *software* deve experimentar para determinar qual abordagem funciona melhor em um contexto local. Porter e seus colegas (1995) fornecem essa lente diretriz para esse tipo de experimentação.

Revisões Guiadas por Amostras

Em um cenário ideal todo produto do trabalho de engenharia de *software* deveria passar por uma revisão técnica formal. Nos projetos de *software* do mundo real, recursos são limitados e o prazo é curto. Como consequência, revisões são frequentemente puladas, mesmo que seu valor seja reconhecido como um mecanismo de controle de qualidade. Thelin e seus colegas (2001) tratam esse tópico quando afirmam:

Inspeções [FTRs] são somente vista como eficientes se muitas falhas forem encontradas durante a parte de busca por falha. Se muitas falhas são encontradas nos artefatos [produtos de trabalho], as inspeções são necessárias. Se, por outro lado, somente poucas falhas são encontradas, a inspeção foi um desperdício de tempo para muitas pessoas nela envolvida.

Além disso, projetos de *software* que estão atrasados frequentemente diminuem o tempo da atividade de inspeção, o que leva à falta de qualidade. Uma solução seria estabelecer prioridades para o uso de recursos destinados às atividades de inspeção e, consequentemente, concentrar os recursos disponíveis nos artefatos que são mais propensos a erros.

Thelin e seus colegas(2001) sugerem um processo de revisão baseado em amostra nas quais todos os produtos de trabalho de engenharia de *software* são inspecionadas para determinar quais produtos de trabalho são mais propensos a erro.

Recursos FTR completos são enfocados somente naqueles produtos de trabalho que são provavelmente (baseados em dados coletados durante a amostragem) propensos a erro.

Para ser efetivo, o processo de revisão guiado por amostras deve tentar quantificar aqueles produtos de trabalho que são alvos principais para FTRs completas. Para conseguir isso os seguintes passos são sugeridos (THELIN et al, 2001):

- Inspecione a fração de cada produto de trabalho de *software*, i. Registre o número de falhas f encontrado em a;

- Desenvolva uma estimativa grosseira do número de falhas em um produto de trabalho i pela multiplicação de f por 1/a;
- Ordene os produtos de trabalho de ordem descendente de acordo com a estimativa grosseira do número estimado de falhas;
- Enfoque os recursos de revisão disponíveis naqueles produtos de trabalho que têm o maior número estimado de falhas;

A fração dos produtos de trabalho que é amostrada deve (1) ser representativa do produto de trabalho como um todo e (2) suficientemente grande para ser significativa para o(s) revisor (es) que faz(em) a amostragem.

À medida que a aumenta, a probabilidade de que a amostra seja uma representação válida do produto de trabalho também aumenta. No entanto, os recursos necessários para fazer a amostragem também aumentam. Uma equipe de engenharia de *software* deve estabelecer o melhor valor de a para todos os tipos de produtos de trabalho produzidos.

Abordagens Formais para SQA

Durante as duas últimas décadas, um pequeno mais ativo segmento da comunidade de engenharia de *software* tem argumentado que uma abordagem mais formal para a garantia de qualidade é necessária. Pode-se alegar que um programa de computador é um objeto matemático [SOM96].

Uma sintaxe e semânticas rigorosas podem ser definidas para cada linguagem de programação e uma abordagem rigorosa para a especificação de requisitos de *software* está disponível. Se o modelo de requisitos (especificação) e uma linguagem de programação podem ser representados de um modo rigoroso, deve ser possível aplicar prova matemática de correção para demonstrar que um programa satisfaz exatamente as suas especificações.

Garantia Estatística de Qualidade de *Software*

Garantia estatística de qualidade reflete uma tendência crescente em toda a indústria, de tornar-se mais quantitativa a respeito da qualidade. Para o *software*, a garantia estatística de qualidade implica os seguintes passos:

1. Informação sobre defeitos de *software* é coletada e categorizada;
2. Uma tentativa de rastrear cada defeito até sua casa adjacente (por exemplo, não-conformidade com as especificações, erro de projeto, violação de normas, pouca comunicação com o cliente) é feita;
3. Usando o princípio de Pareto (80% dos defeitos podem ser rastreados até 20% de todas as causas possíveis), isole os 20% (os "pouco vitais");
4. Uma vez identificadas as poucas causas vitais, trate de corrigir os problemas que causaram os defeitos.

Esse conceito relativamente simples representa um importante passo em direção à criação de um processo adaptativo de *software* no qual são feitas modificações para aperfeiçoar os elementos do processo que introduzem erro.

Um Exemplo Genérico

Para ilustrar o uso de métodos estatísticos no trabalho de engenharia de *software*, considere que uma organização de engenharia de *software* coleta informações sobre defeitos durante o período de um ano. Alguns defeitos são descobertos à medida que o *software* é desenvolvido.

Outros são encontrados depois que o *software* foi entregue a seus usuários finais. Apesar de centenas de diferentes erros serem descobertos, todos podem ser rastreados até uma (ou mais) das seguintes causas:

- Especificações incompletas ou errôneas (*Incomplete or Erroneous Specifications* – IES);
- Má interpretação da comunicação com o cliente (*Misinterpretation of Customer Communication* – MCC);
- Desvio intencional das especificações (*Intentional Desviation from Specifications* – IDS);
- Violação das normas de programação (*Violation of Programming Standards* – VPS);
- Erro na representação dos dados (*Error in Data Representation* – IDR);

- Interface de componente inconsistente (*Inconsistent Component Interface* – ICI)
- Erro na lógica do projeto (*Error in Design Logic* – EDL);
- Teste incompleto ou errôneo (*Incomplete or Erroneous Testing* – IET);
- Documentação imprecisa ou incompleta (*Innacurate or Incomplete Documentation* – IID);
- Erro na tradução do projeto para a linguagem de programação (*Error in Programming Language Translation Design* – PLT);
- Interface homem-computador ambígua ou inconsistente (*Ambinguous of inconsistente Human/Computer Interface* – HCI);
- Miscelânia (*Miscellaneous*, MIS).

Para aplicar SQA estatística, a tabela a seguir foi construída. A tabela indica que IES, MCC e EDR são as poucas causas vitais que correspondem a 53% e todos os erros. Deve-se notar, todavia, que IES, EDR, PLE e EDL seriam selecionadas como as poucas causas vitais, se apenas fossem considerados erros sérios.

Uma vez determinada as poucas causas vitais, a organização de engenharia de *software* pode iniciar uma ação corretiva. Por exemplo, para corrigir MCC, o desenvolvedor de *software* poderia implementar técnicas facilitadas de coleta de requisitos para aperfeiçoar a qualidade das especificações e da comunicação com o cliente.

Para aperfeiçoar EDR, o desenvolvedor poderia adquirir ferramentas para a modelagem de dados e realizar revisões mais estritas do projeto dos dados.

É importante notar que a ação corretiva focalizada principalmente em poucas causas vitais. À medida que as poucas causas vitais são corrigidas, novas candidatas despontam o topo da pilha.

Tem sido mostrado que as técnicas de garantia estatística de qualidade de *software* propiciam substancias aperfeiçoamentos a qualidade [ART97]. Em alguns casos, organizações de *software* conseguiram uma redução de 50% dos defeitos por ano, depois de aplicar essas técnicas.

A aplicação de SQA estatística e do princípio de Pareto podem ser resumidos em uma única sentença: gaste seu tempo focalizando as coisas que realmente importam, mas primeiro esteja certo de que você compreende o que realmente importa!

	Total		Grave		Moderado		Trivial	
Erro	Quantidade	%	Quantidade	%	Quantidade	%	Quantidade	%
IES	205	22%	34	27%	68	18%	103	24%
MCC	156	17%	12	9%	68	18%	76	17%
IDS	48	5%	1	1%	24	6%	23	5%
VPS	130	3%	0	0%	15	4%	10	2%
EDR	58	14%	26	20%	68	18%	36	8%
ICI	45	6%	9	7%	18	5%	31	7%
EDL	95	5%	14	11%	12	3%	19	4%
IET	36	10%	12	9%	35	9%	48	11%
IID	60	4%	2	2%	20	5%	14	3%
PLT	28	6%	15	12%	19	5%	26	6%
HCI	56	3%	3	2%	1	4%	8	2%
MIS	942	6%	0	0%	15	4%	41	9%
Totais		100%	128	100%	379	100%	435	100%

Fonte : Pressman, Roger S. Engenharia de Software / 2010.

Seis Sigma para Engenharia de *Software*

Seis Sigma (*Six Sigma*) é a estratégia mais amplamente usada para garantia de qualidade estatística na indústria hoje em dia. Originalmente, popularizada pela Motorola na década de 1980, a estratégia Seis Sigma "é uma metodologia rigorosa e disciplinada que usam dados e análise estatística para medir e aperfeiçoar o desempenho operacional de uma empresa pela identificação e eliminação de 'defeitos' nos processos de fabricação e relacionados a serviço".

O termo "seis sigma" é derivado de seis desvios é derivado de seis desvios padrão – instâncias (defeitos) por milhão de ocorrências – implicando uma norma de qualidade extremamente alta. A metodologia Seis Sigma define três passos centrais:

- Defina os requisitos do cliente, os artefatos passíveis de entrega e os objetivos do projeto por meio de métodos bem definidos de comunicação com o cliente;
- Meça o processo existente e sua saída para determinar as poucas causas vitais. Se um processo de *software* existente está em ação, mas é necessário aperfeiçoamento, Seis Sigmas sugerem dois passos adicionais:
- Aperfeiçoe o processo pela eliminação das causas básicas dos defeitos;

- Controle o processo para garantir que o trabalho futuro não reintroduza as causas dos defeitos.

Esses passos centrais e adicionais são algumas vezes referidos como o método DMAIC (Defina [*Define*], Meça [*Measure*], Melhore [*Improve*] e Controle [*Control*].

Se uma organização estiver desenvolvendo um processo de *software* (em vez de aperfeiçoando um processo existente), os passos centrais são ampliados como se segue:

- Projete o processo para (1) evitar as causas básicas de defeitos e (2) para satisfazer os requisitos do cliente;
- Verifique se o modelo de processo vai, de fato, evitar os defeitos e satisfizer os requisitos do cliente.

Essa variação é algumas vezes chamada de método DMADV (Defina [*Define*], Meça [*Measure*], Analise [*Analyze*], Projete [*Design*] e Verifique [*Verify*]).

Confiabilidade de *Software*

Confiabilidade de *software*, ao contrário de muitos outros fatores de qualidade, pode ser medida diretamente e estimada usando dados históricos e de desenvolvimento. Confiabilidade de *software* é definida em termos estatísticos como "a probabilidade de operação livre de falhas de um programa de computador, em um ambiente especificado, durante um tempo especificado" [MUS87]. Para ilustrar, é estimado que o programa X tenha uma confiabilidade de 0,96, por oito horas corridas de processamento. Em outras palavras, se o programa X tiver que ser executado 100 vezes e exigir oito horas de tempo corrido de processamento (tempo de execução), é provável que funcione corretamente (sem falha) 96 das 100 vezes.

Sempre que a confiabilidade de *software* é discutida, surge uma questão-chave: O que se quer dizer com o termo falha? No contexto de qualquer discussão de qualidade e confiabilidade de *software*, falha é a não-conformidade com os requisitos do *software*. Todavia, mesmo dentro dessa definição, há graduações. Falhas podem ser apenas incômodas ou catastróficas. Uma falha pode ser corrigida em segundos, enquanto outra exige semanas ou meses para ser corrigida. Complicando ainda

mais o assunto, a correção de uma falha pode, na verdade, resultar na introdução de outros erros, que em última análise resultarão em outras falhas.

Medidas de Confiabilidade e Disponibilidade

Os primeiros trabalhos em confiabilidade de *software* tentaram extrapolar a matemática da teoria da confiabilidade do *hardware* (por exemplo, [ALV64]) para a previsão da confiabilidade de *software*. A maioria dos modelos de confiabilidade relacionados a *hardware* é voltada a falas devidas a desgaste físico (por exemplo: os efeitos de temperatura, de corrosão e de choque) são mais prováveis do que uma falha relacionada a projeto. Infelizmente, o oposto é verdadeiro para o *software*. De fato, todas as falhas de *software* podem ser rastreadas até problemas de projeto ou implementação; desgaste não entra nesse quadro.

Tem havido debate sobre o relacionamento entre conceitos-chave de confiabilidade de *hardware* e sua aplicação ao *software* (por exemplo, [LIT89], [ROO90]). Apesar de ainda ser estabelecida uma ligação irrefutável, vale a pena considerar alguns conceitos simples que se aplicam a ambos os elementos do sistema.

Se considerarmos um sistema baseado em computador, uma medida simples de confiabilidade é o tempo médio entre falhas(*Mean-Time-Between-Failure* – MTBF) em que:

MTBF = MTTF + MTTR

As siglas MTTF e MTTR significam tempo médio até a falha (*Mean-Time-To-Failure*) e tempo médio de reparo (*Mean-Time-To-Repair*), respectivamente.

Muitos pesquisadores argumentam que MTBF é uma métrica muito mais útil do que defeitos/KLOC ou defeitos/FP. Dito de maneira simples, o usuário final está preocupado com falhas, não com a contagem total de erros. Como cada erro contido em um programa não tem a mesma taxa de falha, a contagem total de erros fornece pouca indicação da confiabilidade de um sistema.

Em adição a uma medida de confiabilidade, visamos desenvolver uma medida de disponibilidade. Disponibilidade de *software* é a probabilidade de que um programa esteja operando de acordo com os requisitos em um dado momento, e é definida como:

Disponibilidade = [MTTF/ (MTTF + MTTR)] x 100%

A medida de confiabilidade MTBF é sensível igualmente a MTTF e MTTR. A medida de disponibilidade é de alguma forma mais sensível a MTTR, que é uma medida indireta da manutenibilidade de *software*.

Segurança de *Software*

Segurança de *software* [LEV86] é uma atividade de garantia da qualidade deste que focaliza a identificação e a avaliação de riscos potenciais, que podem afetar o *software* negativamente e causar a falha de todo o sistema. Se esses riscos puderem ser identificados cedo, no processo de *software*, podem ser especificadas características do projeto de software que eliminem ou controlem esses riscos em potencial.

Um processo de modelagem e análise é conduzido como parte da segurança de *software*. Inicialmente, os riscos são identificados e categorizados por críticas e risco. Por exemplo, alguns dos riscos associados com um controle de cruzeiro, para um automóvel baseado em computador, poderiam ser:

- Causa aceleração descontrolada que não pode ser interrompida;
- Não reage ao acionamento do pedal do freio (desligando);
- Não liga quando a chave é ativada;
- Lentamente perde ou ganha velocidade.

Uma vez identificados esses riscos do sistema, são usadas técnicas de análise para atribuir severidade e probabilidade de ocorrência. Para ser eficaz, o *software* precisa ser analisado no contexto de todo o sistema. Por exemplo, um erro sutil de entrada do usuário (pessoas são componentes do sistema) pode ser ampliado por uma falha de *software* para produzir dados de controle que posicionam inadequadamente um dispositivo mecânico.

Se um conjunto de condições ambientais externas ocorre (e somente se elas ocorrem), a posição inadequada do dispositivo mecânico vai causar uma falha desastrosa. Técnicas de análise como falhas de análise em árvore [VES81], lógica de tempo real [JAN86] ou modelos de rede de Petri [LEV86] podem ser usadas para prever a cadeia de eventos que pode causar riscos e a probabilidade de cada evento ocorrer para

criar a cadeia.

Uma vez identificados e analisados os riscos, requisitos relacionados à segurança podem ser especificados para o *software*. Isto é, a especificação pode conter uma de eventos indesejáveis e as respostas desejadas do sistema a esses eventos. O papel do *software* na gestão de eventos indesejáveis é tão indicado.

Apesar de a confiabilidade de *software* e da segurança de *software* ser intimamente relacionadas entre si, é importante entender a diferença sutil entre elas.

Confiabilidade de *software* usa análise estatística para determinar a probabilidade de que uma falha ocorra. Todavia, a ocorrência de uma falha não necessariamente resulta num risco ou infortúnio. A segurança examina os modos pelos quais as falhas resultam em condições que podem levar ao infortúnio. Isto é, as falhas não são consideradas em um vazio, mas são avaliadas no contexto de todo o sistema baseado em computador e seu ambiente.

As Normas de Qualidade ISSO 9000

Um sistema de garantia da qualidade pode ser definido como a estrutura organizacional, responsabilidades, procedimentos, processos e recursos para implementar a gestão de qualidade [ABNT00]. Sistemas de garantia de qualidade são criados para ajudar organizações a garantir que seus produtos e serviços satisfaçam as expectativas do cliente, estando de acordo com suas especificações.

ISSO 9000 descreve os elementos de garantia em termos genéricos, que podem ser aplicados a qualquer negócio independentemente dos produtos ou serviços oferecidos.

Para registrar em um dos modelos de sistema de garantia de qualidade contidos na ISSO 9000, o sistema e operações de qualidade da empresa são escrutinados por auditores externos para verificação de atendimento das normas e da efetividade da operação.

Após o registro bem-sucedido, a empresa recebe um certificado de um corpo de registro representado pelos auditores. Auditorias de vigilância semestrais garantem a continuidade de conformidade com as normas.

Os requisitos delineados pela ISSO 9001:2000 tratam de tópicos, tais como responsabilidade de gestão, sistema de qualidade, revisão de contrato, controle de projeto, controle de documento e dados, identificação

de produtos e rastreabilidade, controle de processo, inspeção de teste, ação corretiva e preventiva, registros de controle de qualidade, auditoriais de qualidade interna, treinamento, serviço e técnicas estatísticas. Para uma organização de *software* tornar-se certificada pela ISSO 9901:2000, ela deve estabelecer políticas e procedimentos para tratar cada um dos requisitos mencionados (e outros) e depois ser capaz de demonstrar que essas políticas e procedimentos estão sendo seguidos. Para mais informações sobre ISSO 9001, consultar [ABNT2000].

A Norma ISSO 9001:2000

Aseguinte diretriz define os elementos básicos da norma ISSO 9001:2000. Informações abrangentes sobre a normas pode ser obtida de international organizatioon for stardadization (www.iso.ch) e outras fontes da internet (por exemplo, www.praxiom.com).
Estabelecer os elementos de um sistema de gestão de qualidade.
Desenvolver, implementar e aperfeiçoar o sistema.
Definir uma politica que enfatiza a importância do sitema.
Documentar o sistema de qualidade.
Descrever o processo.
Produzir um manual operacional.
Desenvolver métodos para controlar (atualizar) documentos.
Estabelecer métodos para manter registros.
Para atividades técnicas (por exemplo, analise, projeto e teste).
Para monitoramento e gestão de projeto.
Definir métodos para reparação.
Apoiar o controle e a garantia de qualidade.
Promover a importância da qualidade entre todos os interessados.
Enfocar a satisfação do cliente.
Definir um plano de qualidade que entenda os objetivos, as responsabilidades e a autoridade.
Definir mecanismos de comunicação entre os interessados.
Estabelecer mecanismos de revisão para sistema de gestão de qualidade.
Identificar métodos de revisão e mecanismos de realimentação.
Definir procedimentos de acompanhamento.
Identificar recursos de qualidade incluindo pessoal.
Treinamento e elementos de infra-estrutura.
Estabelecer mecanismos de controle.
Para planejamento.
Para requisitos do cliente
Avaliar dados e métricas de qualidade.
Definir abordagens para aperfeiçoamento continuo de processo de qualidade.

Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Os Planos de SQA

O plano de SQA fornece um roteiro para a instituição da garantia de *software*. Desenvolvido pelo grupo de SQA (ou equipe de *software* se um grupo de SQA não existir), o plano serve como gabarito para as atividades de SQA que são instituídas para cada projeto de *software*.

Uma norma para planos de SQA foi recomendada pelo IEEE [IEE94]. A norma recomenda uma estrutura que identifique (1) o objetivo e o escopo do plano; (2) uma descrição de todos os produtos de trabalho de engenharia de *software* (por exemplo, modelos, documentos, código-fonte) que ficam no âmbito da SQA; (3) todas as normas e práticas aplicáveis que são aplicadas durante o processo de *software*; (4) ações e tarefas de SQA (inclusive audições e auditoriais) e sua colocação ao longo do processo de *software*; (5) as ferramentas e os métodos que apoiam as ações e tarefas de SQA; (6) procedimento de gestão de configuração de *software* para gestão de modificação; (7) métodos para montagem, proteção e manutenção de todos os registros relacionados a SQA; (8) papéis e responsabilidades organizacionais relativos à qualidade do produto.

Gestão de Qualidade de *Software*

Objetivo: o objetivo das ferramentas de SQA é apoiar uma equipe de projeto de avaliação e no aperfeiçoamento da qualidade de produto de trabalho de *software*.

Mecânica: a mecânica das ferramentas varia. Em geral, o objetivo é avaliar a qualidade de um produto de trabalho especifico. Obs.: uma vasta game de ferramentas de teste de *software* é frequentemente incluída na categoria de ferramentas de SQA.

Ferramentas representantes:

ARM, desenvolvida pela NASA (**satc.gscf.nasa.org/tools/index.html**), fornecem medidas que podem ser usadas para avaliar a qualidade de um documento de requisitos de *software*.

QPR ProcessGuide and Scorecard, desenvolvida por QPRSoftware (**www.qpronline.com**), fornece um apoio para Seis Sigmas e outras abordagens de gestão de qualidade;

Quality Tools Cookbook, desenvolvida pela Systma e Manle (**www.systma.com/tqmtools/tqmtoolmenu.html**), fornece descrições uteis de ferramentas de gestão clássica de qualidade tal como diagramas de controle, diagramas de espalhamento, diagrama de afinidade e diagramas matriciais.

Quality Tools asn Templates, desenvolvida por iSixSigma (**http://www.isixsigma.com/tt/**),
descreve uma ampla gama de ferramentas e métodos uteis para a gestão de qualidade.]

TQM Toolsm, desenvolvida por Bain & Company (**www.bain.com**), fornece discrições uteis de uma variedade de ferramentas de gestão usadas para TQM e métodos de gestão de qualidade relacionados.

Fonte : Pressman, Roger S. Engenharia de *Software* / 2010.

Exercícios Propostos



1. Pesquise sobre Qualidade de projeto e Qualidade de conformidade.
2. Qual a importância do Controle de qualidade?
3. Qual a importância das inspeções, revisões e testes usados ao longo do processo de *software*?
4. Determine o Impacto nos Custos de Defeitos de *Software*.

Saiba Mais



VON Staa, Arndt. Engenharia de Programas. LTC. Capitulo 2 – Qualidade de Software, 1983.

http://www.mswconsult.com.br/qualidade.html

http://internativa.com.br/artigo_software.html

Para Pensar

As revisões técnicas formais é achar erros durante o processo, de modo que eles não se transformem em defeitos depois da entrega do *software*.

Resumindo



O capítulo mostrou a definição de qualidade, sendo especificada Qualidade de projeto e Qualidade de conformidade. Qualidade de projeto refere-se a características que os projetistas especificam para certo item. Qualidade de conformidade é o grau com que as especificações de projeto

são seguidas durante a fabricação. Controle de qualidade que envolve a série de inspeções, revisões e testes usados ao longo do processo de *software* para garantir que cada produto de trabalho satisfaça os requisitos para ele estabelecidos.

A combinação de medida e realimentação permite ajustar o processo quando os produtos de trabalho criados deixam de satisfazer suas especificações. Garantia da qualidade consiste de um conjunto de funções para auditar e relatar que avalia a efetividade e completeza das atividades do controle de qualidade.

Sua meta de garantia de qualidade é fornecer à gerência os dados necessários para que fique informada sobre a qualidade do produto, ganhando assim compreensão e confiança de que a qualidade do produto está satisfazendo suas metas. O custo da qualidade inclui todos os custos decorrentes da busca da qualidade ou da execução das atividades relacionadas à qualidade.

Por fim, o capítulo mostrou as revisões e o Impacto nos Custos de Defeitos de *Software*.



Referências

PRESSMAN, Roger S. **Engenharia de Software** / Roger S. Pressman ; tradução Rosângela Ap. D. Penteado. - 6. ed. - Porto Alegre : AMGH. 2010.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **Sistemas de gestão da qualidade – fundamentos e vocabulário: NBR ISO 9000**. Rio de Janeiro, 2000.

B. Booch, J. Rumbaugh, I. Jacobson. **UM guia do usuário.** Campus, 12ª edição, 2000.

BERTOLLO, Gleidson. Dissertação de Mestrado. **Definição de processos em um ambiente de desenvolvimento de software**. UFES, 2006.

FEIGENBAUM, E. A. and FELDMAN, J. (eds.) (1963) **"Computers and**

Thought". *McGraw-Hill***,** 1985.

GOLDBARG, M. C. **Qualidade Substantiva**. São Paulo: BookMark, 2000.

MCCALL, J.; RICHARDS, P. e WALTERS, G**. Factors in Software Quality**_NTIS AD-AO49-014,015,055. Nov.1977.

[LEV86] Leveson, N.G., **"Software Safety: Why, What, and How,"** ACM Computing Surveys, vol. 18, no. 2, June 1986, pp. 125–163.

OSBORN, Sylvia L. **Towards a universal relation interface**. In: Fifth International Conference on Very Large Data Bases, 1979.

[IEE94] Software Engineering Standards, 1994 edition, IEEE, 1994.

[JAN86] Jahanian, F. and A.K. Mok, "Safety Analysis of Timing Properties of Real-Time Systems", IEEE Trans. Software Engineering, vol. SE-12, no. 9, September 1986, pp. 890–904.

OSTERWEIL, L. **Directions in Software Quality**. ACM Computing Surveys, vol 28, n. 4, 739-750, 1996,

PÁDUA FILHO, Wilson. **Engenharia de software: fundamentos, Métodos e Padrões. LTC**. Capítulo 5: Processos de Software, 2009.

PFLEEGER, S.L. **Engenharia de Software: Teoria e Prática.** São Paulo: Prentice Hall, 2ª edição, 2004.

PRESSMAN, Roger. **Engenharia de software.** McGraw Hill. Capítulo 3 – Modelos Prescritivos de Processo; Capítulo 4 – Desenvolvimento Ágil, 2006.

ROCHA, A. R. C., J. C. MALDONADO, K. C. Weber, **Qualidade de Software: Teoria e Prática**, São Paulo: Prentice Hall, 2001.

SOMMERVILLE, Ian. **Engenharia de software**. Pearson Education. Capítulo 4 – Processo de Software, 2006.

VON Staa, Arndt. **Engenharia de Programas**. LTC. Capitulo 2 – Qualidade

de Software, 1983.

TOFFLER, Alvin. P**owershift: Knowledge, Wealth, and Violence at the Edge of the 21st Century**. Bantam, 1990.

Material variado sobre Engenharia de Software - Blog do Prof. Jair C Leite, UFRN.Disponível em http://engenhariadesoftware.blogspot.com/2007_03_01_archive.html; Acesso em: 15/02/2011.

[LEV95] Levy, S., **"The Luddites Are Back", Newsweek,** p. 55, 12 jun. 1995.

[KAR99] Karlson, E. and J. Kolber, **A Basic Introduction to Y2K: How the Year 2000 Computer Crisis Affects YOU**, Next Era Publications, 1999.

LIPPMAN, Andrew. **O arquiteto do futuro. Meio & Mensagem,** São Paulo, n. 792, 26 jan.1998. Entrevista. LIPPMAN, Andrew. O arquiteto do futuro. Meio & Mensagem, São Paulo, n. 792, 26 jan. 1998. Entrevista

LIU YF, Zan LS, Li K, Zhao SP, Xin YP, Lin Q, Tian WQ, Wang ZW 1998). **A novel polymorphism of GDF5 gene and its association with body measurement traits in Bos taurus and Bos indicus breeds.** Mol.Biol. Rep. 37:429-434.

[PAU93] Paulk, M. et al., **"Capability Maturity Model for Software," Software Engineering Institute,** Carnegie Mellon University, Pittsburgh, PA, 1993.

[LIT89] Littlewood, B., **"Forecasting Software Reliability," in Software Reliability: Modeling and Identification**, (S. Bittanti, ed.), Springer-Verlag, 1989, pp. 141–209.

[ROO90] Rook, J., **Software Reliability Handbook, Elsevier**, 1990.

[ALV64] Alvin, W.H. von (ed.), **Reliability Engineering, Prentice-Hal**l, 1964.

[MUS87] Musa, J.D., A. Iannino, and K. Okumoto, **Engineering and**

Managing Software with Reliability Measures, McGraw-Hill, 1987.

[ART97] Arthur, L.J., **"Quantum Improvements in Software System Quality",** CACM, vol. 40, no. 6, June 1997, pp. 47–52.

[SOM96] Somerville, I., **Software Engineering,** 5th ed., Addison-Wesley, 1996.

[FRE90] Freedman, D.P. and G.M. Weinberg, H**andbook of Walkthroughs, Inspections and Technical Reviews**, 3rd ed., Dorset House, 1990.

[GIL93] Gilb, T. and D. Graham, S**oftware Inspections,** Addison-Wesley, 1993.

[IBM81] "Implementing Software Inspections," course notes, IBM Systems Sciences Institute, IBM Corporation, 1981.

[JON86] Jones, T.C., Programming Productivity, McGraw-Hill, 1986.

[SCH98] Schulmeyer, G.C. and J.I. McManus (eds.), Handbook of Software Quality Assurance, 3rd ed., Prentice-Hall, 1998.

[IEE94] Software Engineering Standards, 1994 ed., IEEE Computer Society, 1994.

[DEM99] DeMarco, T., "Management Can Make Quality (Im)possible," Cutter IT Summit, Boston, April 1999.

[PAR96] Park, R.E., W.B. Goethert, and W.A. Florac, Goal Driven Software Measurement - A Guidebook, CMU/SEI-96-BH-002, Software Engineering Institute, Carnegie Mellon University, August 1996.

[CHR95] Christensen, S.R., "Software Reuse Initiatives at Lockheed," CrossTalk, vol.8, no. 5, May 1995, pp. 26–31.

[HEN95] Henry, E. and B. Faller, "Large Scale Industrial Reuse to Reduce Cost and Cycle Time," IEEE Software, September 1995, pp. 47–53.

[MCM95] McMahon, P.E., "Pattern-Based Architecture: Bridging Software

Reuse and Cost Management," Crosstalk, vol. 8, no. 3, March 1995, pp. 10–16.

[CAV78] Cavano, J.P. and J.A. McCall, "A Framework for the Measurement of Software Quality," Proc. ACM Software Quality Assurance Workshop, November 1978, pp.133–139.

[MCC77] McCall, J., P. Richards, and G. Walters, "Factors in Software Quality," three volumes, NTIS AD-A049-014, 015, 055, November 1977.

[GRA87] Grady, R.B. and D.L. Caswell, Software Metrics: Establishing a Company-Wide Program, Prentice-Hall, 1987.

[ZUL92] Zultner, R., "Quality Function Deployment for Software: Satisfying Customers," American Programmer, February 1992, pp. 28–41.

[BOS91] Bossert, J.L., Quality Function Deployment: A Practitioner's Approach, ASQC Press, 1991.

[DAV95] Davis, M.J., "Process and Product: Dichotomy or Duality," Software Engineering Notes, ACM Press, vol. 20, no. 2, April 1995, pp. 17–18.

[DUN82] Dunn, R. and R. Ullman, Quality Assurance for Computer Software, McGrawHill, 1982.

[SPI92] Spivey, J.M., The Z Notation: A Reference Manual, Prentice-Hall, 1992.

[NAU69] Naur, P. and B. Randall (eds.), Software Engineering: A Report on a Conference Sponsored by the NATO Science Committee, NATO, 1969.

BAETJER, Jr., H., S**oftware as Capital, IEEE Computer Society Press**, 1998,p. 85.

Artigo: Modelos de Ciclo de Vida: Por que precisamos deles no desenvolvimento? Disponível em: http://www.imasters.com.br/artigo/1861. Acesso em 28/01/2001.

Artigo: Introdução à Ciclo de Vida de Software. Disponível em: http://www.cordeiro.pro.br/aulas/engenharia/processoDeSoftware/ciclos.pdf.

Sobre autor



Acesso em 05/02/2011.
[VES81] Veseley, W.E., et al., Fault Tree Handbook, U.S. Nuclear Regulatory Commission, NUREG-0492, January 1981.

Moacir Orsano Pereira

Doutorando em Administração pela a Universidade de San Carlos. Mestre em Administração pela Universidade Federal da Paraíba. Especialista em Análise de Sistemas pela Universidade Estadual do Piauí. Graduado em Bacharelado em Ciência da Computação pela Universidade Federal da Paraíba.

Gilcifran Vieira de Sousa

Mestre em Computação Aplicada (UECE/IFCE) e Especialização em redes de computadores (FATEPI). Sua graduação é em Bacharelado em Sistemas de Informação (FATEPI).





www.uapi.ufpi.br